Basics of Linear Algebra for Machine Learning

Discover the Mathematical Language of Data in Python

Jason Brownlee

MACHINE LEARNING MASTERY



Disclaimer

The information contained within this eBook is strictly for educational purposes. If you wish to apply ideas contained in this eBook, you are taking full responsibility for your actions.
The author has made every effort to ensure the accuracy of the information within this book was correct at time of publication. The author does not assume and hereby disclaims any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from accident, negligence, or any other cause.
No part of this eBook may be reproduced or transmitted in any form or by any means, electronic or mechanical, recording or by any information storage and retrieval system, without written permission from the author.

Acknowledgements

Special thanks to my copy editor Sarah Martin and my technical editors Arun Koshy and Andrei Cheremskoy.

Copyright

Basics of Linear Algebra for Machine Learning
© Copyright 2018 Jason Brownlee. All Rights Reserved.

Edition: v1.1

Contents

Copyright i

Contents ii

Preface iii

I Introduction v

Welcome vi
- Who Is This Book For? vi
- About Your Outcomes vi
- How to Read This Book vii
- About the Book Structure vii
- About Python Code Examples viii
- About Further Reading ix
- About Getting Help ix
- Summary ix

II Foundations 1

1 Introduction to Linear Algebra 2
- 1.1 Tutorial Overview 2
- 1.2 Linear Algebra 2
- 1.3 Numerical Linear Algebra 3
- 1.4 Linear Algebra and Statistics 4
- 1.5 Applications of Linear Algebra 5
- 1.6 Further Reading 5
- 1.7 Summary 6

2 Linear Algebra and Machine Learning 7
- 2.1 Reasons to NOT Learn Linear Algebra 7
- 2.2 Learn Linear Algebra Notation 8
- 2.3 Learn Linear Algebra Arithmetic 8
- 2.4 Learn Linear Algebra for Statistics 9
- 2.5 Learn Matrix Factorization 9

2.6 Learn Linear Least Squares . . . 9
2.7 One More Reason . . . 9
2.8 Summary . . . 10

3 Examples of Linear Algebra in Machine Learning 11
3.1 Overview . . . 11
3.2 Dataset and Data Files . . . 12
3.3 Images and Photographs . . . 12
3.4 One Hot Encoding . . . 13
3.5 Linear Regression . . . 13
3.6 Regularization . . . 13
3.7 Principal Component Analysis . . . 14
3.8 Singular-Value Decomposition . . . 14
3.9 Latent Semantic Analysis . . . 14
3.10 Recommender Systems . . . 15
3.11 Deep Learning . . . 15
3.12 Summary . . . 15

III NumPy 16

4 Introduction to NumPy Arrays 17
4.1 Tutorial Overview . . . 17
4.2 NumPy N-dimensional Array . . . 17
4.3 Functions to Create Arrays . . . 18
4.4 Combining Arrays . . . 19
4.5 Extensions . . . 21
4.6 Further Reading . . . 21
4.7 Summary . . . 22

5 Index, Slice and Reshape NumPy Arrays 23
5.1 Tutorial Overview . . . 23
5.2 From List to Arrays . . . 23
5.3 Array Indexing . . . 25
5.4 Array Slicing . . . 27
5.5 Array Reshaping . . . 30
5.6 Extensions . . . 32
5.7 Further Reading . . . 33
5.8 Summary . . . 33

6 NumPy Array Broadcasting 35
6.1 Tutorial Overview . . . 35
6.2 Limitation with Array Arithmetic . . . 35
6.3 Array Broadcasting . . . 36
6.4 Broadcasting in NumPy . . . 36
6.5 Limitations of Broadcasting . . . 39
6.6 Extensions . . . 41

6.7 Further Reading . . . 41
6.8 Summary . . . 42

IV Matrices 43

7 Vectors and Vector Arithmetic 44
7.1 Tutorial Overview . . . 44
7.2 What is a Vector . . . 44
7.3 Defining a Vector . . . 45
7.4 Vector Arithmetic . . . 45
7.5 Vector Dot Product . . . 49
7.6 Vector-Scalar Multiplication . . . 50
7.7 Extensions . . . 51
7.8 Further Reading . . . 51
7.9 Summary . . . 52

8 Vector Norms 53
8.1 Tutorial Overview . . . 53
8.2 Vector Norm . . . 53
8.3 Vector L^1 Norm . . . 54
8.4 Vector L^2 Norm . . . 55
8.5 Vector Max Norm . . . 55
8.6 Extensions . . . 56
8.7 Further Reading . . . 56
8.8 Summary . . . 57

9 Matrices and Matrix Arithmetic 58
9.1 Tutorial Overview . . . 58
9.2 What is a Matrix . . . 58
9.3 Defining a Matrix . . . 59
9.4 Matrix Arithmetic . . . 59
9.5 Matrix-Matrix Multiplication . . . 64
9.6 Matrix-Vector Multiplication . . . 66
9.7 Matrix-Scalar Multiplication . . . 67
9.8 Extensions . . . 69
9.9 Further Reading . . . 69
9.10 Summary . . . 70

10 Types of Matrices 71
10.1 Tutorial Overview . . . 71
10.2 Square Matrix . . . 72
10.3 Symmetric Matrix . . . 72
10.4 Triangular Matrix . . . 73
10.5 Diagonal Matrix . . . 74
10.6 Identity Matrix . . . 75
10.7 Orthogonal Matrix . . . 76

10.8 Extensions . . . 78
10.9 Further Reading . . . 78
10.10Summary . . . 79

11 Matrix Operations **80**
11.1 Tutorial Overview . . . 80
11.2 Transpose . . . 80
11.3 Inverse . . . 81
11.4 Trace . . . 83
11.5 Determinant . . . 84
11.6 Rank . . . 85
11.7 Extensions . . . 87
11.8 Further Reading . . . 87
11.9 Summary . . . 88

12 Sparse Matrices **90**
12.1 Tutorial Overview . . . 90
12.2 Sparse Matrix . . . 91
12.3 Problems with Sparsity . . . 91
12.4 Sparse Matrices in Machine Learning . . . 92
12.5 Working with Sparse Matrices . . . 93
12.6 Sparse Matrices in Python . . . 94
12.7 Extensions . . . 95
12.8 Further Reading . . . 95
12.9 Summary . . . 96

13 Tensors and Tensor Arithmetic **98**
13.1 Tutorial Overview . . . 98
13.2 What are Tensors . . . 98
13.3 Tensors in Python . . . 99
13.4 Tensor Arithmetic . . . 100
13.5 Tensor Product . . . 104
13.6 Extensions . . . 105
13.7 Further Reading . . . 106
13.8 Summary . . . 107

V Factorization **108**

14 Matrix Decompositions **109**
14.1 Tutorial Overview . . . 109
14.2 What is a Matrix Decomposition . . . 109
14.3 LU Decomposition . . . 110
14.4 QR Decomposition . . . 111
14.5 Cholesky Decomposition . . . 112
14.6 Extensions . . . 114
14.7 Further Reading . . . 114

14.8 Summary . . . 115

15 Eigendecomposition **116**
15.1 Tutorial Overview . . . 116
15.2 Eigendecomposition of a Matrix . . . 117
15.3 Eigenvectors and Eigenvalues . . . 118
15.4 Calculation of Eigendecomposition . . . 118
15.5 Confirm an Eigenvector and Eigenvalue . . . 119
15.6 Reconstruct Matrix . . . 120
15.7 Extensions . . . 121
15.8 Further Reading . . . 121
15.9 Summary . . . 122

16 Singular Value Decomposition **123**
16.1 Tutorial Overview . . . 123
16.2 What is the Singular-Value Decomposition . . . 124
16.3 Calculate Singular-Value Decomposition . . . 124
16.4 Reconstruct Matrix . . . 125
16.5 Pseudoinverse . . . 127
16.6 Dimensionality Reduction . . . 129
16.7 Extensions . . . 131
16.8 Further Reading . . . 132
16.9 Summary . . . 133

VI Statistics **135**

17 Introduction to Multivariate Statistics **136**
17.1 Tutorial Overview . . . 136
17.2 Expected Value and Mean . . . 136
17.3 Variance and Standard Deviation . . . 138
17.4 Covariance and Correlation . . . 141
17.5 Covariance Matrix . . . 143
17.6 Extensions . . . 144
17.7 Further Reading . . . 144
17.8 Summary . . . 146

18 Principal Component Analysis **147**
18.1 Tutorial Overview . . . 147
18.2 What is Principal Component Analysis . . . 147
18.3 Calculate Principal Component Analysis . . . 149
18.4 Principal Component Analysis in scikit-learn . . . 150
18.5 Extensions . . . 151
18.6 Further Reading . . . 151
18.7 API . . . 151
18.8 Articles . . . 151
18.9 Summary . . . 152

19 Linear Regression **153**
19.1 Tutorial Overview . . . 153
19.2 What is Linear Regression . . . 154
19.3 Matrix Formulation of Linear Regression . . . 154
19.4 Linear Regression Dataset . . . 155
19.5 Solve via Inverse . . . 157
19.6 Solve via QR Decomposition . . . 158
19.7 Solve via SVD and Pseudoinverse . . . 160
19.8 Solve via Convenience Function . . . 162
19.9 Extensions . . . 163
19.10 Further Reading . . . 164
19.11 Summary . . . 165

VII Appendix **167**

A Getting Help **168**
A.1 Linear Algebra on Wikipedia . . . 168
A.2 Linear Algebra Textbooks . . . 168
A.3 Linear Algebra University Courses . . . 169
A.4 Linear Algebra Online Courses . . . 170
A.5 NumPy Resources . . . 170
A.6 Ask Questions About Linear Algebra . . . 171
A.7 How to Ask Questions . . . 171
A.8 Contact the Author . . . 171

B How to Setup a Workstation for Python **172**
B.1 Overview . . . 172
B.2 Download Anaconda . . . 172
B.3 Install Anaconda . . . 174
B.4 Start and Update Anaconda . . . 176
B.5 Further Reading . . . 179
B.6 Summary . . . 179

C Linear Algebra Cheat Sheet **180**
C.1 Array Creation . . . 180
C.2 Vectors . . . 180
C.3 Matrices . . . 181
C.4 Types of Matrices . . . 182
C.5 Matrix Operations . . . 183
C.6 Factorization . . . 183
C.7 Statistics . . . 184

D Basic Math Notation **186**
D.1 Tutorial Overview . . . 186
D.2 The Frustration with Math Notation . . . 187
D.3 Arithmetic Notation . . . 187

D.4 Greek Alphabet 189
D.5 Sequence Notation 190
D.6 Set Notation 191
D.7 Other Notation 192
D.8 Tips for Getting More Help 192
D.9 Further Reading 194
D.10 Summary 194

VIII Conclusions **195**

How Far You Have Come **196**

Preface

I wrote this book to help machine learning practitioners, like you, get on top of linear algebra, fast.

Linear Algebra Is Important in Machine Learning

There is no doubt that linear algebra is important in machine learning. Linear algebra is the mathematics of data. It's all vectors and matrices of numbers. Modern statistics is described using the notation of linear algebra and modern statistical methods harness the tools of linear algebra. Modern machine learning methods are described the same way, using the notations and tools drawn directly from linear algebra. Even some classical methods used in the field, such as linear regression via linear least squares and singular-value decomposition, are linear algebra methods, and other methods, such as principal component analysis, were born from the marriage of linear algebra and statistics. To read and understand machine learning, you must be able to read and understand linear algebra.

Practitioners Study Linear Algebra Too Early

If you ask how to get started in machine learning, you will very likely be told to start with linear algebra. We know that knowledge of linear algebra is critically important, but it does not have to be the place to start. Learning linear algebra first, then calculus, probability, statistics, and eventually machine learning theory is a long and slow bottom-up path. A better fit for developers is to start with systematic procedures that get results, and work back to the deeper understanding of theory, using working results as a context. I call this the top-down or results-first approach to machine learning, and linear algebra is not the first step, but perhaps the second or third.

Practitioners Study Too Much Linear Algebra

When practitioners do circle back to study linear algebra, they learn far more of the field than is required for or relevant to machine learning. Linear algebra is a large field of study that has tendrils into engineering, physics and quantum physics. There are also theorems and derivations for nearly everything, most of which will not help you get better skill from or a deeper understanding of your machine learning model. Only a specific subset of linear algebra is required, though you can always go deeper once you have the basics.

Practitioners Study Linear Algebra Wrong

Linear algebra textbooks will teach you linear algebra in the classical university bottom-up approach. This is too slow (and painful) for your needs as a machine learning practitioner. Like learning machine learning itself, take the top-down approach. Rather than starting with theorems and abstract concepts, you can learn the basics of linear algebra in a concrete way with data structures and worked examples of operations on those data structures. It's so much faster. Once you know how operations work, you can circle back and learn how they were derived.

A Better Way

This book was born out of my frustrations at seeing practitioner after practitioner diving into linear algebra textbooks and online courses designed for undergraduate students and giving up. The bottom-up approach is hard, especially if you already have a full time job. Linear algebra is not only important to machine learning, but it is also a lot of fun, or can be if it is approached in the right way. I put together this book to help you see the field the way I see it: as just another set of tools we can harness on our journey toward machine learning mastery.

Jason Brownlee
2018

Part I

Introduction

Welcome

Welcome to *Basics of Linear Algebra for Machine Learning*. Linear algebra is a pillar of machine learning.

The field started to be formalized about 150 years ago, but it was only about 70 years ago that modern linear algebra came into existence. It's a huge field of study that has made an impact on other areas of mathematics, such as statistics, as well as engineering and physics. Thankfully, we don't need to know the breadth and depth of the field of linear algebra in order to improve our understanding and application of machine learning.

I designed this book to teach you step-by-step the basics of linear algebra with concrete and executable examples in Python.

Who Is This Book For?

Before we get started, let's make sure you are in the right place. This book is for developers that may know some applied machine learning. Maybe you know how to work through a predictive modeling problem end-to-end, or at least most of the main steps, with popular tools. The lessons in this book do assume a few things about you, such as:

- You know your way around basic Python for programming.
- You may know some basic NumPy for array manipulation.
- You want to learn linear algebra to deepen your understanding and application of machine learning.

This guide was written in the top-down and results-first machine learning style that you're used to from MachineLearningMastery.com.

About Your Outcomes

This book will teach you the basics of linear algebra that you need to know as a machine learning practitioner. After reading and working through this book, you will know:

- What linear algebra is and why it is relevant and important to machine learning.
- How to create, index, and generally manipulate data in NumPy arrays.
- What a vector is and how to perform vector arithmetic and calculate vector norms.

- What a matrix is and how to perform matrix arithmetic, including matrix multiplication.
- A suite of types of matrices, their properties, and advanced operations involving matrices.
- What a tensor is and how to perform basic tensor arithmetic.
- Matrix factorization methods, including the eigendecomposition and singular-value decomposition.
- How to calculate and interpret basic statistics using the tools of linear algebra.
- How to implement methods using the tools of linear algebra such as principal component analysis and linear least squares regression.

This new basic understanding of linear algebra will impact your practice of machine learning in the following ways:

- Read the linear algebra mathematics in machine learning papers.
- Implement the linear algebra descriptions of machine learning algorithms.
- Describe your machine learning models using the notation and operations of linear algebra.

This book is not a substitute for an undergraduate course in linear algebra or a textbook for such a course, although it could complement to such materials. For a good list of top courses, textbooks, and other resources on linear algebra, see the *Further Reading* section at the end of each tutorial.

How to Read This Book

This book was written to be read linearly, from start to finish. That being said, if you know the basics and need help with a specific notation or operation, then you can flip straight to that section and get started. This book was designed for you to read on your workstation, on the screen, not on a tablet or eReader. My hope is that you have the book open right next to your editor and run the examples as you read about them.

This book is not intended to be read passively or be placed in a folder as a reference text. It is a playbook, a workbook, and a guidebook intended for you to learn by doing and then apply your new understanding with working Python examples. To get the most out of the book, I would recommend playing with the examples in each tutorial. Extend them, break them, then fix them. Try some of the extensions presented at the end of each lesson and let me know how you do.

About the Book Structure

This book was designed around major data structures, operations, and techniques in linear algebra that are directly relevant to machine learning algorithms. There are a lot of things you could learn about linear algebra, from theory to abstract concepts to APIs. My goal is to take

you straight to developing an intuition for the elements you must understand with laser-focused tutorials.

I designed the tutorials to focus on how to get things done with linear algebra. They give you the tools to both rapidly understand and apply each technique or operation. Each of the tutorials are designed to take you about one hour to read through and complete, excluding the extensions and further reading. You can choose to work through the lessons one per day, one per week, or at your own pace. I think momentum is critically important, and this book is intended to be read and used, not to sit idle. I would recommend picking a schedule and sticking to it.

The tutorials are divided into 5 parts:

- **Part 1: Foundation**. Discover a gentle introduction to the field of linear algebra and the relationship it has with the field of machine learning.
- **Part 2: NumPy**. Discover NumPy tutorials that show you how to create, index, slice, and reshape NumPy arrays, the main data structure used in machine learning and the basis for linear algebra examples in this book.
- **Part 3: Matrices**. Discover the key structures for holding and manipulating data in linear algebra in vectors, matrices, and tensors.
- **Part 4: Factorization**. Discover a suite of methods for decomposing a matrix into its constituent elements in order to make numerical operations more efficient and more stable.
- **Part 5: Statistics**. Discover statistics through the lens of linear algebra and its application to principal component analysis and linear regression.

Each part targets a specific learning outcome, and so does each tutorial within each part. This acts as a filter to ensure you are only focused on the things you need to know to get to a specific result and do not get bogged down in the math or near-infinite number of digressions.

The tutorials were not designed to teach you everything there is to know about each of the theories or techniques of linear algebra. They were designed to give you an understanding of how they work, how to use them, and how to interpret the results the fastest way I know how: to learn by doing.

About Python Code Examples

The code examples were carefully designed to demonstrate the purpose of a given lesson. Code examples are complete and standalone. The code for each lesson will run as-is with no code from prior lessons or third parties required beyond the installation of the required packages. A complete working example is presented with each tutorial for you to inspect and copy-and-paste. All source code is also provided with the book and I would recommend running the provided files whenever possible to avoid any copy-paste issues.

The provided code was developed in a text editor and intended to be run on the command line. No special IDE or notebooks are required. If you are using a more advanced development environment and are having trouble, try running the example from the command line instead. All code examples were tested on a POSIX-compatible machine with Python 3.

About Further Reading

Each lesson includes a list of further reading resources. This may include:

- Books and book chapters.
- API documentation.
- Articles and Webpages.

Wherever possible, I try to list and link to the relevant API documentation for key functions used in each lesson so you can learn more about them. I have tried to link to books on Amazon so that you can learn more about them. I don't know everything, and if you discover a good resource related to a given lesson, please let me know so I can update the book.

About Getting Help

You might need help along the way. Don't worry; you are not alone.

- **Help with a Technique?** If you need help with the technical aspects of a specific operation or technique, see the *Further Reading* sections at the end of each lesson.
- **Help with NumPy?** If you need help with using the NumPy library, see the list of resources in the *Further Reading* section at the end of each lesson, and also see *Appendix A*.
- **Help with your workstation?** If you need help setting up your environment, I would recommend using Anaconda and following my tutorial in *Appendix B*.
- **Help with the math?** I provided a list of locations where you can search for answers and ask questions about linear algebra math in *Appendix A*. You can also see *Appendix D* for a crash course on math notation.
- **Help in general?** You can shoot me an email. My details are in *Appendix A*.

Summary

Are you ready? Let's dive in!

Next

Next up you will discover a gentle introduction to the field of linear algebra.

Part II

Foundations

Chapter 1

Introduction to Linear Algebra

Linear algebra is a field of mathematics that is universally agreed to be a prerequisite to a deeper understanding of machine learning. Although linear algebra is a large field with many esoteric theories and findings, the nuts and bolts tools and notations taken from the field are practical for machine learning practitioners. With a solid foundation of what linear algebra is, it is possible to focus on just the good or relevant parts. In this tutorial, you will discover what exactly linear algebra is from a machine learning perspective. After completing this tutorial, you will know:

- Linear algebra is the mathematics of data.
- Linear algebra has had a marked impact on the field of statistics.
- Linear algebra underlies many practical mathematical tools, such as Fourier series and computer graphics.

Let's get started.

1.1 Tutorial Overview

This tutorial is divided into 4 parts; they are:

1. Linear Algebra.
2. Numerical Linear Algebra.
3. Linear Algebra and Statistics.
4. Applications of Linear Algebra.

1.2 Linear Algebra

Linear algebra is a branch of mathematics, but the truth of it is that linear algebra is the mathematics of data. Matrices and vectors are the language of data. Linear algebra is about linear combinations. That is, using arithmetic on columns of numbers called vectors and arrays

of numbers called matrices, to create new columns and arrays of numbers. Linear algebra is the study of lines and planes, vector spaces and mappings that are required for linear transforms.

It is a relatively young field of study, having initially been formalized in the 1800s in order to find unknowns in systems of linear equations. A linear equation is just a series of terms and mathematical operations where some terms are unknown; for example:

$$y = 4 \times x + 1 \tag{1.1}$$

Equations like this are linear in that they describe a line on a two-dimensional graph. The line comes from plugging in different values into the unknown x to find out what the equation or model does to the value of y. We can line up a system of equations with the same form with two or more unknowns; for example:

$$\begin{aligned} y &= 0.1 \times x_1 + 0.4 \times x_2 \\ y &= 0.3 \times x_1 + 0.9 \times x_2 \\ y &= 0.2 \times x_1 + 0.3 \times x_2 \\ &\dots \end{aligned} \tag{1.2}$$

The column of y values can be taken as a column vector of outputs from the equation. The two columns of integer values are the data columns, say a_1 and a_2, and can be taken as a matrix A. The two unknown values x_1 and x_2 can be taken as the coefficients of the equation and together form a vector of unknowns b to be solved. We can write this compactly using linear algebra notation as:

$$y = A \cdot b \tag{1.3}$$

Problems of this form are generally challenging to solve because there are more unknowns (here we have 2) than there are equations to solve (here we have 3). Further, there is often no single line that can satisfy all of the equations without error. Systems describing problems we are often interested in (such as a linear regression) can have an infinite number of solutions. This gives a small taste of the very core of linear algebra that interests us as machine learning practitioners. Much of the rest of the operations are about making this problem and problems like it easier to understand and solve.

1.3 Numerical Linear Algebra

The application of linear algebra in computers is often called numerical linear algebra.

> "numerical" linear algebra is really applied linear algebra.
>
> — Page ix, *Numerical Linear Algebra*, 1997.

It is more than just the implementation of linear algebra operations in code libraries; it also includes the careful handling of the problems of applied mathematics, such as working with the limited floating point precision of digital computers. Computers are good at performing linear algebra calculations, and much of the dependence on Graphical Processing Units (GPUs) by modern machine learning methods such as deep learning is because of their ability to compute linear algebra operations fast.

Efficient implementations of vector and matrix operations were originally implemented in the FORTRAN programming language in the 1970s and 1980s and a lot of code, or code ported from those implementations, underlies much of the linear algebra performed using modern programming languages, such as Python. Three popular open source numerical linear algebra libraries that implement these functions are:

- Linear Algebra Package, or LAPACK.
- Basic Linear Algebra Subprograms, or BLAS (a standard for linear algebra libraries).
- Automatically Tuned Linear Algebra Software, or ATLAS.

Often, when you are calculating linear algebra operations directly or indirectly via higher-order algorithms, your code is very likely dipping down to use one of these, or similar linear algebra libraries. The name of one of more of these underlying libraries may be familiar to you if you have installed or compiled any of Python's numerical libraries such as SciPy and NumPy.

1.4 Linear Algebra and Statistics

Linear algebra is a valuable tool in other branches of mathematics, especially statistics.

> Usually students studying statistics are expected to have seen at least one semester of linear algebra (or applied algebra) at the undergraduate level.
>
> — Page xv, *Linear Algebra and Matrix Analysis for Statistics*, 2014.

The impact of linear algebra is important to consider, given the foundational relationship both fields have with the field of applied machine learning. Some clear fingerprints of linear algebra on statistics and statistical methods include:

- Use of vector and matrix notation, especially with multivariate statistics.
- Solutions to least squares and weighted least squares, such as for linear regression.
- Estimates of mean and variance of data matrices.
- The covariance matrix that plays a key role in multinomial Gaussian distributions.
- Principal component analysis for data reduction that draws many of these elements together.

As you can see, modern statistics and data analysis, at least as far as the interests of a machine learning practitioner are concerned, depend on the understanding and tools of linear algebra.

1.5 Applications of Linear Algebra

As linear algebra is the mathematics of data, the tools of linear algebra are used in many domains. In his classical book on the topic titled *Introduction to Linear Algebra*, Gilbert Strang provides a chapter dedicated to the applications of linear algebra. In it, he demonstrates specific mathematical tools rooted in linear algebra. Briefly they are:

- Matrices in Engineering, such as a line of springs.
- Graphs and Networks, such as analyzing networks.
- Markov Matrices, Population, and Economics, such as population growth.
- Linear Programming, the simplex optimization method.
- Fourier Series: Linear Algebra for functions, used widely in signal processing.
- Linear Algebra for statistics and probability, such as least squares for regression.
- Computer Graphics, such as the various translation, rescaling and rotation of images.

Another interesting application of linear algebra is that it is the type of mathematics used by Albert Einstein in parts of his theory of relativity. Specifically tensors and tensor calculus. He also introduced a new type of linear algebra notation to physics called Einstein notation, or the Einstein summation convention.

1.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

1.6.1 Books

- *Introduction to Linear Algebra*, 2016.
  http://amzn.to/2j2J0g4
- *Numerical Linear Algebra*, 1997.
  http://amzn.to/2kjEF4S
- *Linear Algebra and Matrix Analysis for Statistics*, 2014.
  http://amzn.to/2A9ceNv

1.6.2 Articles

- Linear Algebra on Wikipedia.
  https://en.wikipedia.org/wiki/Linear_algebra
- Linear Algebra Category on Wikipedia.
  https://en.wikipedia.org/wiki/Category:Linear_algebra

- Linear Algebra List of Topics on Wikipedia.
https://en.wikipedia.org/wiki/List_of_linear_algebra_topics
- LAPACK on Wikipedia.
https://en.wikipedia.org/wiki/LAPACK
- Basic Linear Algebra Subprograms on Wikipedia.
https://en.wikipedia.org/wiki/Basic_Linear_Algebra_Subprograms
- Automatically Tuned Linear Algebra Software on Wikipedia.
https://en.wikipedia.org/wiki/Automatically_Tuned_Linear_Algebra_Software
- Einstein notation on Wikipedia.
https://en.wikipedia.org/wiki/Einstein_notation
- Mathematics of general relativity on Wikipedia.
https://en.wikipedia.org/wiki/Mathematics_of_general_relativity

1.7 Summary

In this tutorial, you discovered a gentle introduction to linear algebra from a machine learning perspective. Specifically, you learned:

- Linear algebra is the mathematics of data.
- Linear algebra has had a marked impact on the field of statistics.
- Linear algebra underlies many practical mathematical tools, such as Fourier series and computer graphics.

1.7.1 Next

In the next chapter you will discover why linear algebra is important to machine learning.

Chapter 2

Linear Algebra and Machine Learning

Linear algebra is a field of mathematics that could be called the mathematics of data. It is undeniably a pillar of the field of machine learning, and many recommend it as a prerequisite subject to study prior to getting started in machine learning. This is misleading advice, as linear algebra makes more sense to a practitioner once they have a context of the applied machine learning process in which to interpret it. In this chapter, you will discover why machine learning practitioners should study linear algebra to improve their skills and capabilities as practitioners. After reading this chapter, you will know:

- Not everyone should learn linear algebra, that it depends where you are in your process of learning machine learning.
- 5 Reasons why a deeper understanding of linear algebra is required for intermediate machine learning practitioners.
- That linear algebra can be fun if approached in the right way.

Let's get started.

2.1 Reasons to NOT Learn Linear Algebra

Before we go through the reasons that you should learn linear algebra, let's start off by taking a small look at the reason why you should not. I think you should not study linear algebra if you are just getting started with applied machine learning.

- **It's not required**. Having an appreciation for the abstract operations that underly some machine learning algorithms is not required in order to use machine learning as a tool to solve problems.
- **It's slow**. Taking months to years to study an entire related field before machine learning will delay you achieving your goals of being able to work through predictive modeling problems.
- **It's a huge field**. Not all of linear algebra is relevant to theoretical machine learning, let alone applied machine learning.

I recommend a breadth-first approach to getting started in applied machine learning. I call this approach a results-first approach. It is where you start by learning and practicing the steps for working through a predictive modeling problem end-to-end (e.g. how to get results) with a tool (such as scikit-learn and Pandas in Python). This process then provides the skeleton and context for progressively deepening your knowledge, such as how algorithms work and eventually the math that underlies them. After you know how to work through a predictive modeling problem, let's look at why you should deepen your understanding of linear algebra.

> Linear algebra is a branch of mathematics that is widely used throughout science and engineering. However, because linear algebra is a form of continuous rather than discrete mathematics, many computer scientists have little experience with it.
>
> — Page 31, *Deep Learning*, 2016.

2.2 Learn Linear Algebra Notation

You need to be able to read and write vector and matrix notation. Algorithms are described in books, papers and on websites using vector and matrix notation. Linear algebra is the mathematics of data and the notation allows you to describe operations on data precisely with specific operators. You need to be able to read and write this notation. This skill will allow you to:

- Read descriptions of existing algorithms in textbooks.
- Interpret and implement descriptions of new methods in research papers.
- Concisely describe your own methods to other practitioners.

Further, programming languages such as Python offer efficient ways of implementing linear algebra notation directly. An understanding of the notation and how it is realized in your language or library will allow for shorter and perhaps more efficient implementations of machine learning algorithms.

2.3 Learn Linear Algebra Arithmetic

In partnership with the notation of linear algebra are the arithmetic operations performed. You need to know how to add, subtract, and multiply scalars, vectors, and matrices. A challenge for newcomers to the field of linear algebra are operations such as matrix multiplication and tensor multiplication that are not implemented as the direct multiplication of the elements of these structures, and at first glance appear nonintuitive.

Again, most if not all of these operations are implemented efficiently and provided via API calls in modern linear algebra libraries. An understanding of how vector and matrix operations are implemented is required as a part of being able to effectively read and write matrix notation.

2.4 Learn Linear Algebra for Statistics

You must learn linear algebra in order to be able to learn statistics. Especially multivariate statistics. Statistics and data analysis are another pillar field of mathematics to support machine learning. They are primarily concerned with describing and understanding data. As the mathematics of data, linear algebra has left its fingerprint on many related fields of mathematics, including statistics.

In order to be able to read and interpret statistics, you must learn the notation and operations of linear algebra. Modern statistics uses both the notation and tools of linear algebra to describe the tools and techniques of statistical methods. From vectors for the means and variances of data, to covariance matrices that describe the relationships between multiple Gaussian variables. The results of some collaborations between the two fields are also staple machine learning methods, such as the Principal Component Analysis, or PCA for short, used for data reduction.

2.5 Learn Matrix Factorization

Building on notation and arithmetic is the idea of matrix factorization, also called matrix decomposition. You need to know how to factorize a matrix and what it means. Matrix factorization is a key tool in linear algebra and used widely as an element of many more complex operations in both linear algebra (such as the matrix inverse) and machine learning (least squares).

Further, there are a range of different matrix factorization methods, each with different strengths and capabilities, some of which you may recognize as "machine learning" methods, such as Singular-Value Decomposition, or SVD for short, for data reduction. In order to read and interpret higher-order matrix operations, you must understand matrix factorization.

2.6 Learn Linear Least Squares

You need to know how to use matrix factorization to solve linear least squares. Linear algebra was originally developed to solve systems of linear equations. These are equations where there are more equations than there are unknown variables. As a result, they are challenging to solve arithmetically because there is no single solution as there is no line or plane can fit the data without some error. Problems of this type can be framed as the minimization of squared error, called least squares, and can be recast in the language of linear algebra, called linear least squares.

Linear least squares problems can be solved efficiently on computers using matrix operations such as matrix factorization. Least squares is most known for its role in the solution to linear regression models, but also plays a wider role in a range of machine learning algorithms. In order to understand and interpret these algorithms, you must understand how to use matrix factorization methods to solve least squares problems.

2.7 One More Reason

If I could give one more reason, it would be: because it is fun. Seriously. Learning linear algebra, at least the way I teach it with practical examples and executable code, is a lot of fun. Once you

can see how the operations work on real data, it is hard to avoid developing a strong intuition for the methods. I am not alone in thinking that linear algebra can be fun if approached in the right way:

> Learning linear algebra can also be a lot of fun. Readers will experience *knowledge buzz* as they learn about the connections between concepts, and it's not uncommon to experience mind-expanding moments while studying this subject.
>
> — Page ix, *No Bullshit Guide To Linear Algebra*, 2017.

Why do you want to learn linear algebra? Let me know.

2.8 Summary

In this chapter, you discovered why, as a machine learning practitioner, you should deepen your understanding of linear algebra. Specifically, you learned:

- Not everyone should learn linear algebra, that it depends where you are in your process of learning machine learning.
- 5 Reasons why a deeper understanding of linear algebra is required for intermediate machine learning practitioners.
- That linear algebra can be fun if approached in the right way.

2.8.1 Next

In the next chapter you will discover 10 concrete examples of machine learning concepts and methods that require an understanding of linear algebra.

Chapter 3

Examples of Linear Algebra in Machine Learning

Linear algebra is a sub-field of mathematics concerned with vectors, matrices and linear transforms. It is a key foundation to the field of machine learning from notations used to describe the operation of algorithms, to the implementation of algorithms in code. Although linear algebra is integral to the field of machine learning, the tight relationship is often left unexplained or explained using abstract concepts such as vector spaces or specific matrix operations. In this chapter, you will discover 10 common examples of machine learning that you may be familiar with that use, require and are really best understood using linear algebra. After reading this chapter, you will know:

- The use if linear algebra structures when working with data such as tabular datasets and images.
- Linear algebra concepts when working with data preparation such as one hot encoding and dimensionality reduction.
- The in-grained use of linear algebra notation and methods in sub-fields such as deep learning, natural language processing and recommender systems.

Let's get started.

3.1 Overview

In this chapter, we will review 10 obvious and concrete examples of linear algebra in machine learning. I tried to pick examples that you may be familiar with or have even worked with before. They are:

1. Dataset and Data Files
2. Images and Photographs
3. One Hot Encoding
4. Linear Regression

5. Regularization
6. Principal Component Analysis
7. Singular-Value Decomposition
8. Latent Semantic Analysis
9. Recommender Systems
10. Deep Learning

Do you have your own favorite example of linear algebra in machine learning? Let me know.

3.2 Dataset and Data Files

In machine learning, you fit a model on a dataset. This is the table like set of numbers where each row represents an observation and each column represents a feature of the observation. For example, below is a snippet of the Iris flowers dataset[1]:

```
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
...
```

Listing 3.1: Sample output of the iris flowers dataset.

This data is in fact a matrix, a key data structure in linear algebra. Further, when you split the data into inputs and outputs to fit a supervised machine learning model, such as the measurements and the flower species, you have a matrix (X) and a vector (y). The vector is another key data structure in linear algebra. Each row has the same length, i.e. the same number of columns, therefore we can say that the data is vectorized where rows can be provided to a model one at a time or in batch and the model can be pre-configured to expect rows of a fixed width.

3.3 Images and Photographs

Perhaps you are more used to working with images or photographs in computer vision applications. Each image that you work with is itself a table structure with a width and height and one pixel value in each cell for black and white images or 3 pixel values in each cell for a color image. A photo is yet another example of a matrix from linear algebra. Operations on the image, such as cropping, scaling, shearing and so on are all described using the notation and operations of linear algebra.

[1] http://archive.ics.uci.edu/ml/datasets/Iris

3.4 One Hot Encoding

Sometimes you work with categorical data in machine learning. Perhaps the class labels for classification problems, or perhaps categorical input variables. It is common to encode categorical variables to make their easier to work with and learn by some techniques. A popular encoding for categorical variables is the one hot encoding. A one hot encoding is where a table is created to represent the variable with one column for each category and a row for each example in the dataset. A check or one-value is added in the column for the categorical value for a given row, and a zero-value is added to all other columns. For example, the variable color variable with the 3 rows:

```
red
green
blue
...
```

Listing 3.2: Example of a categorical variable.

Might be encoded as:

```
red, green, blue
  1,    0,    0
  0,    1,    0
  0,    0,    1
...
```

Listing 3.3: Example of a one hot encoded categorical variable.

Each row is encoded as a binary vector, a vector with zero or one values and this is an example of a sparse representation, a whole sub-field of linear algebra.

3.5 Linear Regression

Linear regression is an old method from statistics for describing the relationships between variables. It is often used in machine learning for predicting numerical values in simpler regression problems. There are many ways to describe and solve the linear regression problem, i.e. finding a set of coefficients that when multiplied by each of the input variables and added together results in the best prediction of the output variable. If you have used a machine learning tool or library, the most common way of solving linear regression is via a least squares optimization that is solved using matrix factorization methods from linear regression, such as an LU decomposition or an singular-value decomposition or SVD. Even the common way of summarizing the linear regression equation uses linear algebra notation:

$$y = A \cdot b \tag{3.1}$$

Where y is the output variable A is the dataset and b are the model coefficients.

3.6 Regularization

In applied machine learning, we often seek the simplest possible models that achieve the best skill on our problem. Simpler models are often better at generalizing from specific examples

to unseen data. In many methods that involve coefficients, such as regression methods and artificial neural networks, simpler models are often characterized by models that have smaller coefficient values. A technique that is often used to encourage a model to minimize the size of coefficients while it is being fit on data is called regularization. Common implementations include the L^2 and L^1 forms of regularization. Both of these forms of regularization are in fact a measure of the magnitude or length of the coefficients as a vector and are methods lifted directly from linear algebra called the vector norm.

3.7 Principal Component Analysis

Often a dataset has many columns, perhaps tens, hundreds, thousands or more. Modeling data with many features is challenging, and models built from data that include irrelevant features are often less skillful than models trained from the most relevant data. It is hard to know which features of the data are relevant and which are not. Methods for automatically reducing the number of columns of a dataset are called dimensionality reduction, and perhaps the most popular is method is called the principal component analysis or PCA for short. This method is used in machine learning to create projections of high-dimensional data for both visualization and for training models. The core of the PCA method is a matrix factorization method from linear algebra. The eigendecomposition can be used and more robust implementations may use the singular-value decomposition or SVD.

3.8 Singular-Value Decomposition

Another popular dimensionality reduction method is the singular-value decomposition method or SVD for short. As mentioned and as the name of the method suggests, it is a matrix factorization method from the field of linear algebra. It has wide use in linear algebra and can be used directly in applications such as feature selection, visualization, noise reduction and more. We will see two more cases below of using the SVD in machine learning.

3.9 Latent Semantic Analysis

In the sub-field of machine learning for working with text data called natural language processing, it is common to represent documents as large matrices of word occurrences. For example, the columns of the matrix may be the known words in the vocabulary and rows may be sentences, paragraphs, pages or documents of text with cells in the matrix marked as the count or frequency of the number of times the word occurred. This is a sparse matrix representation of the text. Matrix factorization methods such as the singular-value decomposition can be applied to this sparse matrix which has the effect of distilling the representation down to its most relevant essence. Documents processed in thus way are much easier to compare, query and use as the basis for a supervised machine learning model. This form of data preparation is called Latent Semantic Analysis or LSA for short, and is also known by the name Latent Semantic Indexing or LSI.

3.10 Recommender Systems

Predictive modeling problems that involve the recommendation of products are called recommender systems, a sub-field of machine learning. Examples include the recommendation of books based on previous purchases and purchases by customers like you on Amazon, and the recommendation of movies and TV shows to watch based on your viewing history and viewing history of subscribers like you on Netflix. The development of recommender systems is primarily concerned with linear algebra methods. A simple example is in the calculation of the similarity between sparse customer behavior vectors using distance measures such as Euclidean distance or dot products. Matrix factorization methods like the singular-value decomposition are used widely in recommender systems to distill item and user data to their essence for querying and searching and comparison.

3.11 Deep Learning

Artificial neural networks are nonlinear machine learning algorithms that are inspired by elements of the information processing in the brain and have proven effective at a range of problems not least predictive modeling. Deep learning is the recent resurged use of artificial neural networks with newer methods and faster hardware that allow for the development and training of larger and deeper (more layers) networks on very large datasets. Deep learning methods are routinely achieve state-of-the-art results on a range of challenging problems such as machine translation, photo captioning, speech recognition and much more.

At their core, the execution of neural networks involves linear algebra data structures multiplied and added together. Scaled up to multiple dimensions, deep learning methods work with vectors, matrices and even tensors of inputs and coefficients, where a tensor is a matrix with more than two dimensions. Linear algebra is central to the description of deep learning methods via matrix notation to the implementation of deep learning methods such as Google's TensorFlow Python library that has the word "tensor" in its name.

3.12 Summary

In this chapter, you discovered 10 common examples of machine learning that you may be familiar with that use and require linear algebra. Specifically, you learned:

- The use of linear algebra structures when working with data such as tabular datasets and images.
- Linear algebra concepts when working with data preparation such as one hot encoding and dimensionality reduction.
- The in-grained use of linear algebra notation and methods in sub-fields such as deep learning, natural language processing and recommender systems.

3.12.1 Next

This is the end of the first part, in the next part you will discover how to manipulate arrays of data in Python using NumPy.

Part III

NumPy

Chapter 4

Introduction to NumPy Arrays

Arrays are the main data structure used in machine learning. In Python, arrays from the NumPy library, called N-dimensional arrays or the `ndarray`, are used as the primary data structure for representing data. In this tutorial, you will discover the N-dimensional array in NumPy for representing numerical and manipulating data in Python. After completing this tutorial, you will know:

- What the `ndarray` is and how to create and inspect an array in Python.
- Key functions for creating new empty arrays and arrays with default values.
- How to combine existing arrays to create new arrays.

Let's get started.

4.1 Tutorial Overview

This tutorial is divided into 3 parts; they are:

1. NumPy N-dimensional Array
2. Functions to Create Arrays
3. Combining Arrays

4.2 NumPy N-dimensional Array

NumPy is a Python library that can be used for scientific and numerical applications and is the tool to use for linear algebra operations. The main data structure in NumPy is the `ndarray`, which is a shorthand name for N-dimensional array. When working with NumPy, data in an `ndarray` is simply referred to as an array. It is a fixed-sized array in memory that contains data of the same type, such as integers or floating point values.

The data type supported by an array can be accessed via the `dtype` attribute on the array. The dimensions of an array can be accessed via the `shape` attribute that returns a tuple describing the length of each dimension. There are a host of other attributes. A simple way

to create an array from data or simple Python data structures like a list is to use the `array()` function. The example below creates a Python list of 3 floating point values, then creates an `ndarray` from the list and access the arrays' shape and data type.

```
# create array
from numpy import array
# create array
l = [1.0, 2.0, 3.0]
a = array(l)
# display array
print(a)
# display array shape
print(a.shape)
# display array data type
print(a.dtype)
```

Listing 4.1: Example of creating an array with the `array()` function.

Running the example prints the contents of the `ndarray`, the shape, which is a one-dimensional array with 3 elements, and the data type, which is a 64-bit floating point.

```
[ 1.  2.  3.]
(3,)
float64
```

Listing 4.2: Sample output of creating an array with the `array()` function.

4.3 Functions to Create Arrays

There are more convenience functions for creating fixed-sized arrays that you may encounter or be required to use. Let's look at just a few.

4.3.1 Empty

The `empty()` function will create a new array of the specified shape. The argument to the function is an array or tuple that specifies the length of each dimension of the array to create. The values or content of the created array will be random and will need to be assigned before use. The example below creates an empty 3×3 two-dimensional array.

```
# create empty array
from numpy import empty
a = empty([3,3])
print(a)
```

Listing 4.3: Example of creating an array with the `empty()` function.

Running the example prints the content of the empty array. Your specific array contents will vary.

```
[[ 0.00000000e+000 0.00000000e+000 2.20802703e-314]
 [ 2.20803350e-314 2.20803353e-314 2.20803356e-314]
 [ 2.20803359e-314 2.20803362e-314 2.20803366e-314]]
```

Listing 4.4: Sample output of creating an array with the `empty()` function.

4.3.2 Zeros

The `zeros()` function will create a new array of the specified size with the contents filled with zero values. The argument to the function is an array or tuple that specifies the length of each dimension of the array to create. The example below creates a 3×5 zero two-dimensional array.

```
# create zero array
from numpy import zeros
a = zeros([3,5])
print(a)
```

Listing 4.5: Example of creating an array with the `zeros()` function.

Running the example prints the contents of the created zero array.

```
[[ 0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.]]
```

Listing 4.6: Sample output of creating an array with the `zeros()` function.

4.3.3 Ones

The `ones()` function will create a new array of the specified size with the contents filled with one values. The argument to the function is an array or tuple that specifies the length of each dimension of the array to create. The example below creates a 5-element one-dimensional array.

```
# create one array
from numpy import ones
a = ones([5])
print(a)
```

Listing 4.7: Example of creating an array with the `ones()` function.

Running the example prints the contents of the created ones array.

```
[ 1.  1.  1.  1.  1.]
```

Listing 4.8: Sample output of creating an array with the `ones()` function.

4.4 Combining Arrays

NumPy provides many functions to create new arrays from existing arrays. Let's look at two of the most popular functions you may need or encounter.

4.4.1 Vertical Stack

Given two or more existing arrays, you can stack them vertically using the `vstack()` function. For example, given two one-dimensional arrays, you can create a new two-dimensional array with two rows by vertically stacking them. This is demonstrated in the example below.

```
# create array with vstack
from numpy import array
from numpy import vstack
# create first array
a1 = array([1,2,3])
print(a1)
# create second array
a2 = array([4,5,6])
print(a2)
# vertical stack
a3 = vstack((a1, a2))
print(a3)
print(a3.shape)
```

Listing 4.9: Example of creating an array from other arrays using the `vstack()` function.

Running the example first prints the two separately defined one-dimensional arrays. The arrays are vertically stacked resulting in a new 2×3 array, the contents and shape of which are printed.

```
[1 2 3]

[4 5 6]

[[1 2 3]
 [4 5 6]]

(2, 3)
```

Listing 4.10: Sample output of creating an array from other arrays with the `vstack()` function.

4.4.2 Horizontal Stack

Given two or more existing arrays, you can stack them horizontally using the `hstack()` function. For example, given two one-dimensional arrays, you can create a new one-dimensional array or one row with the columns of the first and second arrays concatenated. This is demonstrated in the example below.

```
# create array with hstack
from numpy import array
from numpy import hstack
# create first array
a1 = array([1,2,3])
print(a1)
# create second array
a2 = array([4,5,6])
print(a2)
# create horizontal stack
a3 = hstack((a1, a2))
print(a3)
print(a3.shape)
```

Listing 4.11: Example of creating an array from other arrays using the `hstack()` function.

Running the example first prints the two separately defined one-dimensional arrays. The arrays are then horizontally stacked resulting in a new one-dimensional array with 6 elements, the contents and shape of which are printed.

```
[1 2 3]

[4 5 6]

[1 2 3 4 5 6]

(6,)
```

Listing 4.12: Sample output of creating an array from other arrays with the `hstack()` function.

4.5 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Experiment with the different ways of creating arrays to your own sizes or with new data.
- Locate and develop an example for 3 additional NumPy functions for creating arrays.
- Locate and develop an example for 3 additional NumPy functions for combining arrays.

If you explore any of these extensions, I'd love to know.

4.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

4.6.1 Books

- *Python for Data Analysis*, 2017.
  http://amzn.to/2B1sfXi
- *Elegant SciPy*, 2017.
  http://amzn.to/2yujXnT
- *Guide to NumPy*, 2015.
  http://amzn.to/2j3kEzd

4.6.2 References

- NumPy Reference.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/
- The N-dimensional array.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/arrays.ndarray.html
- Array creation routines.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/routines.array-creation.html

4.6.3 API

- `numpy.array()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.array.html
- `numpy.empty()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.empty.html
- `numpy.zeros()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.zeros.html
- `numpy.ones()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.ones.html
- `numpy.vstack()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.vstack.html
- `numpy.hstack()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.hstack.html

4.7 Summary

In this tutorial, you discovered the N-dimensional array in NumPy for representing numerical and manipulating data in Python. Specifically, you learned:

- What the `ndarray` is and how to create and inspect an array in Python.
- Key functions for creating new empty arrays and arrays with default values.
- How to combine existing arrays to create new arrays.

4.7.1 Next

In the next chapter you will discover how to slice, index, and reshape NumPy arrays.

Chapter 5

Index, Slice and Reshape NumPy Arrays

Machine learning data is represented as arrays. In Python, data is almost universally represented as NumPy arrays. If you are new to Python, you may be confused by some of the Pythonic ways of accessing data, such as negative indexing and array slicing. In this tutorial, you will discover how to manipulate and access your data correctly in NumPy arrays. After completing this tutorial, you will know:

- How to convert your list data to NumPy arrays.
- How to access data using Pythonic indexing and slicing.
- How to resize your data to meet the expectations of some machine learning APIs.

Let's get started.

5.1 Tutorial Overview

This tutorial is divided into 4 parts; they are:

1. From List to Arrays
2. Array Indexing
3. Array Slicing
4. Array Reshaping

5.2 From List to Arrays

In general, I recommend loading your data from file using Pandas or even NumPy functions. This section assumes you have loaded or generated your data by other means and it is now represented using Python lists. Let's look at converting your data in lists to NumPy arrays.

5.2.1 One-Dimensional List to Array

You may load your data or generate your data and have access to it as a list. You can convert a one-dimensional list of data to an array by calling the `array()` NumPy function.

```
# create one-dimensional array
from numpy import array
# list of data
data = [11, 22, 33, 44, 55]
# array of data
data = array(data)
print(data)
print(type(data))
```

Listing 5.1: Example of creating one-dimensional array.

Running the example converts the one-dimensional list to a NumPy array.

```
[11 22 33 44 55]
<class 'numpy.ndarray'>
```

Listing 5.2: Sample output of creating a one-dimensional array.

5.2.2 Two-Dimensional List of Lists to Array

It is more likely in machine learning that you will have two-dimensional data. That is a table of data where each row represents a new observation and each column a new feature. Perhaps you generated the data or loaded it using custom code and now you have a list of lists. Each list represents a new observation. You can convert your list of lists to a NumPy array the same way as above, by calling the `array()` function.

```
# create two-dimensional array
from numpy import array
# list of data
data = [[11, 22],
    [33, 44],
    [55, 66]]
# array of data
data = array(data)
print(data)
print(type(data))
```

Listing 5.3: Example of creating two-dimensional array.

Running the example shows the data successfully converted.

```
[[11 22]
 [33 44]
 [55 66]]
<class 'numpy.ndarray'>
```

Listing 5.4: Sample output of creating a two-dimensional array.

5.3 Array Indexing

Once your data is represented using a NumPy array, you can access it using indexing. Let's look at some examples of accessing data via indexing.

5.3.1 One-Dimensional Indexing

Generally, indexing works just like you would expect from your experience with other programming languages, like Java, C#, and C++. For example, you can access elements using the bracket operator [] specifying the zero-offset index for the value to retrieve.

```
# index a one-dimensional array
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
# index data
print(data[0])
print(data[4])
```

Listing 5.5: Example of indexing a one-dimensional array.

Running the example prints the first and last values in the array.

```
11
55
```

Listing 5.6: Sample output from indexing a one-dimensional array.

Specifying integers too large for the bound of the array will cause an error.

```
# index array out of bounds
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
# index data
print(data[5])
```

Listing 5.7: Example of an error when indexing a one-dimensional array.

Running the example prints the following error:

```
IndexError: index 5 is out of bounds for axis 0 with size 5
```

Listing 5.8: Sample error output from indexing a one-dimensional array.

One key difference is that you can use negative indexes to retrieve values offset from the end of the array. For example, the index -1 refers to the last item in the array. The index -2 returns the second last item all the way back to -5 for the first item in the current example.

```
# negative array indexing
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
# index data
print(data[-1])
print(data[-5])
```

Listing 5.9: Example of negative indexing a one-dimensional array.

Running the example prints the last and first items in the array.

```
55
11
```

Listing 5.10: Sample output from negative indexing a one-dimensional array.

5.3.2 Two-Dimensional Indexing

Indexing two-dimensional data is similar to indexing one-dimensional data, except that a comma is used to separate the index for each dimension.

```
data[0,0]
```

Listing 5.11: Example of indexing a two-dimensional array in Python.

This is different from C-based languages where a separate bracket operator is used for each dimension.

```
data[0][0]
```

Listing 5.12: Example of indexing a two-dimensional array in C-like languages.

For example, we can access the first row and the first column as follows:

```
# index two-dimensional array
from numpy import array
# define array
data = array([
  [11, 22],
  [33, 44],
  [55, 66]])
# index data
print(data[0,0])
```

Listing 5.13: Example of indexing a two-dimensional array.

Running the example prints the first item in the dataset.

```
11
```

Listing 5.14: Sample output from indexing a two-dimensional array.

If we are interested in all items in the first row, we could leave the second dimension index empty, for example:

```
# index row of two-dimensional array
from numpy import array
# define array
data = array([
  [11, 22],
  [33, 44],
  [55, 66]])
# index data
print(data[0,])
```

Listing 5.15: Example of indexing the first column of a two-dimensional array.

This prints the first row of data.

```
[11 22]
```

Listing 5.16: Sample output from indexing the first column of a two-dimensional array.

5.4 Array Slicing

So far, so good; creating and indexing arrays looks familiar. Now we come to array slicing, and this is one feature that causes problems for beginners to Python and NumPy arrays. Structures like lists and NumPy arrays can be sliced. This means that a subsequence of the structure can be indexed and retrieved. This is most useful in machine learning when specifying input variables and output variables, or splitting training rows from testing rows. Slicing is specified using the colon operator : with a *from* and *to* index before and after the column respectively. The slice extends from the *from* index and ends one item before the *to* index.

```
data[from:to]
```

Listing 5.17: Example of the syntax of array slicing.

5.4.1 One-Dimensional Slicing

You can access all data in an array dimension by specifying the slice ':' with no indexes.

```
# slice a one-dimensional array
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
print(data[:])
```

Listing 5.18: Example of slicing a one-dimensional array.

Running the example prints all elements in the array.

```
[11 22 33 44 55]
```

Listing 5.19: Sample output from slicing a one-dimensional array.

The first item of the array can be sliced by specifying a slice that starts at index 0 and ends at index 1 (one item before the *to* index).

```
# slice a subset of a one-dimensional array
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
print(data[0:1])
```

Listing 5.20: Example of slicing a subset of a one-dimensional array.

Running the example returns a sub-array with the first element.

```
[11]
```

Listing 5.21: Sample output from slicing a subset of a one-dimensional array.

We can also use negative indexes in slices. For example, we can slice the last two items in the list by starting the slice at -2 (the second last item) and not specifying a *to* index; that takes the slice to the end of the dimension.

```
# negative slicing of a one-dimensional array
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
print(data[-2:])
```

Listing 5.22: Example of slicing with a negative index on a one-dimensional array.

Running the example returns a sub-array with the last two items only.

```
[44 55]
```

Listing 5.23: Sample output from slicing with a negative index on a one-dimensional array.

5.4.2 Two-Dimensional Slicing

Let's look at the two examples of two-dimensional slicing you are most likely to use in machine learning.

Split Input and Output Features

It is common to split your loaded data into input variables (`X`) and the output variable (`y`). We can do this by slicing all rows and all columns up to, but before the last column, then separately indexing the last column. For the input features, we can select all rows and all columns except the last one by specifying `:` for in the rows index, and `:-1` in the columns index.

```
X = [:, :-1]
```

Listing 5.24: Example of slicing input variables.

For the output column, we can select all rows again using `:` and index just the last column by specifying the -1 index.

```
y = [:, -1]
```

Listing 5.25: Example of slicing the output variable.

Putting all of this together, we can separate a 3-column 2D dataset into input and output data as follows:

```
# split input and output data
from numpy import array
# define array
data = array([
  [11, 22, 33],
  [44, 55, 66],
  [77, 88, 99]])
# separate data
X, y = data[:, :-1], data[:, -1]
print(X)
print(y)
```

Listing 5.26: Example of slicing a dataset into input and output variables.

Running the example prints the separated X and y elements. Note that `X` is a 2D array and `y` is a 1D array.

```
[[11 22]
 [44 55]
 [77 88]]
[33 66 99]
```

Listing 5.27: Sample output slicing a dataset into input and output variables.

Split Train and Test Rows

It is common to split a loaded dataset into separate train and test sets. This is a splitting of rows where some portion will be used to train the model and the remaining portion will be used to estimate the skill of the trained model. This would involve slicing all columns by specifying `:` in the second dimension index. The training dataset would be all rows from the beginning to the split point.

```
train = data[:split, :]
```

Listing 5.28: Example of slicing a train set from a dataset.

The test dataset would be all rows starting from the split point to the end of the dimension.

```
test = data[split:, :]
```

Listing 5.29: Example of slicing a test set from a dataset.

Putting all of this together, we can split the dataset at the contrived split point of 2.

```
# split train and test data
from numpy import array
# define array
data = array([
  [11, 22, 33],
  [44, 55, 66],
  [77, 88, 99]])
# separate data
split = 2
train,test = data[:split,:],data[split:,:]
print(train)
print(test)
```

Listing 5.30: Example of slicing a dataset into train and test subsets.

Running the example selects the first two rows for training and the last row for the test set.

```
[[11 22 33]
[44 55 66]]
[[77 88 99]]
```

Listing 5.31: Sample output slicing a dataset into train and test subsets.

5.5 Array Reshaping

After slicing your data, you may need to reshape it. For example, some libraries, such as scikit-learn, may require that a one-dimensional array of output variables (y) be shaped as a two-dimensional array with one column and outcomes for each column. Some algorithms, like the Long Short-Term Memory recurrent neural network in Keras, require input to be specified as a three-dimensional array comprised of samples, timesteps, and features. It is important to know how to reshape your NumPy arrays so that your data meets the expectation of specific Python libraries. We will look at these two examples.

5.5.1 Data Shape

NumPy arrays have a `shape` attribute that returns a tuple of the length of each dimension of the array. For example:

```
# shape of one-dimensional array
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
print(data.shape)
```

Listing 5.32: Example of accessing `shape` for a one-dimensional array.

Running the example prints a tuple for the one dimension.

```
(5,)
```

Listing 5.33: Sample output `shape` for a one-dimensional array.

A tuple with two lengths is returned for a two-dimensional array.

```
# shape of a two-dimensional array
from numpy import array
# list of data
data = [[11, 22],
    [33, 44],
    [55, 66]]
# array of data
data = array(data)
print(data.shape)
```

Listing 5.34: Example of accessing `shape` for a two-dimensional array.

Running the example returns a tuple with the number of rows and columns.

```
(3, 2)
```

Listing 5.35: Sample output `shape` for a two-dimensional array.

You can use the size of your array dimensions in the `shape` dimension, such as specifying parameters. The elements of the tuple can be accessed just like an array, with the 0th index for the number of rows and the 1st index for the number of columns. For example:

```
# row and column shape of two-dimensional array
from numpy import array
# list of data
data = [[11, 22],
```

```
    [33, 44],
    [55, 66]]
# array of data
data = array(data)
print('Rows: %d' % data.shape[0])
print('Cols: %d' % data.shape[1])
```

Listing 5.36: Example of accessing `shape` for a two-dimensional array in terms of rows and columns.

Running the example accesses the specific size of each dimension.

```
Rows: 3
Cols: 2
```

Listing 5.37: Sample output `shape` for a two-dimensional array in terms of rows and columns.

5.5.2 Reshape 1D to 2D Array

It is common to need to reshape a one-dimensional array into a two-dimensional array with one column and multiple arrays. NumPy provides the `reshape()` function on the NumPy array object that can be used to reshape the data. The `reshape()` function takes a single argument that specifies the new shape of the array. In the case of reshaping a one-dimensional array into a two-dimensional array with one column, the tuple would be the shape of the array as the first dimension (`data.shape[0]`) and 1 for the second dimension.

```
data = data.reshape((data.shape[0], 1))
```

Listing 5.38: Example the `reshape()` function for reshaping from 1D to 2D data.

Putting this all together, we get the following worked example.

```
# reshape 1D array to 2D
from numpy import array
# define array
data = array([11, 22, 33, 44, 55])
print(data.shape)
# reshape
data = data.reshape((data.shape[0], 1))
print(data.shape)
```

Listing 5.39: Example of changing the shape of a one-dimensional array with the `reshape()` function.

Running the example prints the shape of the one-dimensional array, reshapes the array to have 5 rows with 1 column, then prints this new shape.

```
(5,)
(5, 1)
```

Listing 5.40: Sample output of changing the shape of a one-dimensional array with the `reshape()` function.

5.5.3 Reshape 2D to 3D Array

It is common to need to reshape two-dimensional data where each row represents a sequence into a three-dimensional array for algorithms that expect multiple samples of one or more time steps and one or more features. A good example is the LSTM recurrent neural network model in the Keras deep learning library. The reshape function can be used directly, specifying the new dimensionality. This is clear with an example where each sequence has multiple time steps with one observation (feature) at each time step. We can use the sizes in the shape attribute on the array to specify the number of samples (rows) and columns (time steps) and fix the number of features at 1.

```
data.reshape((data.shape[0], data.shape[1], 1))
```

Listing 5.41: Example the `reshape()` function for reshaping from 2D to 3D data.

Putting this all together, we get the following worked example.

```
# reshape 2D array to 3D
from numpy import array
# list of data
data = [[11, 22],
    [33, 44],
    [55, 66]]
# array of data
data = array(data)
print(data.shape)
# reshape
data = data.reshape((data.shape[0], data.shape[1], 1))
print(data.shape)
```

Listing 5.42: Example of changing the shape of a two-dimensional array with the `reshape()` function.

Running the example first prints the size of each dimension in the 2D array, reshapes the array, then summarizes the shape of the new 3D array.

```
(3, 2)
(3, 2, 1)
```

Listing 5.43: Sample output of changing the shape of a two-dimensional array with the `reshape()` function.

5.6 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Develop one example of indexing, slicing and reshaping your own small data arrays.
- Load a small real dataset from CSV file and split it into input and output elements
- Load a small real dataset from CSV file and split it into train and test elements.

If you explore any of these extensions, I'd love to know.

5.7 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

5.7.1 Books

- *Python for Data Analysis*, 2017.
  http://amzn.to/2B1sfXi
- *Elegant SciPy*, 2017.
  http://amzn.to/2yujXnT
- *Guide to NumPy*, 2015.
  http://amzn.to/2j3kEzd

5.7.2 References

- NumPy Reference.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/
- The N-dimensional array.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/arrays.ndarray.html
- Array creation routines.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/routines.array-creation.html
- NumPy Indexing.
  https://docs.scipy.org/doc/numpy-1.13.0/user/basics.indexing.html
- SciPy Indexing.
  https://docs.scipy.org/doc/numpy/reference/arrays.indexing.html
- Indexing routines.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/routines.indexing.html

5.7.3 API

- `numpy.array()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.array.html
- `numpy.reshape()` API.
  https://docs.scipy.org/doc/numpy/reference/generated/numpy.reshape.html

5.8 Summary

In this tutorial, you discovered how to access and reshape data in NumPy arrays with Python. Specifically, you learned:

- How to convert your list data to NumPy arrays.

- How to access data using Pythonic indexing and slicing.
- How to resize your data to meet the expectations of some machine learning APIs.

5.8.1 Next

In the next chapter you will discover array broadcasting and the rules that govern it in Python.

Chapter 6

NumPy Array Broadcasting

Arrays with different sizes cannot be added, subtracted, or generally be used in arithmetic. A way to overcome this is to duplicate the smaller array so that it has the dimensionality and size as the larger array. This is called array broadcasting and is available in NumPy when performing array arithmetic, which can greatly reduce and simplify your code. In this tutorial, you will discover the concept of array broadcasting and how to implement it in NumPy. After completing this tutorial, you will know:

- The problem of arithmetic with arrays with different sizes.
- The solution of broadcasting and common examples in one and two dimensions.
- The rule of array broadcasting and when broadcasting fails.

Let's get started.

6.1 Tutorial Overview

This tutorial is divided into 4 parts; they are:

1. Limitation with Array Arithmetic
2. Array Broadcasting
3. Broadcasting in NumPy
4. Limitations of Broadcasting

6.2 Limitation with Array Arithmetic

You can perform arithmetic directly on NumPy arrays, such as addition and subtraction. For example, two arrays can be added together to create a new array where the values at each index are added together. For example, an array a can be defined as $[1, 2, 3]$ and array b can be defined as $[1, 2, 3]$ and adding together will result in a new array with the values $[2, 4, 6]$.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a + b
c = [1 + 1, 2 + 2, 3 + 3]
```

Strictly, arithmetic may only be performed on arrays that have the same dimensions and dimensions with the same size. This means that a one-dimensional array with the length of 10 can only perform arithmetic with another one-dimensional array with the length 10. This limitation on array arithmetic is quite limiting indeed. Thankfully, NumPy provides a built-in workaround to allow arithmetic between arrays with differing sizes.

6.3 Array Broadcasting

Broadcasting is the name given to the method that NumPy uses to allow array arithmetic between arrays with a different shape or size. Although the technique was developed for NumPy, it has also been adopted more broadly in other numerical computational libraries, such as Theano, TensorFlow, and Octave. Broadcasting solves the problem of arithmetic between arrays of differing shapes by in effect replicating the smaller array along the last mismatched dimension.

> Vectors are built from components, which are ordinary numbers. You can think of a vector as a list of numbers, and vector algebra as operations performed on the numbers in the list.
>
> — *Broadcasting*, SciPy.org.

NumPy does not actually duplicate the smaller array; instead, it makes memory and computationally efficient use of existing structures in memory that in effect achieve the same result. The concept has also permeated linear algebra notation to simplify the explanation of simple operations.

> In the context of deep learning, we also use some less conventional notation. We allow the addition of matrix and a vector, yielding another matrix: $C = A + b$, where $C_{i,j} = A_{i,j} + b_j$. In other words, the vector b is added to each row of the matrix. This shorthand eliminates the need to define a matrix with b copied into each row before doing the addition. This implicit copying of b to many locations is called broadcasting.
>
> — Page 34, *Deep Learning*, 2016.

6.4 Broadcasting in NumPy

We can make broadcasting concrete by looking at three examples in NumPy. The examples in this section are not exhaustive, but instead are common to the types of broadcasting you may see or implement.

6.4.1 Scalar and One-Dimensional Array

A single value or scalar can be used in arithmetic with a one-dimensional array. For example, we can imagine a one-dimensional array a with three values $[a1, a2, a3]$ added to a scalar b.

```
a = [a1, a2, a3]
b
```

The scalar will need to be broadcast across the one-dimensional array by duplicating the value it 2 more times.

```
b = [b1, b2, b3]
```

The two one-dimensional arrays can then be added directly.

```
c = a + b
c = [a1 + b1, a2 + b2, a3 + b3]
```

The example below demonstrates this in NumPy.

```
# broadcast scalar to one-dimensional array
from numpy import array
# define array
a = array([1, 2, 3])
print(a)
# define scalar
b = 2
print(b)
# broadcast
c = a + b
print(c)
```

Listing 6.1: Example of broadcasting a scalar to a one-dimensional array in NumPy.

Running the example first prints the defined one-dimensional array, then the scalar, followed by the result where the scalar is added to each value in the array.

```
[1 2 3]

2

[3 4 5]
```

Listing 6.2: Results from broadcasting a scalar to a one-dimensional array in NumPy.

6.4.2 Scalar and Two-Dimensional Array

A scalar value can be used in arithmetic with a two-dimensional array. For example, we can imagine a two-dimensional array A with 2 rows and 3 columns added to the scalar b.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \end{pmatrix} \tag{6.1}$$

The scalar will need to be broadcast across each row of the two-dimensional array by duplicating it 5 more times.

$$B = \begin{pmatrix} b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,1} & b_{2,2} & b_{2,3} \end{pmatrix} \tag{6.2}$$

The two two-dimensional arrays can then be added directly.

$$C = A + B \tag{6.3}$$

$$C = \begin{pmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} & a_{1,3} + b_{1,3} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} & a_{2,3} + b_{2,3} \end{pmatrix} \tag{6.4}$$

The example below demonstrates this in NumPy.

```
# broadcast scalar to two-dimensional array
from numpy import array
# define array
A = array([
  [1, 2, 3],
  [1, 2, 3]])
print(A)
# define scalar
b = 2
print(b)
# broadcast
C = A + b
print(C)
```

Listing 6.3: Example of broadcasting a scalar to a two-dimensional array in NumPy.

Running the example first prints the defined two-dimensional array, then the scalar, then the result of the addition with the value 2 added to each value in the array.

```
[[1 2 3]
 [1 2 3]]

2

[[3 4 5]
 [3 4 5]]
```

Listing 6.4: Results from broadcasting a scalar to a two-dimensional array in NumPy.

6.4.3 One-Dimensional and Two-Dimensional Arrays

A one-dimensional array can be used in arithmetic with a two-dimensional array. For example, we can imagine a two-dimensional array A with 2 rows and 3 columns added to a one-dimensional array b with 3 values.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \end{pmatrix} \tag{6.5}$$

$$b = \begin{pmatrix} b_1 & b_2 & b_3 \end{pmatrix} \tag{6.6}$$

The one-dimensional array is broadcast across each row of the two-dimensional array by creating a second copy to result in a new two-dimensional array B.

$$B = \begin{pmatrix} b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,1} & b_{2,2} & b_{2,3} \end{pmatrix} \tag{6.7}$$

The two two-dimensional arrays can then be added directly.

$$C = A + B \tag{6.8}$$

$$C = \begin{pmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} & a_{1,3} + b_{1,3} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} & a_{2,3} + b_{2,3} \end{pmatrix} \tag{6.9}$$

The example below demonstrates this in NumPy.

```
# broadcast one-dimensional array to two-dimensional array
from numpy import array
# define two-dimensional array
A = array([
  [1, 2, 3],
  [1, 2, 3]])
print(A)
# define one-dimensional array
b = array([1, 2, 3])
print(b)
# broadcast
C = A + b
print(C)
```

Listing 6.5: Example of broadcasting a one-dimensional array to a two-dimensional array in NumPy.

Running the example first prints the defined two-dimensional array, then the defined one-dimensional array, followed by the result C where in effect each value in the two-dimensional array is doubled.

```
[[1 2 3]
 [1 2 3]]

[1 2 3]

[[2 4 6]
 [2 4 6]]
```

Listing 6.6: Results from broadcasting a one-dimensional to a two-dimensional array in NumPy.

6.5 Limitations of Broadcasting

Broadcasting is a handy shortcut that proves very useful in practice when working with NumPy arrays. That being said, it does not work for all cases, and in fact imposes a strict rule that must be satisfied for broadcasting to be performed. Arithmetic, including broadcasting, can only be performed when the shape of each dimension in the arrays are equal or one has the dimension size of 1. The dimensions are considered in reverse order, starting with the trailing dimension; for example, looking at columns before rows in a two-dimensional case.

This make more sense when we consider that NumPy will in effect pad missing dimensions with a size of 1 when comparing arrays. Therefore, the comparison between a two-dimensional array A with 2 rows and 3 columns and a vector b with 3 elements:

```
A.shape = (2 x 3)
b.shape = (3)
```

In effect, this becomes a comparison between:

```
A.shape = (2 x 3)
b.shape = (1 x 3)
```

This same notion applies to the comparison between a scalar that is treated as an array with the required number of dimensions:

```
A.shape = (2 x 3)
b.shape = (1)
```

This becomes a comparison between:

```
A.shape = (2 x 3)
b.shape = (1 x 1)
```

When the comparison fails, the broadcast cannot be performed, and an error is raised.

The example below attempts to broadcast a two-element array to a 2×3 array. This comparison is in effect:

```
A.shape = (2 x 3)
b.shape = (1 x 2)
```

We can see that the last dimensions (columns) do not match and we would expect the broadcast to fail. The example below demonstrates this in NumPy.

```
# broadcasting error
from numpy import array
# define two-dimensional array
A = array([
  [1, 2, 3],
  [1, 2, 3]])
print(A.shape)
# define one-dimensional array
b = array([1, 2])
print(b.shape)
# attempt broadcast
C = A + b
print(C)
```

Listing 6.7: Example of broadcasting error in NumPy.

Running the example first prints the shapes of the arrays then raises an error when attempting to broadcast, as we expected.

```
(2, 3)
(2,)
ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

Listing 6.8: Example output of a broadcast error..

6.6 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Create three new and different examples of broadcasting with NumPy arrays.
- Implement your own broadcasting function for manually broadcasting in one and two-dimensional cases.
- Benchmark NumPy broadcasting and your own custom broadcasting functions with one and two dimensional cases with very large arrays.

If you explore any of these extensions, I'd love to know.

6.7 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

6.7.1 Books

- Chapter 2, *Deep Learning*, 2016.
  http://amzn.to/2EnS7x5

6.7.2 Articles

- Broadcasting, NumPy API, SciPy.org.
  https://docs.scipy.org/doc/numpy-1.13.0/user/basics.broadcasting.html
- Broadcasting semantics in TensorFlow.
  https://www.tensorflow.org/performance/xla/broadcasting
- Array Broadcasting in NumPy, EricsBroadcastingDoc.
  http://scipy.github.io/old-wiki/pages/EricsBroadcastingDoc
- Broadcasting, Theano.
  http://deeplearning.net/software/theano/tutorial/broadcasting.html
- Broadcasting arrays in NumPy, 2015.
  https://eli.thegreenplace.net/2015/broadcasting-arrays-in-numpy/
- Broadcasting in Octave.
  https://www.gnu.org/software/octave/doc/v4.2.1/Broadcasting.html

6.8 Summary

In this tutorial, you discovered the concept of array broadcasting and how to implement in NumPy. Specifically, you learned:

- The problem of arithmetic with arrays with different sizes.
- The solution of broadcasting and common examples in one and two dimensions.
- The rule of array broadcasting and when broadcasting fails.

6.8.1 Next

This is the end of this part, in the next part you will discover vectors and matrices, the main data structures used in linear algebra.

Part IV

Matrices

Chapter 7

Vectors and Vector Arithmetic

Vectors are a foundational element of linear algebra. Vectors are used throughout the field of machine learning in the description of algorithms and processes such as the target variable (y) when training an algorithm. In this tutorial, you will discover linear algebra vectors for machine learning. After completing this tutorial, you will know:

- What a vector is and how to define one in Python with NumPy.
- How to perform vector arithmetic such as addition, subtraction, multiplication and division.
- How to perform additional operations such as dot product and multiplication with a scalar.

Let's get started.

7.1 Tutorial Overview

This tutorial is divided into 5 parts; they are:

1. What is a Vector
2. Defining a Vector
3. Vector Arithmetic
4. Vector Dot Product
5. Vector-Scalar Multiplication

7.2 What is a Vector

A vector is a tuple of one or more values called scalars.

> Vectors are built from components, which are ordinary numbers. You can think of a vector as a list of numbers, and vector algebra as operations performed on the numbers in the list.
>
> — Page 69, *No Bullshit Guide To Linear Algebra*, 2017.

Vectors are often represented using a lowercase character such as v; for example:

$$v = (v_1, v_2, v_3) \tag{7.1}$$

Where v_1, v_2, v_3 are scalar values, often real values.
Vectors are also shown using a vertical representation or a column; for example:

$$v = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \tag{7.2}$$

It is common to represent the target variable as a vector with the lowercase y when describing the training of a machine learning algorithm. It is common to introduce vectors using a geometric analogy, where a vector represents a point or coordinate in an n-dimensional space, where n is the number of dimensions, such as 2. The vector can also be thought of as a line from the origin of the vector space with a direction and a magnitude.

These analogies are good as a starting point, but should not be held too tightly as we often consider very high dimensional vectors in machine learning. I find the vector-as-coordinate the most compelling analogy in machine learning. Now that we know what a vector is, let's look at how to define a vector in Python.

7.3 Defining a Vector

We can represent a vector in Python as a NumPy array. A NumPy array can be created from a list of numbers. For example, below we define a vector with the length of 3 and the integer values 1, 2 and 3.

```
# create a vector
from numpy import array
# define vector
v = array([1, 2, 3])
print(v)
```

Listing 7.1: Example of defining a vector.

The example defines a vector with 3 elements. Running the example prints the defined vector.

```
[1 2 3]
```

Listing 7.2: Sample output from defining a vector.

7.4 Vector Arithmetic

In this section will demonstrate simple vector-vector arithmetic, where all operations are performed element-wise between two vectors of equal length to result in a new vector with the same length

7.4.1 Vector Addition

Two vectors of equal length can be added together to create a new third vector.

$$c = a + b \tag{7.3}$$

The new vector has the same length as the other two vectors. Each element of the new vector is calculated as the addition of the elements of the other vectors at the same index; for example:

$$c = (a_1 + b_1, a_2 + b_2, a_3 + b_3) \tag{7.4}$$

Or, put another way:

$$\begin{aligned} c[0] &= a[0] + b[0] \\ c[1] &= a[1] + b[1] \\ c[2] &= a[2] + b[2] \end{aligned} \tag{7.5}$$

We can add vectors directly in Python by adding NumPy arrays.

```
# vector addition
from numpy import array
# define first vector
a = array([1, 2, 3])
print(a)
# define second vector
b = array([1, 2, 3])
print(b)
# add vectors
c = a + b
print(c)
```

Listing 7.3: Example of vector addition.

The example defines two vectors with three elements each, then adds them together. Running the example first prints the two parent vectors then prints a new vector that is the addition of the two vectors.

```
[1 2 3]

[1 2 3]

[2 4 6]
```

Listing 7.4: Sample output from vector addition.

7.4.2 Vector Subtraction

One vector can be subtracted from another vector of equal length to create a new third vector.

$$c = a - b \tag{7.6}$$

As with addition, the new vector has the same length as the parent vectors and each element of the new vector is calculated as the subtraction of the elements at the same indices.

$$c = (a_1 - b_1, a_2 - b_2, a_3 - b_3) \tag{7.7}$$

Or, put another way:

$$\begin{aligned} c[0] &= a[0] - b[0] \\ c[1] &= a[1] - b[1] \\ c[2] &= a[2] - b[2] \end{aligned} \tag{7.8}$$

The NumPy arrays can be directly subtracted in Python.

```
# vector subtraction
from numpy import array
# define first vector
a = array([1, 2, 3])
print(a)
# define second vector
b = array([0.5, 0.5, 0.5])
print(b)
# subtract vectors
c = a - b
print(c)
```

Listing 7.5: Example of vector subtraction.

The example defines two vectors with three elements each, then subtracts the first from the second. Running the example first prints the two parent vectors then prints the new vector that is the first minus the second.

```
[1 2 3]

[ 0.5 0.5 0.5]

[ 0.5 1.5 2.5]
```

Listing 7.6: Sample output from vector subtraction.

7.4.3 Vector Multiplication

Two vectors of equal length can be multiplied together.

$$c = a \times b \tag{7.9}$$

As with addition and subtraction, this operation is performed element-wise to result in a new vector of the same length.

$$c = (a_1 \times b_1, a_2 \times b_2, a_3 \times b_3) \tag{7.10}$$

or

$$c = (a_1 b_1, a_2 b_2, a_3 b_3) \tag{7.11}$$

Or, put another way:

$$\begin{aligned} c[0] &= a[0] \times b[0] \\ c[1] &= a[1] \times b[1] \\ c[2] &= a[2] \times b[2] \end{aligned} \tag{7.12}$$

We can perform this operation directly in NumPy.

```
# vector multiplication
from numpy import array
# define first vector
a = array([1, 2, 3])
print(a)
# define second vector
b = array([1, 2, 3])
print(b)
# multiply vectors
c = a * b
print(c)
```

Listing 7.7: Example of vector multiplication.

The example defines two vectors with three elements each, then multiplies the vectors together. Running the example first prints the two parent vectors, then the new vector is printed.

```
[1 2 3]

[1 2 3]

[1 4 9]
```

Listing 7.8: Sample output from vector multiplication.

7.4.4 Vector Division

Two vectors of equal length can be divided.

$$c = \frac{a}{b} \tag{7.13}$$

As with other arithmetic operations, this operation is performed element-wise to result in a new vector of the same length.

$$c = (\frac{a_1}{b_1}, \frac{a_2}{b_2}, \frac{a_3}{b_3}) \tag{7.14}$$

Or, put another way:

$$\begin{aligned} c[0] &= a[0]/b[0] \\ c[1] &= a[1]/b[1] \\ c[2] &= a[2]/b[2] \end{aligned} \tag{7.15}$$

We can perform this operation directly in NumPy.

```
# vector division
from numpy import array
# define first vector
a = array([1, 2, 3])
print(a)
# define second vector
b = array([1, 2, 3])
print(b)
# divide vectors
c = a / b
print(c)
```

Listing 7.9: Example of vector division.

The example defines two vectors with three elements each, then divides the first by the second. Running the example first prints the two parent vectors, followed by the result of the vector division.

```
[1 2 3]

[1 2 3]

[ 1.  1.  1.]
```

Listing 7.10: Sample output from vector division.

7.5 Vector Dot Product

We can calculate the sum of the multiplied elements of two vectors of the same length to give a scalar. This is called the dot product, named because of the dot operator used when describing the operation.

> The dot product is the key tool for calculating vector projections, vector decompositions, and determining orthogonality. The name dot product comes from the symbol used to denote it.
>
> — Page 110, *No Bullshit Guide To Linear Algebra*, 2017.

$$c = a \cdot b \tag{7.16}$$

The operation can be used in machine learning to calculate the weighted sum of a vector. The dot product is calculated as follows:

$$c = (a_1 \times b_1 + a_2 \times b_2 + a_3 \times b_3) \tag{7.17}$$

or

$$c = (a_1 b_1 + a_2 b_2 + a_3 b_3) \tag{7.18}$$

We can calculate the dot product between two vectors in Python using the `dot()` function on a NumPy array.

```
# vector dot product
from numpy import array
# define first vector
a = array([1, 2, 3])
print(a)
# define second vector
b = array([1, 2, 3])
print(b)
# multiply vectors
c = a.dot(b)
print(c)
```

Listing 7.11: Example of vector dot product.

The example defines two vectors with three elements each, then calculates the dot product. Running the example first prints the two parent vectors, then the scalar dot product.

```
[1 2 3]

[1 2 3]

14
```

Listing 7.12: Sample output from vector dot product.

7.6 Vector-Scalar Multiplication

A vector can be multiplied by a scalar, in effect scaling the magnitude of the vector. To keep notation simple, we will use lowercase s to represent the scalar value.

$$c = s \times v \tag{7.19}$$

or

$$c = sv \tag{7.20}$$

The multiplication is performed on each element of the vector to result in a new scaled vector of the same length.

$$c = (s \times v_1, s \times v_2, s \times v_3) \tag{7.21}$$

Or, put another way:

$$\begin{aligned} c[0] &= v[0] \times s \\ c[1] &= v[1] \times s \\ c[2] &= v[2] \times s \end{aligned} \tag{7.22}$$

We can perform this operation directly with the NumPy array.

```
# vector-scalar multiplication
from numpy import array
# define vector
a = array([1, 2, 3])
```

```
print(a)
# define scalar
s = 0.5
print(s)
# multiplication
c = s * a
print(c)
```

Listing 7.13: Example of vector-scalar multiplication.

The example first defines the vector and the scalar then multiplies the vector by the scalar. Running the example first prints the parent vector, then scalar, and then the result of multiplying the two together.

```
[1 2 3]

0.5

[ 0.5  1.   1.5]
```

Listing 7.14: Sample output from vector-scalar multiplication.

Similarly, vector-scalar addition, subtraction, and division can be performed in the same way.

7.7 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Create one example using each operation using your own small array data.
- Implement each vector arithmetic operation manually for vectors defined as lists.
- Search machine learning papers and find 1 example of each operation being used.

If you explore any of these extensions, I'd love to know.

7.8 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

7.8.1 Books

- Section 1.15, Vectors. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Section 2.2, Vector operations. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Section 1.1 Vectors and Linear Combinations, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2j2J0g4

- Section 2.1 Scalars, Vectors, Matrices and Tensors, *Deep Learning*, 2016.
  http://amzn.to/2j4oKuP
- Section 1.B Definition of Vector Space, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI

7.8.2 API

- `numpy.array()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.array.html
- `numpy.dot()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.dot.html

7.8.3 Articles

- Vector space on Wikipedia.
  https://en.wikipedia.org/wiki/Vector_space
- Dot product on Wikipedia.
  https://en.wikipedia.org/wiki/Dot_product

7.9 Summary

In this tutorial, you discovered linear algebra vectors for machine learning. Specifically, you learned:

- What a vector is and how to define one in Python with NumPy.
- How to perform vector arithmetic such as addition, subtraction, multiplication and division.
- How to perform additional operations such as dot product and multiplication with a scalar.

7.9.1 Next

In the next chapter you will discover vector norms for calculating the magnitude of vectors.

Chapter 8

Vector Norms

Calculating the length or magnitude of vectors is often required either directly as a regularization method in machine learning, or as part of broader vector or matrix operations. In this tutorial, you will discover the different ways to calculate vector lengths or magnitudes, called the vector norm. After completing this tutorial, you will know:

- The L^1 norm that is calculated as the sum of the absolute values of the vector.
- The L^2 norm that is calculated as the square root of the sum of the squared vector values.
- The max norm that is calculated as the maximum vector values.

Let's get started.

8.1 Tutorial Overview

This tutorial is divided into 4 parts; they are:

1. Vector Norm
2. Vector L^1 Norm
3. Vector L^2 Norm
4. Vector Max Norm

8.2 Vector Norm

Calculating the size or length of a vector is often required either directly or as part of a broader vector or vector-matrix operation. The length of the vector is referred to as the vector norm or the vector's magnitude.

> The length of a vector is a nonnegative number that describes the extent of the vector in space, and is sometimes referred to as the vector's magnitude or the norm.
>
> — Page 112, *No Bullshit Guide To Linear Algebra*, 2017.

The length of the vector is always a positive number, except for a vector of all zero values. It is calculated using some measure that summarizes the distance of the vector from the origin of the vector space. For example, the origin of a vector space for a vector with 3 elements is $(0, 0, 0)$. Notations are used to represent the vector norm in broader calculations and the type of vector norm calculation almost always has its own unique notation. We will take a look at a few common vector norm calculations used in machine learning.

8.3 Vector L^1 Norm

The length of a vector can be calculated using the L^1 norm, where the 1 is a superscript of the L. The notation for the L^1 norm of a vector is $||v||_1$, where 1 is a subscript. As such, this length is sometimes called the taxicab norm or the Manhattan norm.

$$L^1(v) = ||v||_1 \tag{8.1}$$

The L^1 norm is calculated as the sum of the absolute vector values, where the absolute value of a scalar uses the notation $|a_1|$. In effect, the norm is a calculation of the Manhattan distance from the origin of the vector space.

$$||v||_1 = |a_1| + |a_2| + |a_3| \tag{8.2}$$

> In several machine learning applications, it is important to discriminate between elements that are exactly zero and elements that are small but nonzero. In these cases, we turn to a function that grows at the same rate in all locations, but retains mathematical simplicity: the L^1 norm.
>
> — Pages 39-40, *Deep Learning*, 2016.

The L^1 norm of a vector can be calculated in NumPy using the `norm()` function with a parameter to specify the norm order, in this case 1.

```
# vector L1 norm
from numpy import array
from numpy.linalg import norm
# define vector
a = array([1, 2, 3])
print(a)
# calculate norm
l1 = norm(a, 1)
print(l1)
```

Listing 8.1: Example of calculating the L^1 vector norm.

First, a 3-element vector is defined, then the L^1 norm of the vector is calculated. Running the example first prints the defined vector and then the vector's L^1 norm.

```
[1 2 3]

6.0
```

Listing 8.2: Sample output from calculating the L^1 vector norm.

The L^1 norm is often used when fitting machine learning algorithms as a regularization method, e.g. a method to keep the coefficients of the model small, and in turn, the model less complex.

8.4 Vector L^2 Norm

The length of a vector can be calculated using the L^2 norm, where the 2 is a superscript of the L. The notation for the L^2 norm of a vector is $||v||_2$ where 2 is a subscript.

$$L^2(v) = ||v||_2 \tag{8.3}$$

The L^2 norm calculates the distance of the vector coordinate from the origin of the vector space. As such, it is also known as the Euclidean norm as it is calculated as the Euclidean distance from the origin. The result is a positive distance value. The L^2 norm is calculated as the square root of the sum of the squared vector values.

$$||v||_2 = \sqrt{a_1^2 + a_2^2 + a_3^2} \tag{8.4}$$

The L^2 norm of a vector can be calculated in NumPy using the `norm()` function with default parameters.

```
# vector L2 norm
from numpy import array
from numpy.linalg import norm
# define vector
a = array([1, 2, 3])
print(a)
# calculate norm
l2 = norm(a)
print(l2)
```

Listing 8.3: Example of calculating the L^2 vector norm.

First, a 3-element vector is defined, then the L^2 norm of the vector is calculated. Running the example first prints the defined vector and then the vector's L^2 norm.

```
[1 2 3]

3.74165738677
```

Listing 8.4: Sample output from calculating the L^2 vector norm.

Like the L^1 norm, the L^2 norm is often used when fitting machine learning algorithms as a regularization method, e.g. a method to keep the coefficients of the model small and, in turn, the model less complex. By far, the L^2 norm is more commonly used than other vector norms in machine learning.

8.5 Vector Max Norm

The length of a vector can be calculated using the maximum norm, also called max norm. Max norm of a vector is referred to as L^{inf} where inf is a superscript and can be represented with

the infinity symbol. The notation for max norm is $||v||_{inf}$, where inf is a subscript.

$$L^{inf}(v) = ||v||_{inf} \tag{8.5}$$

The max norm is calculated as returning the maximum value of the vector, hence the name.

$$||v||_{inf} = \max a_1, a_2, a_3 \tag{8.6}$$

The max norm of a vector can be calculated in NumPy using the `norm()` function with the order parameter set to `inf`.

```
# vector max norm
from math import inf
from numpy import array
from numpy.linalg import norm
# define vector
a = array([1, 2, 3])
print(a)
# calculate norm
maxnorm = norm(a, inf)
print(maxnorm)
```

Listing 8.5: Example of calculating the max vector norm.

First, a 3×3 vector is defined, then the max norm of the vector is calculated. Running the example first prints the defined vector and then the vector's max norm.

```
[1 2 3]

3.0
```

Listing 8.6: Sample output from calculating the max vector norm.

Max norm is also used as a regularization in machine learning, such as on neural network weights, called max norm regularization.

8.6 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Create one example using each operation using your own small array data.
- Implement each operation manually for vectors defined as lists of lists.
- Search machine learning papers and find 1 example of each operation being used.

If you explore any of these extensions, I'd love to know.

8.7 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

8.7.1 Books

- Section 1.2 Lengths and Dot Products, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2j2J0g4
- Section 2.5 Norms, *Deep Learning*, 2016.
  http://amzn.to/2j4oKuP
- Section 6.A Inner Products and Norms, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI
- Lecture 3 Norms, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2BI9kRH

8.7.2 API

- `numpy.linalg.norm()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.norm.html

8.7.3 Articles

- Norm (mathematics) on Wikipedia.
  https://en.wikipedia.org/wiki/Norm_(mathematics)

8.8 Summary

In this tutorial, you discovered the different ways to calculate vector lengths or magnitudes, called the vector norm. Specifically, you learned:

- The L^1 norm that is calculated as the sum of the absolute values of the vector.
- The L^2 norm that is calculated as the square root of the sum of the squared vector values.
- The max norm that is calculated as the maximum vector values.

8.8.1 Next

In the next chapter you will discover matrices and basic matrix arithmetic.

Chapter 9

Matrices and Matrix Arithmetic

Matrices are a foundational element of linear algebra. Matrices are used throughout the field of machine learning in the description of algorithms and processes such as the input data variable (X) when training an algorithm. In this tutorial, you will discover matrices in linear algebra and how to manipulate them in Python. After completing this tutorial, you will know:

- What a matrix is and how to define one in Python with NumPy.
- How to perform element-wise operations such as addition, subtraction, and the Hadamard product.
- How to multiply matrices together and the intuition behind the operation.

Let's get started.

9.1 Tutorial Overview

This tutorial is divided into 6 parts; they are:

1. What is a Matrix
2. Defining a Matrix
3. Matrix Arithmetic
4. Matrix-Matrix Multiplication
5. Matrix-Vector Multiplication
6. Matrix-Scalar Multiplication

9.2 What is a Matrix

A matrix is a two-dimensional array of scalars with one or more columns and one or more rows.

> A matrix is a two-dimensional array (a table) of numbers.

— Page 115, *No Bullshit Guide To Linear Algebra*, 2017.

The notation for a matrix is often an uppercase letter, such as A, and entries are referred to by their two-dimensional subscript of row (i) and column (j), such as $a_{i,j}$. For example, we can define a 3-row, 2-column matrix:

$$A = ((a_{1,1}, a_{1,2}), (a_{2,1}, a_{2,2}), (a_{3,1}, a_{3,2})) \tag{9.1}$$

It is more common to see matrices defined using a horizontal notation.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{pmatrix} \tag{9.2}$$

A likely first place you may encounter a matrix in machine learning is in model training data comprised of many rows and columns and often represented using the capital letter X. The geometric analogy used to help understand vectors and some of their operations does not hold with matrices. Further, a vector itself may be considered a matrix with one column and multiple rows. Often the dimensions of the matrix are denoted as m and n or $m \times n$ for the number of rows and the number of columns respectively. Now that we know what a matrix is, let's look at defining one in Python.

9.3 Defining a Matrix

We can represent a matrix in Python using a two-dimensional NumPy array. A NumPy array can be constructed given a list of lists. For example, below is a 2 row, 3 column matrix.

```
# create matrix
from numpy import array
A = array([[1, 2, 3], [4, 5, 6]])
print(A)
```

Listing 9.1: Example of creating a matrix.

Running the example prints the created matrix showing the expected structure.

```
[[1 2 3]
 [4 5 6]]
```

Listing 9.2: Sample output from creating a matrix.

9.4 Matrix Arithmetic

In this section will demonstrate simple matrix-matrix arithmetic, where all operations are performed element-wise between two matrices of equal size to result in a new matrix with the same size.

9.4.1 Matrix Addition

Two matrices with the same dimensions can be added together to create a new third matrix.

$$C = A + B \tag{9.3}$$

The scalar elements in the resulting matrix are calculated as the addition of the elements in each of the matrices being added.

$$C = \begin{pmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} \\ a_{3,1} + b_{3,1} & a_{3,2} + b_{3,2} \end{pmatrix} \tag{9.4}$$

Or, put another way:

$$\begin{aligned} C[0,0] &= A[0,0] + B[0,0] \\ C[1,0] &= A[1,0] + B[1,0] \\ C[2,0] &= A[2,0] + B[2,0] \\ C[0,1] &= A[0,1] + B[0,1] \\ C[1,1] &= A[1,1] + B[1,1] \\ C[2,1] &= A[2,1] + B[2,1] \end{aligned} \tag{9.5}$$

We can implement this in Python using the plus operator directly on the two NumPy arrays.

```
# matrix addition
from numpy import array
# define first matrix
A = array([
  [1, 2, 3],
  [4, 5, 6]])
print(A)
# define second matrix
B = array([
  [1, 2, 3],
  [4, 5, 6]])
print(B)
# add matrices
C = A + B
print(C)
```

Listing 9.3: Example of matrix addition.

The example first defines two 2×3 matrices and then adds them together. Running the example first prints the two parent matrices and then the result of adding them together.

```
[[1 2 3]
 [4 5 6]]

[[1 2 3]
 [4 5 6]]

[[ 2  4  6]
 [ 8 10 12]]
```

Listing 9.4: Sample output from matrix addition.

9.4.2 Matrix Subtraction

Similarly, one matrix can be subtracted from another matrix with the same dimensions.

$$C = A - B \tag{9.6}$$

The scalar elements in the resulting matrix are calculated as the subtraction of the elements in each of the matrices.

$$C = \begin{pmatrix} a_{1,1} - b_{1,1} & a_{1,2} - b_{1,2} \\ a_{2,1} - b_{2,1} & a_{2,2} - b_{2,2} \\ a_{3,1} - b_{3,1} & a_{3,2} - b_{3,2} \end{pmatrix} \tag{9.7}$$

Or, put another way:

$$\begin{aligned} C[0,0] &= A[0,0] - B[0,0] \\ C[1,0] &= A[1,0] - B[1,0] \\ C[2,0] &= A[2,0] - B[2,0] \\ C[0,1] &= A[0,1] - B[0,1] \\ C[1,1] &= A[1,1] - B[1,1] \\ C[2,1] &= A[2,1] - B[2,1] \end{aligned} \tag{9.8}$$

We can implement this in Python using the minus operator directly on the two NumPy arrays.

```
# matrix subtraction
from numpy import array
# define first matrix
A = array([
  [1, 2, 3],
  [4, 5, 6]])
print(A)
# define second matrix
B = array([
  [0.5, 0.5, 0.5],
  [0.5, 0.5, 0.5]])
print(B)
# subtract matrices
C = A - B
print(C)
```

Listing 9.5: Example of matrix subtraction.

The example first defines two 2×3 matrices and then subtracts one from the other. Running the example first prints the two parent matrices and then subtracts the first matrix from the second.

```
[[1 2 3]
 [4 5 6]]

[[ 0.5 0.5  0.5]
 [ 0.5 0.5  0.5]]

[[ 0.5 1.5  2.5]
```

```
 [ 3.5  4.5  5.5]]
```

Listing 9.6: Sample output from matrix subtraction.

9.4.3 Matrix Multiplication (Hadamard Product)

Two matrices with the same size can be multiplied together, and this is often called element-wise matrix multiplication or the Hadamard product. It is not the typical operation meant when referring to matrix multiplication, therefore a different operator is often used, such as a circle $\circ$.

$$C = A \circ B \tag{9.9}$$

As with element-wise subtraction and addition, element-wise multiplication involves the multiplication of elements from each parent matrix to calculate the values in the new matrix.

$$C = \begin{pmatrix} a_{1,1} \times b_{1,1} & a_{1,2} \times b_{1,2} \\ a_{2,1} \times b_{2,1} & a_{2,2} \times b_{2,2} \\ a_{3,1} \times b_{3,1} & a_{3,2} \times b_{3,2} \end{pmatrix} \tag{9.10}$$

Or, put another way:

$$\begin{aligned} C[0,0] &= A[0,0] \times B[0,0] \\ C[1,0] &= A[1,0] \times B[1,0] \\ C[2,0] &= A[2,0] \times B[2,0] \\ C[0,1] &= A[0,1] \times B[0,1] \\ C[1,1] &= A[1,1] \times B[1,1] \\ C[2,1] &= A[2,1] \times B[2,1] \end{aligned} \tag{9.11}$$

We can implement this in Python using the star operator directly on the two NumPy arrays.

```
# matrix Hadamard product
from numpy import array
# define first matrix
A = array([
  [1, 2, 3],
  [4, 5, 6]])
print(A)
# define second matrix
B = array([
  [1, 2, 3],
  [4, 5, 6]])
print(B)
# multiply matrices
C = A * B
print(C)
```

Listing 9.7: Example of matrix Hadamard product.

The example first defines two 2×3 matrices and then multiplies them together. Running the example first prints the two parent matrices and then the result of multiplying them together with a Hadamard Product.

```
[[1 2 3]
 [4 5 6]]

[[1 2 3]
 [4 5 6]]

[[ 1  4  9]
 [16 25 36]]
```

Listing 9.8: Sample output from matrix Hadamard product.

9.4.4 Matrix Division

One matrix can be divided by another matrix with the same dimensions.

$$C = \frac{A}{B} \tag{9.12}$$

The scalar elements in the resulting matrix are calculated as the division of the elements in each of the matrices.

$$C = \begin{pmatrix} \frac{a_{1,1}}{b_{1,1}} & \frac{a_{1,2}}{b_{1,2}} \\ \frac{a_{2,1}}{b_{2,1}} & \frac{a_{2,2}}{b_{2,2}} \\ \frac{a_{3,1}}{b_{3,1}} & \frac{a_{3,2}}{b_{3,2}} \end{pmatrix} \tag{9.13}$$

Or, put another way:

$$\begin{aligned} C[0,0] &= A[0,0]/B[0,0] \\ C[1,0] &= A[1,0]/B[1,0] \\ C[2,0] &= A[2,0]/B[2,0] \\ C[0,1] &= A[0,1]/B[0,1] \\ C[1,1] &= A[1,1]/B[1,1] \\ C[2,1] &= A[2,1]/B[2,1] \end{aligned} \tag{9.14}$$

We can implement this in Python using the division operator directly on the two NumPy arrays.

```
# matrix division
from numpy import array
# define first matrix
A = array([
  [1, 2, 3],
  [4, 5, 6]])
print(A)
# define second matrix
B = array([
  [1, 2, 3],
  [4, 5, 6]])
print(B)
# divide matrices
C = A / B
print(C)
```

Listing 9.9: Example of matrix division.

The example first defines two 2×3 matrices and then divides the first from the second matrix. Running the example first prints the two parent matrices and then divides the first matrix by the second.

```
[[1 2 3]
 [4 5 6]]

[[1 2 3]
 [4 5 6]]

[[ 1.  1.  1.]
 [ 1.  1.  1.]]
```

Listing 9.10: Sample output from matrix division.

9.5 Matrix-Matrix Multiplication

Matrix multiplication, also called the matrix dot product is more complicated than the previous operations and involves a rule as not all matrices can be multiplied together.

$$C = A \cdot B \tag{9.15}$$

or

$$C = AB \tag{9.16}$$

The rule for matrix multiplication is as follows:

- The number of columns (n) in the first matrix (A) must equal the number of rows (m) in the second matrix (B).

For example, matrix A has the dimensions m rows and n columns and matrix B has the dimensions n and k. The n columns in A and n rows in B are equal. The result is a new matrix with m rows and k columns.

$$C(m, k) = A(m, n) \cdot B(n, k) \tag{9.17}$$

This rule applies for a chain of matrix multiplications where the number of columns in one matrix in the chain must match the number of rows in the following matrix in the chain.

> One of the most important operations involving matrices is multiplication of two matrices. The matrix product of matrices A and B is a third matrix C. In order for this product to be defined, A must have the same number of columns as B has rows. If A is of shape $m \times n$ and B is of shape $n \times p$, then C is of shape $m \times p$.
>
> — Page 34, *Deep Learning*, 2016.

The intuition for the matrix multiplication is that we are calculating the dot product between each row in matrix A with each column in matrix B. For example, we can step down rows of column A and multiply each with column 1 in B to give the scalar values in column 1 of C. Below describes the matrix multiplication using matrix notation.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{pmatrix} \tag{9.18}$$

$$B = \begin{pmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{pmatrix} \tag{9.19}$$

$$C = \begin{pmatrix} a_{1,1} \times b_{1,1} + a_{1,2} \times b_{2,1}, a_{1,1} \times b_{1,2} + a_{1,2} \times b_{2,2} \\ a_{2,1} \times b_{1,1} + a_{2,2} \times b_{2,1}, a_{2,1} \times b_{1,2} + a_{2,2} \times b_{2,2} \\ a_{3,1} \times b_{1,1} + a_{3,2} \times b_{2,1}, a_{3,1} \times b_{1,2} + a_{3,2} \times b_{2,2} \end{pmatrix} \tag{9.20}$$

We can describe the matrix multiplication operation using array notation.

$$\begin{aligned} C[0,0] &= A[0,0] \times B[0,0] + A[0,1] \times B[1,0] \\ C[1,0] &= A[1,0] \times B[0,0] + A[1,1] \times B[1,0] \\ C[2,0] &= A[2,0] \times B[0,0] + A[2,1] \times B[1,0] \\ C[0,1] &= A[0,0] \times B[0,1] + A[0,1] \times B[1,1] \\ C[1,1] &= A[1,0] \times B[0,1] + A[1,1] \times B[1,1] \\ C[2,1] &= A[2,0] \times B[0,1] + A[2,1] \times B[1,1] \end{aligned} \tag{9.21}$$

The matrix multiplication operation can be implemented in NumPy using the `dot()` function. It can also be calculated using the newer `@` operator, since Python version 3.5. The example below demonstrates both methods.

```
# matrix dot product
from numpy import array
# define first matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# define second matrix
B = array([
  [1, 2],
  [3, 4]])
print(B)
# multiply matrices
C = A.dot(B)
print(C)
# multiply matrices with @ operator
D = A @ B
print(D)
```

Listing 9.11: Example of matrix-matrix dot product.

The example first defines two 3×2 matrices and then calculates their dot product using the `dot()` function and the `@` operator. Running the example first prints the two parent matrices and then the results of the two dot product operations.

```
[[1 2]
 [3 4]
 [5 6]]

[[1 2]
 [3 4]]

[[ 7 10]
 [15 22]
 [23 34]]

 [[ 7 10]
 [15 22]
 [23 34]]
```

Listing 9.12: Sample output matrix-matrix dot product.

I recommend using the `dot()` function for matrix multiplication for now given the newness of the `@` operator.

9.6 Matrix-Vector Multiplication

A matrix and a vector can be multiplied together as long as the rule of matrix multiplication is observed. Specifically, that the number of columns in the matrix must equal the number of items in the vector. As with matrix multiplication, the operation can be written using the dot notation. Because the vector only has one column, the result is always a vector.

$$c = A \cdot v \tag{9.22}$$

Or without the dot in a compact form.

$$c = Av \tag{9.23}$$

The result is a vector with the same number of rows as the parent matrix.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{pmatrix} \tag{9.24}$$

$$v = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \tag{9.25}$$

$$c = \begin{pmatrix} a_{1,1} \times v_1 + a_{1,2} \times v_2 \\ a_{2,1} \times v_1 + a_{2,2} \times v_2 \\ a_{3,1} \times v_1 + a_{3,2} \times v_2 \end{pmatrix} \tag{9.26}$$

Or, more compactly.

$$c = \begin{pmatrix} a_{1,1}v_1 + a_{1,2}v_2 \\ a_{2,1}v_1 + a_{2,2}v_2 \\ a_{3,1}v_1 + a_{3,2}v_2 \end{pmatrix} \tag{9.27}$$

We can also represent this with array notation.

$$\begin{aligned} c[0] &= A[0,0] \times v[0] + A[0,1] \times v[1] \\ c[1] &= A[1,0] \times v[0] + A[1,1] \times v[1] \\ c[2] &= A[2,0] \times v[0] + A[2,1] \times v[1] \end{aligned} \tag{9.28}$$

The matrix-vector multiplication can be implemented in NumPy using the `dot()` function.

```
# matrix-vector multiplication
from numpy import array
# define matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# define vector
B = array([0.5, 0.5])
print(B)
# multiply
C = A.dot(B)
print(C)
```

Listing 9.13: Example of matrix-vector dot product.

The example first defines a 3×2 matrix and a 2 element vector and then multiplies them together. Running the example first prints the parent matrix and vector and then the result of multiplying them together.

```
[[1 2]
 [3 4]
 [5 6]]

[ 0.5  0.5]

[ 1.5  3.5  5.5]
```

Listing 9.14: Sample output matrix-vector dot product.

9.7 Matrix-Scalar Multiplication

A matrix can be multiplied by a scalar. This can be represented using the dot notation between the matrix and the scalar.

$$C = A \cdot b \tag{9.29}$$

Or without the dot notation.

$$C = Ab \tag{9.30}$$

The result is a matrix with the same size as the parent matrix where each element of the matrix is multiplied by the scalar value.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{pmatrix} \tag{9.31}$$

$$C = \begin{pmatrix} a_{1,1} \times b + a_{1,2} \times b \\ a_{2,1} \times b + a_{2,2} \times b \\ a_{3,1} \times b + a_{3,2} \times b \end{pmatrix} \tag{9.32}$$

or

$$C = \begin{pmatrix} a_{1,1}b + a_{1,2}b \\ a_{2,1}b + a_{2,2}b \\ a_{3,1}b + a_{3,2}b \end{pmatrix} \tag{9.33}$$

We can also represent this with array notation.

$$\begin{aligned} C[0,0] &= A[0,0] \times b \\ C[1,0] &= A[1,0] \times b \\ C[2,0] &= A[2,0] \times b \\ C[0,1] &= A[0,1] \times b \\ C[1,1] &= A[1,1] \times b \\ C[2,1] &= A[2,1] \times b \end{aligned} \tag{9.34}$$

This can be implemented directly in NumPy with the multiplication operator.

```
# matrix-scalar multiplication
from numpy import array
# define matrix
A = array([[1, 2], [3, 4], [5, 6]])
print(A)
# define scalar
b = 0.5
print(b)
# multiply
C = A * b
print(C)
```

Listing 9.15: Example of matrix-scalar dot product.

The example first defines a 3×2 matrix and a scalar and then multiplies them together. Running the example first prints the parent matrix and scalar and then the result of multiplying them together.

```
[[1 2]
 [3 4]
 [5 6]]
```

```
0.5

[[ 0.5 1. ]
 [ 1.5 2. ]
 [ 2.5 3. ]]
```

Listing 9.16: Sample output matrix-scalar dot product.

9.8 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Create one example using each operation using your own small array data.
- Implement each matrix arithmetic operation manually for matrices defined as lists of lists.
- Search machine learning papers and find 1 example of each operation being used.

If you explore any of these extensions, I'd love to know.

9.9 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

9.9.1 Books

- Section 2.3, Matrix operations. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Section 3.3, Matrix multiplication. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Section 1.3 Matrices, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Section 2.4 Rules for Matrix Operations, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Section 2.1 Scalars, Vectors, Matrices and Tensors, *Deep Learning*, 2016.
  http://amzn.to/2j4oKuP
- Section 2.2 Multiplying Matrices and Vectors, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU
- Section 3.C Matrices, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI
- Lecture 1 Matrix-Vector Multiplication, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2BI9kRH

9.9.2 API

- `numpy.array()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.array.html
- `numpy.dot()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.dot.html

9.9.3 Articles

- Matrix (mathematics).
  https://en.wikipedia.org/wiki/Matrix_(mathematics)
- Matrix multiplication on Wikipedia.
  https://en.wikipedia.org/wiki/Matrix_multiplication
- Hadamard product (matrices) on Wikipedia.
  https://en.wikipedia.org/wiki/Hadamard_product_(matrices)
- Dot product on Wikipedia.
  https://en.wikipedia.org/wiki/Dot_product

9.10 Summary

In this tutorial, you discovered matrices in linear algebra and how to manipulate them in Python. Specifically, you learned:

- What a matrix is and how to define one in Python with NumPy.
- How to perform element-wise operations such as addition, subtraction, and the Hadamard product.
- How to multiply matrices together and the intuition behind the operation.

9.10.1 Next

In the next chapter you will discover a suite of different types of matrices.

Chapter 10

Types of Matrices

A lot of linear algebra is concerned with operations on vectors and matrices, and there are many different types of matrices. There are a few types of matrices that you may encounter again and again when getting started in linear algebra, particularity the parts of linear algebra relevant to machine learning. In this tutorial, you will discover a suite of different types of matrices from the field of linear algebra that you may encounter in machine learning. After completing this tutorial, you will know:

- Square, symmetric, triangular, and diagonal matrices that are much as their names suggest.
- Identity matrices that are all zero values except along the main diagonal where the values are 1.
- Orthogonal matrices that generalize the idea of perpendicular vectors and have useful computational properties.

Let's get started.

10.1 Tutorial Overview

This tutorial is divided into 6 parts to cover the main types of matrices; they are:

1. Square Matrix
2. Symmetric Matrix
3. Triangular Matrix
4. Diagonal Matrix
5. Identity Matrix
6. Orthogonal Matrix

10.2 Square Matrix

A square matrix is a matrix where the number of rows (n) is equivalent to the number of columns (m).

$$n \equiv m \tag{10.1}$$

The square matrix is contrasted with the rectangular matrix where the number of rows and columns are not equal. Given that the number of rows and columns match, the dimensions are usually denoted as n, e.g. $n \times n$. The size of the matrix is called the order, so an order 4 square matrix is 4×4. The vector of values along the diagonal of the matrix from the top left to the bottom right is called the main diagonal. Below is an example of an order 3 square matrix.

$$M = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{pmatrix} \tag{10.2}$$

Square matrices are readily added and multiplied together and are the basis of many simple linear transformations, such as rotations (as in the rotations of images).

10.3 Symmetric Matrix

A symmetric matrix is a type of square matrix where the top-right triangle is the same as the bottom-left triangle.

> It is no exaggeration to say that symmetric matrices S are the most important matrices the world will ever see — in the theory of linear algebra and also in the applications.
>
> — Page 338, *Introduction to Linear Algebra*, Fifth Edition, 2016.

To be symmetric, the axis of symmetry is always the main diagonal of the matrix, from the top left to the bottom right. Below is an example of a 5×5 symmetric matrix.

$$M = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 1 & 2 & 3 & 4 \\ 3 & 2 & 1 & 2 & 3 \\ 4 & 3 & 2 & 1 & 2 \\ 5 & 4 & 3 & 2 & 1 \end{pmatrix} \tag{10.3}$$

A symmetric matrix is always square and equal to its own transpose. The transpose is an operation that flips the number of rows and columns. It is explained in more detail in the next lesson.

$$M = M^T \tag{10.4}$$

10.4 Triangular Matrix

A triangular matrix is a type of square matrix that has all values in the upper-right or lower-left of the matrix with the remaining elements filled with zero values. A triangular matrix with values only above the main diagonal is called an upper triangular matrix. Whereas, a triangular matrix with values only below the main diagonal is called a lower triangular matrix. Below is an example of a 3×3 upper triangular matrix.

$$M = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 2 & 3 \\ 0 & 0 & 3 \end{pmatrix} \tag{10.5}$$

Below is an example of a 3×3 lower triangular matrix.

$$M = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 2 & 3 \end{pmatrix} \tag{10.6}$$

NumPy provides functions to calculate a triangular matrix from an existing square matrix. The `tril()` function to calculate the lower triangular matrix from a given matrix and the `triu()` to calculate the upper triangular matrix from a given matrix The example below defines a 3×3 square matrix and calculates the lower and upper triangular matrix from it.

```
# triangular matrices
from numpy import array
from numpy import tril
from numpy import triu
# define square matrix
M = array([
  [1, 2, 3],
  [1, 2, 3],
  [1, 2, 3]])
print(M)
# lower triangular matrix
lower = tril(M)
print(lower)
# upper triangular matrix
upper = triu(M)
print(upper)
```

Listing 10.1: Example of creating a triangular matrices.

Running the example prints the defined matrix followed by the lower and upper triangular matrices.

```
[[1 2 3]
 [1 2 3]
 [1 2 3]]

[[1 0 0]
 [1 2 0]
 [1 2 3]]

[[1 2 3]
 [0 2 3]
```

```
[0 0 3]]
```

Listing 10.2: Sample output from creating triangular matrices.

10.5 Diagonal Matrix

A diagonal matrix is one where values outside of the main diagonal have a zero value, where the main diagonal is taken from the top left of the matrix to the bottom right. A diagonal matrix is often denoted with the variable D and may be represented as a full matrix or as a vector of values on the main diagonal.

> Diagonal matrices consist mostly of zeros and have non-zero entries only along the main diagonal.
>
> — Page 40, *Deep Learning*, 2016.

Below is an example of a 3×3 square diagonal matrix.

$$D = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix} \tag{10.7}$$

As a vector, it would be represented as:

$$d = \begin{pmatrix} d_{1,1} \\ d_{2,2} \\ d_{3,3} \end{pmatrix} \tag{10.8}$$

Or, with the specified scalar values:

$$d = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \tag{10.9}$$

A diagonal matrix does not have to be square. In the case of a rectangular matrix, the diagonal would cover the dimension with the smallest length; for example:

$$D = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 0 & 0 \end{pmatrix} \tag{10.10}$$

NumPy provides the function `diag()` that can create a diagonal matrix from an existing matrix, or transform a vector into a diagonal matrix. The example below defines a 3×3 square matrix, extracts the main diagonal as a vector, and then creates a diagonal matrix from the extracted vector.

```
# diagonal matrix
from numpy import array
from numpy import diag
# define square matrix
M = array([
  [1, 2, 3],
  [1, 2, 3],
  [1, 2, 3]])
print(M)
# extract diagonal vector
d = diag(M)
print(d)
# create diagonal matrix from vector
D = diag(d)
print(D)
```

Listing 10.3: Example of creating a diagonal matrix.

Running the example first prints the defined matrix, followed by the vector of the main diagonal and the diagonal matrix constructed from the vector.

```
[[1 2 3]
 [1 2 3]
 [1 2 3]]

[1 2 3]

[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

Listing 10.4: Sample output from creating a diagonal matrix.

10.6 Identity Matrix

An identity matrix is a square matrix that does not change a vector when multiplied. The values of an identity matrix are known. All of the scalar values along the main diagonal (top-left to bottom-right) have the value one, while all other values are zero.

> An identity matrix is a matrix that does not change any vector when we multiply that vector by that matrix.
>
> — Page 36, *Deep Learning*, 2016.

An identity matrix is often represented using the notation I or with the dimensionality I^n, where n is a subscript that indicates the dimensionality of the square identity matrix. In some notations, the identity may be referred to as the unit matrix, or U, to honor the one value it contains (this is different from a Unitary matrix). For example, an identity matrix with the size 3 or I^3 would be as follows:

$$I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{10.11}$$

In NumPy, an identity matrix can be created with a specific size using the `identity()` function. The example below creates an I^3 identity matrix.

```
# identity matrix
from numpy import identity
I = identity(3)
print(I)
```

Listing 10.5: Example of creating an identity matrix.

Running the example prints the created identity matrix.

```
[[ 1.  0.  0.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]
```

Listing 10.6: Sample output from creating an identity matrix.

Alone, the identity matrix is not that interesting, although it is a component in other import matrix operations, such as matrix inversion.

10.7 Orthogonal Matrix

Two vectors are orthogonal when their dot product equals zero. The length of each vector is 1 then the vectors are called orthonormal because they are both orthogonal and normalized.

$$v \cdot w = 0 \tag{10.12}$$

or

$$v \cdot w^T = 0 \tag{10.13}$$

This is intuitive when we consider that one line is orthogonal with another if it is perpendicular to it. An orthogonal matrix is a type of square matrix whose columns and rows are orthonormal unit vectors, e.g. perpendicular and have a length or magnitude of 1.

> An orthogonal matrix is a square matrix whose rows are mutually orthonormal and whose columns are mutually orthonormal
>
> — Page 41, *Deep Learning*, 2016.

An Orthogonal matrix is often denoted as uppercase Q.

> Multiplication by an orthogonal matrix preserves lengths.
>
> — Page 277, *No Bullshit Guide To Linear Algebra*, 2017.

The Orthogonal matrix is defined formally as follows:

$$Q^T \cdot Q = Q \cdot Q^T = I \tag{10.14}$$

Where Q is the orthogonal matrix, Q^T indicates the transpose of Q, and I is the identity matrix. A matrix is orthogonal if its transpose is equal to its inverse.

$$Q^T = Q^{-1} \tag{10.15}$$

Another equivalence for an orthogonal matrix is if the dot product of the matrix and itself equals the identity matrix.

$$Q \cdot Q^T = I \tag{10.16}$$

Orthogonal matrices are used a lot for linear transformations, such as reflections and permutations. A simple 2×2 orthogonal matrix is listed below, which is an example of a reflection matrix or coordinate reflection.

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{10.17}$$

The example below creates this orthogonal matrix and checks the above equivalences.

```
# orthogonal matrix
from numpy import array
from numpy.linalg import inv
# define orthogonal matrix
Q = array([
  [1, 0],
  [0, -1]])
print(Q)
# inverse equivalence
V = inv(Q)
print(Q.T)
print(V)
# identity equivalence
I = Q.dot(Q.T)
print(I)
```

Listing 10.7: Example of creating an orthogonal matrix.

Running the example first prints the orthogonal matrix, the inverse of the orthogonal matrix, and the transpose of the orthogonal matrix are then printed and are shown to be equivalent. Finally, the identity matrix is printed which is calculated from the dot product of the orthogonal matrix with its transpose.

```
[[ 1  0]
 [ 0 -1]]

[[ 1  0]
 [ 0 -1]]

[[ 1.  0.]
 [-0. -1.]]

[[1 0]
 [0 1]]
```

Listing 10.8: Sample output from creating an orthogonal matrix.

Note, sometimes a number close to zero can be represented as -0 due to the rounding of floating point precision. Just take it as 0.0. Orthogonal matrices are useful tools as they are computationally cheap and stable to calculate their inverse as simply their transpose.

10.8 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Modify each example using your own small contrived array data.
- Write your own functions for creating each matrix type.
- Research one example where each type of array was used in machine learning.

If you explore any of these extensions, I'd love to know.

10.9 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

10.9.1 Books

- Section 6.2 Special types of matrices. *No Bullshit Guide To Linear Algebra*, 2017. http://amzn.to/2k76D4
- *Introduction to Linear Algebra*, 2016. http://amzn.to/2j2J0g4
- Section 2.3 Identity and Inverse Matrices, *Deep Learning*, 2016. http://amzn.to/2B3MsuU
- Section 2.6 Special Kinds of Matrices and Vectors, *Deep Learning*, 2016. http://amzn.to/2B3MsuU

10.9.2 API

- `numpy.tril()` API. https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.tril.html
- `numpy.triu()` API. https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.triu.html
- `numpy.diag()` API. https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.diag.html
- `numpy.identity()` API. https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.identity.html

10.9.3 Articles

- Square matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Square_matrix
- Main diagonal on Wikipedia.
  https://en.wikipedia.org/wiki/Main_diagonal
- Symmetric matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Symmetric_matrix
- Triangular Matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Triangular_matrix
- Diagonal matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Diagonal_matrix
- Identity matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Identity_matrix
- Orthogonal matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Orthogonal_matrix

10.10 Summary

In this tutorial, you discovered a suite of different types of matrices from the field of linear algebra that you may encounter in machine learning. Specifically, you learned:

- Square, symmetric, triangular, and diagonal matrices that are much as their name suggests.
- Identity matrices that are all zero values except along the main diagonal where the values are 1.
- Orthogonal matrices that generalize the idea of perpendicular vectors and have useful computational properties.

10.10.1 Next

In the next chapter you will discover basic operations that you can perform on matrices.

Chapter 11

Matrix Operations

Matrix operations are used in the description of many machine learning algorithms. Some operations can be used directly to solve key equations, whereas others provide useful shorthand or foundation in the description and the use of more complex matrix operations. In this tutorial, you will discover important linear algebra matrix operations used in the description of machine learning methods. After completing this tutorial, you will know:

- The Transpose operation for flipping the dimensions of a matrix.
- The Inverse operations used in solving systems of linear equations.
- The Trace and Determinant operations used as shorthand notation in other matrix operations.

Let's get started.

11.1 Tutorial Overview

This tutorial is divided into 5 parts; they are:

1. Transpose
2. Inverse
3. Trace
4. Determinant
5. Rank

11.2 Transpose

A defined matrix can be transposed, which creates a new matrix with the number of columns and rows flipped. This is denoted by the superscript T next to the matrix A^T.

$$C = A^T \tag{11.1}$$

An invisible diagonal line can be drawn through the matrix from top left to bottom right on which the matrix can be flipped to give the transpose.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \tag{11.2}$$

$$A^T = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix} \tag{11.3}$$

The operation has no effect if the matrix is symmetrical, e.g. has the same number of columns and rows and the same values at the same locations on both sides of the invisible diagonal line.

> The columns of A^T are the rows of A.
>
> — Page 109, *Introduction to Linear Algebra*, Fifth Edition, 2016.

We can transpose a matrix in NumPy by calling the `T` attribute.

```
# transpose matrix
from numpy import array
# define matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# calculate transpose
C = A.T
print(C)
```

Listing 11.1: Example of creating a transpose of a matrix.

Running the example first prints the matrix as it is defined, then the transposed version.

```
[[1 2]
 [3 4]
 [5 6]]

[[1 3 5]
 [2 4 6]]
```

Listing 11.2: Sample output from creating a transpose of a matrix.

The transpose operation provides a short notation used as an element in many matrix operations.

11.3 Inverse

Matrix inversion is a process that finds another matrix that when multiplied with the matrix, results in an identity matrix. Given a matrix A, find matrix B, such that $AB = I^n$ or $BA = I^n$.

$$AB = BA = I^n \tag{11.4}$$

The operation of inverting a matrix is indicated by a -1 superscript next to the matrix; for example, A^{-1}. The result of the operation is referred to as the inverse of the original matrix; for example, B is the inverse of A.

$$B = A^{-1} \tag{11.5}$$

A matrix is invertible if there exists another matrix that results in the identity matrix, where not all matrices are invertible. A square matrix that is not invertible is referred to as singular.

> Whatever A does, A^{-1} undoes.
>
> — Page 83, *Introduction to Linear Algebra*, Fifth Edition, 2016.

The matrix inversion operation is not computed directly, but rather the inverted matrix is discovered through a numerical operation, where a suite of efficient methods may be used, often involving forms of matrix decomposition.

> However, A^1 is primarily useful as a theoretical tool, and should not actually be used in practice for most software applications.
>
> — Page 37, *Deep Learning*, 2016.

A matrix can be inverted in NumPy using the `inv()` function.

```
# invert matrix
from numpy import array
from numpy.linalg import inv
# define matrix
A = array([
  [1.0, 2.0],
  [3.0, 4.0]])
print(A)
# invert matrix
B = inv(A)
print(B)
# multiply A and B
I = A.dot(B)
print(I)
```

Listing 11.3: Example of creating the inverse of a matrix.

First, we define a small 2×2 matrix, then calculate the inverse of the matrix, and then confirm the inverse by multiplying it with the original matrix to give the identity matrix. Running the example prints the original, inverse, and identity matrices.

```
[[ 1.  2.]
 [ 3.  4.]]

[[-2.   1. ]
 [ 1.5 -0.5]]

[[  1.00000000e+00  0.00000000e+00]
 [  8.88178420e-16  1.00000000e+00]]
```

Listing 11.4: Sample output from creating the inverse of a matrix.

Note, your specific results may vary given differences in floating point precision on different hardware and software versions. Matrix inversion is used as an operation in solving systems of equations framed as matrix equations where we are interested in finding vectors of unknowns. A good example is in finding the vector of coefficient values in linear regression.

11.4 Trace

A trace of a square matrix is the sum of the values on the main diagonal of the matrix (top-left to bottom-right).

> The trace operator gives the sum of all of the diagonal entries of a matrix
>
> — Page 46, *Deep Learning*, 2016.

The operation of calculating a trace on a square matrix is described using the notation $tr(A)$ where A is the square matrix on which the operation is being performed.

$$tr(A) \tag{11.6}$$

The trace is calculated as the sum of the diagonal values; for example, in the case of a 3×3 matrix:

$$tr(A) = a_{1,1} + a_{2,2} + a_{3,3} \tag{11.7}$$

Or, using array notation:

$$tr(A) = A[0,0] + A[1,1] + A[2,2] \tag{11.8}$$

We can calculate the trace of a matrix in NumPy using the `trace()` function.

```
# matrix trace
from numpy import array
from numpy import trace
# define matrix
A = array([
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]])
print(A)
# calculate trace
B = trace(A)
print(B)
```

Listing 11.5: Example of creating the trace of a matrix.

First, a 3×3 matrix is created and then the trace is calculated. Running the example, first the array is printed and then the trace.

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

15
```

Listing 11.6: Sample output from creating the trace of a matrix.

Alone, the trace operation is not interesting, but it offers a simpler notation and it is used as an element in other key matrix operations.

11.5 Determinant

The determinant of a square matrix is a scalar representation of the volume of the matrix.

> The determinant describes the relative geometry of the vectors that make up the rows of the matrix. More specifically, the determinant of a matrix A tells you the volume of a box with sides given by rows of A.
>
> — Page 119, *No Bullshit Guide To Linear Algebra*, 2017.

It is denoted by the $det(A)$ notation or $|A|$, where A is the matrix on which we are calculating the determinant.

$$det(A) \tag{11.9}$$

The determinant of a square matrix is calculated from the elements of the matrix. More technically, the determinant is the product of all the eigenvalues of the matrix. Eigenvalues are introduced in the lessons on matrix factorization. The intuition for the determinant is that it describes the way a matrix will scale another matrix when they are multiplied together. For example, a determinant of 1 preserves the space of the other matrix. A determinant of 0 indicates that the matrix cannot be inverted.

> The determinant of a square matrix is a single number. [...] It tells immediately whether the matrix is invertible. The determinant is a zero when the matrix has no inverse.
>
> — Page 247, *Introduction to Linear Algebra*, Fifth Edition, 2016.

In NumPy, the determinant of a matrix can be calculated using the `det()` function.

```
# matrix determinant
from numpy import array
from numpy.linalg import det
# define matrix
A = array([
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]])
print(A)
# calculate determinant
B = det(A)
print(B)
```

Listing 11.7: Example of creating the determinant of a matrix.

First, a 3×3 matrix is defined, then the determinant of the matrix is calculated. Running the example first prints the defined matrix and then the determinant of the matrix.

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

-9.51619735393e-16
```

Listing 11.8: Sample output from creating the determinant of a matrix.

Like the trace operation, alone, the determinant operation is not interesting, but it offers a simpler notation and it is used as an element in other key matrix operations.

11.6 Rank

The rank of a matrix is the estimate of the number of linearly independent rows or columns in a matrix. The rank of a matrix M is often denoted as the function $rank()$.

$$rank(A) \tag{11.10}$$

An intuition for rank is to consider it the number of dimensions spanned by all of the vectors within a matrix. For example, a rank of 0 suggest all vectors span a point, a rank of 1 suggests all vectors span a line, a rank of 2 suggests all vectors span a two-dimensional plane. The rank is estimated numerically, often using a matrix decomposition method. A common approach is to use the Singular-Value Decomposition or SVD for short. NumPy provides the `matrix_rank()` function for calculating the rank of an array. It uses the SVD method to estimate the rank. The example below demonstrates calculating the rank of a matrix with scalar values and another vector with all zero values.

```
# vector rank
from numpy import array
from numpy.linalg import matrix_rank
# rank
v1 = array([1,2,3])
print(v1)
vr1 = matrix_rank(v1)
print(vr1)
# zero rank
v2 = array([0,0,0,0,0])
print(v2)
vr2 = matrix_rank(v2)
print(vr2)
```

Listing 11.9: Example of calculating the rank of vectors.

Running the example prints the first vector and its rank of 1, followed by the second zero vector and its rank of 0.

```
[1 2 3]

1

[0 0 0 0 0]

0
```

Listing 11.10: Sample output from creating the rank of vectors.

The next example makes it clear that the rank is not the number of dimensions of the matrix, but the number of linearly independent directions. Three examples of a 2×2 matrix are provided demonstrating matrices with rank 0, 1 and 2.

```
# matrix rank
from numpy import array
from numpy.linalg import matrix_rank
# rank 0
M0 = array([
  [0,0],
  [0,0]])
print(M0)
mr0 = matrix_rank(M0)
print(mr0)
# rank 1
M1 = array([
  [1,2],
  [1,2]])
print(M1)
mr1 = matrix_rank(M1)
print(mr1)
# rank 2
M2 = array([
  [1,2],
  [3,4]])
print(M2)
mr2 = matrix_rank(M2)
print(mr2)
```

Listing 11.11: Example of creating the rank of matrices.

Running the example first prints a zero 2×2 matrix followed by the rank, then a 2×2 with a rank 1 and finally a 2×2 matrix with a rank of 2.

```
[[0 0]
 [0 0]]

0

[[1 2]
 [1 2]]

1

[[1 2]
 [3 4]]

2
```

Listing 11.12: Sample output from creating the rank of matrices.

11.7 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Modify each example using your own small contrived array data.
- Write your own functions to implement one operation.
- Research one example where each operation was used in machine learning.

If you explore any of these extensions, I'd love to know.

11.8 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

11.8.1 Books

- Section 3.4 Determinants. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Section 3.5 Matrix inverse. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Section 5.1 The Properties of Determinants, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Section 2.3 Identity and Inverse Matrices, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU
- Section 2.11 The Determinant, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU
- Section 3.D Invertibility and Isomorphic Vector Spaces, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI
- Section 10.A Trace, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI
- Section 10.B Determinant, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI

11.8.2 API

- `numpy.ndarray.T` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.ndarray.T.html
- `numpy.linalg.inv()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.inv.html
- `numpy.trace()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.trace.html
- `numpy.linalg.det()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.det.html
- `numpy.linalg.matrix_rank()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.matrix_rank.html

11.8.3 Articles

- Transpose on Wikipedia.
  https://en.wikipedia.org/wiki/Transpose
- Invertible matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Invertible_matrix
- Trace (linear algebra) on Wikipedia.
  https://en.wikipedia.org/wiki/Trace_(linear_algebra)
- Determinant on Wikipedia.
  https://en.wikipedia.org/wiki/Determinant
- Rank (linear algebra) on Wikipedia.
  https://en.wikipedia.org/wiki/Rank_(linear_algebra)

11.9 Summary

In this tutorial, you discovered important linear algebra matrix operations used in the description of machine learning methods. Specifically, you learned:

- The Transpose operation for flipping the dimensions of a matrix.
- The Inverse operations used in solving systems of linear equations.
- The Trace and Determinant operations used as shorthand notation in other matrix operations.

11.9.1 Next

In the next chapter you will discover sparsity and sparse matrices.

Chapter 12

Sparse Matrices

Matrices that contain mostly zero values are called sparse, distinct from matrices where most of the values are non-zero, called dense. Large sparse matrices are common in general and especially in applied machine learning, such as in data that contains counts, data encodings that map categories to counts, and even in whole subfields of machine learning such as natural language processing. It is computationally expensive to represent and work with sparse matrices as though they are dense, and much improvement in performance can be achieved by using representations and operations that specifically handle the matrix sparsity. In this tutorial, you will discover sparse matrices, the issues they present, and how to work with them directly in Python. After completing this tutorial, you will know:

- That sparse matrices contain mostly zero values and are distinct from dense matrices.
- The myriad of areas where you are likely to encounter sparse matrices in data, data preparation, and sub-fields of machine learning.
- That there are many efficient ways to store and work with sparse matrices and SciPy provides implementations that you can use directly.

Let's get started.

12.1 Tutorial Overview

This tutorial is divided into 5 parts; they are:

1. Sparse Matrix
2. Problems with Sparsity
3. Sparse Matrices in Machine Learning
4. Working with Sparse Matrices
5. Sparse Matrices in Python

12.2 Sparse Matrix

A sparse matrix is a matrix that is comprised of mostly zero values. Sparse matrices are distinct from matrices with mostly non-zero values, which are referred to as dense matrices.

> A matrix is sparse if many of its coefficients are zero. The interest in sparsity arises because its exploitation can lead to enormous computational savings and because many large matrix problems that occur in practice are sparse.
>
> — Page 1, *Direct Methods for Sparse Matrices*, Second Edition, 2017.

The sparsity of a matrix can be quantified with a score, which is the number of zero values in the matrix divided by the total number of elements in the matrix.

$$\text{sparsity} = \frac{\text{count of non-zero elements}}{\text{total elements}} \tag{12.1}$$

Below is an example of a small 3×6 sparse matrix.

$$A = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 1 \\ 0 & 0 & 0 & 2 & 0 & 0 \end{pmatrix} \tag{12.2}$$

The example has 13 zero values of the 18 elements in the matrix, giving this matrix a sparsity score of 0.722 or about 72%.

12.3 Problems with Sparsity

Sparse matrices can cause problems with regards to space and time complexity.

12.3.1 Space Complexity

Very large matrices require a lot of memory, and some very large matrices that we wish to work with are sparse.

> In practice, most large matrices are sparse — almost all entries are zeros.
>
> — Page 465, *Introduction to Linear Algebra*, Fifth Edition, 2016.

An example of a very large matrix that is too large to be stored in memory is a link matrix that shows the links from one website to another. An example of a smaller sparse matrix might be a word or term occurrence matrix for words in one book against all known words in English. In both cases, the matrix contained is sparse with many more zero values than data values. The problem with representing these sparse matrices as dense matrices is that memory is required and must be allocated for each 32-bit or even 64-bit zero value in the matrix. This is clearly a waste of memory resources as those zero values do not contain any information.

12.3.2 Time Complexity

Assuming a very large sparse matrix can be fit into memory, we will want to perform operations on this matrix. Simply, if the matrix contains mostly zero-values, i.e. no data, then performing operations across this matrix may take a long time where the bulk of the computation performed will involve adding or multiplying zero values together.

> It is wasteful to use general methods of linear algebra on such problems, because most of the O(N^3) arithmetic operations devoted to solving the set of equations or inverting the matrix involve zero operands.
>
> — Page 75, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.

This is a problem of increased time complexity of matrix operations that increases with the size of the matrix. This problem is compounded when we consider that even trivial machine learning methods may require many operations on each row, column, or even across the entire matrix, resulting in vastly longer execution times.

12.4 Sparse Matrices in Machine Learning

Sparse matrices turn up a lot in applied machine learning. In this section, we will look at some common examples to motivate you to be aware of the issues of sparsity.

12.4.1 Data

Sparse matrices come up in some specific types of data, most notably observations that record the occurrence or count of an activity. Three examples include:

- Whether or not a user has watched a movie in a movie catalog.
- Whether or not a user has purchased a product in a product catalog.
- Count of the number of listens of a song in a song catalog.

12.4.2 Data Preparation

Sparse matrices come up in encoding schemes used in the preparation of data. Three common examples include:

- One hot encoding, used to represent categorical data as sparse binary vectors.
- Count encoding, used to represent the frequency of words in a vocabulary for a document
- TF-IDF encoding, used to represent normalized word frequency scores in a vocabulary.

12.4.3 Areas of Study

Some areas of study within machine learning must develop specialized methods to address sparsity directly as the input data is almost always sparse. Three examples include:

- Natural language processing for working with documents of text.
- Recommender systems for working with product usage within a catalog.
- Computer vision when working with images that contain lots of black pixels.

> If there are 100,000 words in the language model, then the feature vector has length 100,000, but for a short email message almost all the features will have count zero.
>
> — Page 22, *Artificial Intelligence: A Modern Approach*, Third Edition, 2009.

12.5 Working with Sparse Matrices

The solution to representing and working with sparse matrices is to use an alternate data structure to represent the sparse data. The zero values can be ignored and only the data or non-zero values in the sparse matrix need to be stored or acted upon. There are multiple data structures that can be used to efficiently construct a sparse matrix; three common examples are listed below.

- **Dictionary of Keys**. A dictionary is used where a row and column index is mapped to a value.
- **List of Lists**. Each row of the matrix is stored as a list, with each sublist containing the column index and the value.
- **Coordinate List**. A list of tuples is stored with each tuple containing the row index, column index, and the value.

There are also data structures that are more suitable for performing efficient operations; two commonly used examples are listed below.

- **Compressed Sparse Row**. The sparse matrix is represented using three one-dimensional arrays for the non-zero values, the extents of the rows, and the column indexes.
- **Compressed Sparse Column**. The same as the Compressed Sparse Row method except the column indices are compressed and read first before the row indices.

The Compressed Sparse Row, also called CSR for short, is often used to represent sparse matrices in machine learning given the efficient access and matrix multiplication that it supports.

12.6 Sparse Matrices in Python

SciPy provides tools for creating sparse matrices using multiple data structures, as well as tools for converting a dense matrix to a sparse matrix. Many linear algebra NumPy and SciPy functions that operate on NumPy arrays can transparently operate on SciPy sparse arrays. Further, machine learning libraries that use NumPy data structures can also operate transparently on SciPy sparse arrays, such as scikit-learn for general machine learning and Keras for deep learning.

A dense matrix stored in a NumPy array can be converted into a sparse matrix using the CSR representation by calling the `csr_matrix()` function. In the example below, we define a 3×6 sparse matrix as a dense array (e.g. an `ndarray`), convert it to a CSR sparse representation, and then convert it back to a dense array by calling the `todense()` function.

```
# sparse matrix
from numpy import array
from scipy.sparse import csr_matrix
# create dense matrix
A = array([
  [1, 0, 0, 1, 0, 0],
  [0, 0, 2, 0, 0, 1],
  [0, 0, 0, 2, 0, 0]])
print(A)
# convert to sparse matrix (CSR method)
S = csr_matrix(A)
print(S)
# reconstruct dense matrix
B = S.todense()
print(B)
```

Listing 12.1: Example of converting between dense and sparse matrices.

Running the example first prints the defined dense array, followed by the CSR representation, and then the reconstructed dense matrix.

```
[[1 0 0 1 0 0]
 [0 0 2 0 0 1]
 [0 0 0 2 0 0]]

  (0, 0) 1
  (0, 3) 1
  (1, 2) 2
  (1, 5) 1
  (2, 3) 2

[[1 0 0 1 0 0]
 [0 0 2 0 0 1]
 [0 0 0 2 0 0]]
```

Listing 12.2: Sample output from converting between dense and sparse matrices.

NumPy does not provide a function to calculate the sparsity of a matrix. Nevertheless, we can calculate it easily by first finding the density of the matrix and subtracting it from one. The number of non-zero elements in a NumPy array can be given by the `count_nonzero()` function and the total number of elements in the array can be given by the size property of the array. Array sparsity can therefore be calculated as

```
sparsity = 1.0 - count_nonzero(A) / A.size
```

Listing 12.3: Example of the manual sparsity calculation.

The example below demonstrates how to calculate the sparsity of an array.

```
# sparsity calculation
from numpy import array
from numpy import count_nonzero
# create dense matrix
A = array([
  [1, 0, 0, 1, 0, 0],
  [0, 0, 2, 0, 0, 1],
  [0, 0, 0, 2, 0, 0]])
print(A)
# calculate sparsity
sparsity = 1.0 - count_nonzero(A) / A.size
print(sparsity)
```

Listing 12.4: Example of calculating sparsity.

Running the example first prints the defined sparse matrix followed by the sparsity of the matrix.

```
[[1 0 0 1 0 0]
 [0 0 2 0 0 1]
 [0 0 0 2 0 0]]

0.7222222222222222
```

Listing 12.5: Sample output from calculating sparsity.

12.7 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Develop your own examples for converting a dense array to sparse and calculating sparsity.
- Develop an example for the each sparse matrix representation method supported by SciPy.
- Select one sparsity representation method and implement it yourself from scratch.

If you explore any of these extensions, I'd love to know.

12.8 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

12.8.1 Books

- *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Section 2.7 Sparse Linear Systems, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.
  http://amzn.to/2CF5atj
- *Artificial Intelligence: A Modern Approach*, Third Edition, 2009.
  http://amzn.to/2C4LhMW
- *Direct Methods for Sparse Matrices*, Second Edition, 2017.
  http://amzn.to/2DcsQVU

12.8.2 API

- Sparse matrices (`scipy.sparse`) API.
  https://docs.scipy.org/doc/scipy/reference/sparse.html
- `scipy.sparse.csr_matrix()` API.
  https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csr_matrix.html
- `numpy.count_nonzero()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.count_nonzero.html
- `numpy.ndarray.size` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.ndarray.size.html

12.8.3 Articles

- Sparse matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Sparse_matrix

12.9 Summary

In this tutorial, you discovered sparse matrices, the issues they present, and how to work with them directly in Python. Specifically, you learned:

- That sparse matrices contain mostly zero values and are distinct from dense matrices.
- The myriad of areas where you are likely to encounter sparse matrices in data, data preparation, and sub-fields of machine learning.
- That there are many efficient ways to store and work with sparse matrices and SciPy provides implementations that you can use directly.

12.9.1 Next

In the next chapter you will discover tensors and tensor arithmetic.

Chapter 13

Tensors and Tensor Arithmetic

In deep learning it is common to see a lot of discussion around tensors as the cornerstone data structure. Tensor even appears in name of Google's flagship machine learning library: *TensorFlow*. Tensors are a type of data structure used in linear algebra, and like vectors and matrices, you can calculate arithmetic operations with tensors. In this tutorial, you will discover what tensors are and how to manipulate them in Python with NumPy. After completing this tutorial, you will know:

- That tensors are a generalization of matrices and are represented using n-dimensional arrays.
- How to implement element-wise operations with tensors.
- How to perform the tensor product.

Let's get started.

13.1 Tutorial Overview

This tutorial is divided into 3 parts; they are:

1. What are Tensors
2. Tensors in Python
3. Tensor Arithmetic
4. Tensor Product

13.2 What are Tensors

A tensor is a generalization of vectors and matrices and is easily understood as a multidimensional array.

> In the general case, an array of numbers arranged on a regular grid with a variable number of axes is known as a tensor.

— Page 33, *Deep Learning*, 2016.

A vector is a one-dimensional or first order tensor and a matrix is a two-dimensional or second order tensor. Tensor notation is much like matrix notation with a capital letter representing a tensor and lowercase letters with subscript integers representing scalar values within the tensor. For example, below defines a $3 \times 3 \times 3$ three-dimensional tensor T with dimensions index as $t_{i,j,k}$.

$$T = \begin{pmatrix} t_{1,1,1} & t_{1,2,1} & t_{1,3,1} \\ t_{2,1,1} & t_{2,2,1} & t_{2,3,1} \\ t_{3,1,1} & t_{3,2,1} & t_{3,3,1} \end{pmatrix}, \begin{pmatrix} t_{1,1,2} & t_{1,2,2} & t_{1,3,2} \\ t_{2,1,2} & t_{2,2,2} & t_{2,3,2} \\ t_{3,1,2} & t_{3,2,2} & t_{3,3,2} \end{pmatrix}, \begin{pmatrix} t_{1,1,3} & t_{1,2,3} & t_{1,3,3} \\ t_{2,1,3} & t_{2,2,3} & t_{2,3,3} \\ t_{3,1,3} & t_{3,2,3} & t_{3,3,3} \end{pmatrix} \tag{13.1}$$

Many of the operations that can be performed with scalars, vectors, and matrices can be reformulated to be performed with tensors. As a tool, tensors and tensor algebra is widely used in the fields of physics and engineering. Some operations in machine learning such as the training and operation of deep learning models can be described in terms of tensors.

13.3 Tensors in Python

Like vectors and matrices, tensors can be represented in Python using the N-dimensional array (`ndarray`). A tensor can be defined in-line to the constructor of `array()` as a list of lists. The example below defines a $3 \times 3 \times 3$ tensor as a NumPy `ndarray`. Three dimensions is easier to wrap your head around. Here, we first define rows, then a list of rows stacked as columns, then a list of columns stacked as levels in a cube.

```
# create tensor
from numpy import array
T = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
print(T.shape)
print(T)
```

Listing 13.1: Example of creating a tensor.

Running the example first prints the shape of the tensor, then the values of the tensor itself. You can see that, at least in three-dimensions, the tensor is printed as a series of matrices, one for each layer. For this 3D tensor, axis 0 specifies the level (like height), axis 1 specifies the column, and axis 2 specifies the row.

```
(3, 3, 3)
[[[ 1  2  3]
  [ 4  5  6]
  [ 7  8  9]]

 [[11 12 13]
  [14 15 16]
  [17 18 19]]

 [[21 22 23]
  [24 25 26]
```

```
  [27 28 29]]]
```

Listing 13.2: Sample output from creating a tensor.

13.4 Tensor Arithmetic

As with matrices, we can perform element-wise arithmetic between tensors. In this section, we will work through the four main arithmetic operations.

13.4.1 Tensor Addition

The element-wise addition of two tensors with the same dimensions results in a new tensor with the same dimensions where each scalar value is the element-wise addition of the scalars in the parent tensors.

$$A = \begin{pmatrix} a_{1,1,1} & a_{1,2,1} & a_{1,3,1} \\ a_{2,1,1} & a_{2,2,1} & a_{2,3,1} \end{pmatrix}, \begin{pmatrix} a_{1,1,2} & a_{1,2,2} & a_{1,3,2} \\ a_{2,1,2} & a_{2,2,2} & a_{2,3,2} \end{pmatrix} \tag{13.2}$$

$$B = \begin{pmatrix} b_{1,1,1} & b_{1,2,1} & b_{1,3,1} \\ b_{2,1,1} & b_{2,2,1} & b_{2,3,1} \end{pmatrix}, \begin{pmatrix} b_{1,1,2} & b_{1,2,2} & b_{1,3,2} \\ b_{2,1,2} & b_{2,2,2} & b_{2,3,2} \end{pmatrix} \tag{13.3}$$

$$C = A + B \tag{13.4}$$

$$C = \begin{pmatrix} a_{1,1,1} + b_{1,1,1} & a_{1,2,1} + b_{1,2,1} & a_{1,3,1} + b_{1,3,1} \\ a_{2,1,1} + b_{2,1,1} & a_{2,2,1} + b_{2,2,1} & a_{2,3,1} + b_{2,3,1} \end{pmatrix}, \begin{pmatrix} a_{1,1,2} + b_{1,1,2} & a_{1,2,2} + b_{1,2,2} & a_{1,3,2} + b_{1,3,2} \\ a_{2,1,2} + b_{2,1,2} & a_{2,2,2} + b_{2,2,2} & a_{2,3,2} + b_{2,3,2} \end{pmatrix} \tag{13.5}$$

In NumPy, we can add tensors directly by adding arrays.

```
# tensor addition
from numpy import array
# define first tensor
A = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# define second tensor
B = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# add tensors
C = A + B
print(C)
```

Listing 13.3: Example of adding tensors.

Running the example prints the addition of the two parent tensors.

```
[[[ 2  4  6]
  [ 8 10 12]
  [14 16 18]]

 [[22 24 26]
  [28 30 32]
  [34 36 38]]

 [[42 44 46]
  [48 50 52]
  [54 56 58]]]
```

Listing 13.4: Sample output from adding tensors.

13.4.2 Tensor Subtraction

The element-wise subtraction of one tensor from another tensor with the same dimensions results in a new tensor with the same dimensions where each scalar value is the element-wise subtraction of the scalars in the parent tensors.

$$A = \begin{pmatrix} a_{1,1,1} & a_{1,2,1} & a_{1,3,1} \\ a_{2,1,1} & a_{2,2,1} & a_{2,3,1} \end{pmatrix}, \begin{pmatrix} a_{1,1,2} & a_{1,2,2} & a_{1,3,2} \\ a_{2,1,2} & a_{2,2,2} & a_{2,3,2} \end{pmatrix} \tag{13.6}$$

$$B = \begin{pmatrix} b_{1,1,1} & b_{1,2,1} & b_{1,3,1} \\ b_{2,1,1} & b_{2,2,1} & b_{2,3,1} \end{pmatrix}, \begin{pmatrix} b_{1,1,2} & b_{1,2,2} & b_{1,3,2} \\ b_{2,1,2} & b_{2,2,2} & b_{2,3,2} \end{pmatrix} \tag{13.7}$$

$$C = A - B \tag{13.8}$$

$$C = \begin{pmatrix} a_{1,1,1} - b_{1,1,1} & a_{1,2,1} - b_{1,2,1} & a_{1,3,1} - b_{1,3,1} \\ a_{2,1,1} - b_{2,1,1} & a_{2,2,1} - b_{2,2,1} & a_{2,3,1} - b_{2,3,1} \end{pmatrix}, \begin{pmatrix} a_{1,1,2} - b_{1,1,2} & a_{1,2,2} - b_{1,2,2} & a_{1,3,2} - b_{1,3,2} \\ a_{2,1,2} - b_{2,1,2} & a_{2,2,2} - b_{2,2,2} & a_{2,3,2} - b_{2,3,2} \end{pmatrix} \tag{13.9}$$

In NumPy, we can subtract tensors directly by subtracting arrays.

```
# tensor subtraction
from numpy import array
# define first tensor
A = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# define second tensor
B = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# subtract tensors
C = A - B
print(C)
```

Listing 13.5: Example of subtracting tensors.

Running the example prints the result of subtracting the first tensor from the second.

```
[[[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]]
```

Listing 13.6: Sample output from subtracting tensors.

13.4.3 Tensor Hadamard Product

The element-wise multiplication of one tensor with another tensor with the same dimensions results in a new tensor with the same dimensions where each scalar value is the element-wise multiplication of the scalars in the parent tensors. As with matrices, the operation is referred to as the Hadamard Product to differentiate it from tensor multiplication. Here, we will use the $\circ$ operator to indicate the Hadamard product operation between tensors.

$$A = \begin{pmatrix} a_{1,1,1} & a_{1,2,1} & a_{1,3,1} \\ a_{2,1,1} & a_{2,2,1} & a_{2,3,1} \end{pmatrix}, \begin{pmatrix} a_{1,1,2} & a_{1,2,2} & a_{1,3,2} \\ a_{2,1,2} & a_{2,2,2} & a_{2,3,2} \end{pmatrix} \tag{13.10}$$

$$B = \begin{pmatrix} b_{1,1,1} & b_{1,2,1} & b_{1,3,1} \\ b_{2,1,1} & b_{2,2,1} & b_{2,3,1} \end{pmatrix}, \begin{pmatrix} b_{1,1,2} & b_{1,2,2} & b_{1,3,2} \\ b_{2,1,2} & b_{2,2,2} & b_{2,3,2} \end{pmatrix} \tag{13.11}$$

$$C = A \circ B \tag{13.12}$$

$$C = \begin{pmatrix} a_{1,1,1} \times b_{1,1,1} & a_{1,2,1} \times b_{1,2,1} & a_{1,3,1} \times b_{1,3,1} \\ a_{2,1,1} \times b_{2,1,1} & a_{2,2,1} \times b_{2,2,1} & a_{2,3,1} \times b_{2,3,1} \end{pmatrix}, \begin{pmatrix} a_{1,1,2} \times b_{1,1,2} & a_{1,2,2} \times b_{1,2,2} & a_{1,3,2} \times b_{1,3,2} \\ a_{2,1,2} \times b_{2,1,2} & a_{2,2,2} \times b_{2,2,2} & a_{2,3,2} \times b_{2,3,2} \end{pmatrix} \tag{13.13}$$

In NumPy, we can multiply tensors directly by multiplying arrays.

```
# tensor Hadamard product
from numpy import array
# define first tensor
A = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# define second tensor
B = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# multiply tensors
C = A * B
print(C)
```

Listing 13.7: Example of tensor Hadamard product.

Running the example prints the result of multiplying the tensors.

```
[[[  1   4   9]
  [ 16  25  36]
  [ 49  64  81]]

 [[121 144 169]
  [196 225 256]
  [289 324 361]]

 [[441 484 529]
  [576 625 676]
  [729 784 841]]]
```

Listing 13.8: Sample output from tensor Hadamard product.

13.4.4 Tensor Division

The element-wise division of one tensor with another tensor with the same dimensions results in a new tensor with the same dimensions where each scalar value is the element-wise division of the scalars in the parent tensors.

$$A = \begin{pmatrix} a_{1,1,1} & a_{1,2,1} & a_{1,3,1} \\ a_{2,1,1} & a_{2,2,1} & a_{2,3,1} \end{pmatrix}, \begin{pmatrix} a_{1,1,2} & a_{1,2,2} & a_{1,3,2} \\ a_{2,1,2} & a_{2,2,2} & a_{2,3,2} \end{pmatrix} \tag{13.14}$$

$$B = \begin{pmatrix} b_{1,1,1} & b_{1,2,1} & b_{1,3,1} \\ b_{2,1,1} & b_{2,2,1} & b_{2,3,1} \end{pmatrix}, \begin{pmatrix} b_{1,1,2} & b_{1,2,2} & b_{1,3,2} \\ b_{2,1,2} & b_{2,2,2} & b_{2,3,2} \end{pmatrix} \tag{13.15}$$

$$C = \frac{A}{B} \tag{13.16}$$

$$C = \begin{pmatrix} \frac{a_{1,1,1}}{b_{1,1,1}} & \frac{a_{1,2,1}}{b_{1,2,1}} & \frac{a_{1,3,1}}{b_{1,3,1}} \\ \frac{a_{2,1,1}}{b_{2,1,1}} & \frac{a_{2,2,1}}{b_{2,2,1}} & \frac{a_{2,3,1}}{b_{2,3,1}} \end{pmatrix}, \begin{pmatrix} \frac{a_{1,1,2}}{b_{1,1,2}} & \frac{a_{1,2,2}}{b_{1,2,2}} & \frac{a_{1,3,2}}{b_{1,3,2}} \\ \frac{a_{2,1,2}}{b_{2,1,2}} & \frac{a_{2,2,2}}{b_{2,2,2}} & \frac{a_{2,3,2}}{b_{2,3,2}} \end{pmatrix} \tag{13.17}$$

In NumPy, we can divide tensors directly by dividing arrays.

```
# tensor division
from numpy import array
# define first tensor
A = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# define second tensor
B = array([
  [[1,2,3],   [4,5,6],   [7,8,9]],
  [[11,12,13], [14,15,16], [17,18,19]],
  [[21,22,23], [24,25,26], [27,28,29]]])
# divide tensors
C = A / B
```

```
print(C)
```

Listing 13.9: Example of diving tensors.

Running the example prints the result of dividing the tensors.

```
[[[ 1.  1.  1.]
  [ 1.  1.  1.]
  [ 1.  1.  1.]]

 [[ 1.  1.  1.]
  [ 1.  1.  1.]
  [ 1.  1.  1.]]

 [[ 1.  1.  1.]
  [ 1.  1.  1.]
  [ 1.  1.  1.]]]
```

Listing 13.10: Sample output from dividing tensors.

13.5 Tensor Product

The tensor product operator is often denoted as a circle with a small x in the middle. We will denote it here as $\otimes$. Given a tensor A with q dimensions and tensor B with r dimensions, the product of these tensors will be a new tensor with the order of $q + r$ or, said another way, $q + r$ dimensions. The tensor product is not limited to tensors, but can also be performed on matrices and vectors, which can be a good place to practice in order to develop the intuition for higher dimensions. Let's take a look at the tensor product for vectors.

$$a = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \tag{13.18}$$

$$b = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \tag{13.19}$$

$$C = a \otimes b \tag{13.20}$$

$$C = \begin{pmatrix} a_1 \times \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \\ a_2 \times \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \end{pmatrix} \tag{13.21}$$

Or, unrolled:

$$C = \begin{pmatrix} a_1 \times b_1 & a_1 \times b_2 \\ a_2 \times b_1 & a_2 \times b_2 \end{pmatrix} \tag{13.22}$$

Let's take a look at the tensor product for matrices.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix} \tag{13.23}$$

$$B = \begin{pmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{pmatrix} \tag{13.24}$$

$$C = A \otimes B \tag{13.25}$$

$$C = \begin{pmatrix} a_{1,1} \times \begin{pmatrix} b_{1,1}, b_{1,2} \\ b_{2,1}, b_{2,2} \end{pmatrix} & a_{1,2} \times \begin{pmatrix} b_{1,1}, b_{1,2} \\ b_{2,1}, b_{2,2} \end{pmatrix} \\ a_{2,1} \times \begin{pmatrix} b_{1,1}, b_{1,2} \\ b_{2,1}, b_{2,2} \end{pmatrix} & a_{2,2} \times \begin{pmatrix} b_{1,1}, b_{1,2} \\ b_{2,1}, b_{2,2} \end{pmatrix} \end{pmatrix} \tag{13.26}$$

Or, unrolled:

$$C = \begin{pmatrix} a_{1,1} \times b_{1,1} & a_{1,1} \times b_{1,2} & a_{1,2} \times b_{1,1} & a_{1,2} \times b_{1,2} \\ a_{1,1} \times b_{2,1} & a_{1,1} \times b_{2,2} & a_{1,2} \times b_{2,1} & a_{1,2} \times b_{2,2} \\ a_{2,1} \times b_{1,1} & a_{2,1} \times b_{1,2} & a_{2,2} \times b_{1,1} & a_{2,2} \times b_{1,2} \\ a_{2,1} \times b_{2,1} & a_{2,1} \times b_{2,2} & a_{2,2} \times b_{2,1} & a_{2,2} \times b_{2,2} \end{pmatrix} \tag{13.27}$$

The tensor product can be implemented in NumPy using the `tensordot()` function. The function takes as arguments the two tensors to be multiplied and the axis on which to sum the products over, called the sum reduction. To calculate the tensor product, also called the tensor dot product in NumPy, the axis must be set to 0. In the example below, we define two order-1 tensors (vectors) with and calculate the tensor product.

```
# tensor product
from numpy import array
from numpy import tensordot
# define first vector
A = array([1,2])
# define second vector
B = array([3,4])
# calculate tensor product
C = tensordot(A, B, axes=0)
print(C)
```

Listing 13.11: Example of tensor product.

Running the example prints the result of the tensor product. The result is an order-2 tensor (matrix) with the lengths 2×2.

```
[[3 4]
 [6 8]]
```

Listing 13.12: Sample output from tensor product.

The tensor product is the most common form of tensor multiplication that you may encounter, but there are many other types of tensor multiplications that exist, such as the tensor dot product and the tensor contraction.

13.6 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Update each example using your own small contrived tensor array data.
- Implement three other types of tensor multiplication not covered in this tutorial with small vector or matrix data.
- Write your own functions to implement each tensor arithmetic operation.

If you explore any of these extensions, I'd love to know.

13.7 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

13.7.1 Books

- *A Student's Guide to Vectors and Tensors*, 2011.
  http://amzn.to/2kmUvvF
- Chapter 12, Special Topics, *Matrix Computations*, 2012.
  http://amzn.to/2B9xnLD
- *Tensor Algebra and Tensor Analysis for Engineers*, 2015.
  http://amzn.to/2C6gzCu

13.7.2 API

- The N-dimensional array.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/arrays.ndarray.html
- `numpy.tensordot()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.tensordot.html

13.7.3 Articles

- Tensor algebra on Wikipedia.
  https://en.wikipedia.org/wiki/Tensor_algebra
- Tensor on Wikipedia.
  https://en.wikipedia.org/wiki/Tensor
- Tensor product on Wikipedia.
  https://en.wikipedia.org/wiki/Tensor_product
- Outer product on Wikipedia.
  https://en.wikipedia.org/wiki/Outer_product

13.8 Summary

In this tutorial, you discovered what tensors are and how to manipulate them in Python with NumPy. Specifically, you learned:

- That tensors are a generalization of matrices and are represented using n-dimensional arrays.
- How to implement element-wise operations with tensors.
- How to perform the tensor product.

13.8.1 Next

This was the end of the part on matrices, next is the part on matrix factorization, starting with a gentle introduction to matrix decomposition methods.

Part V

Factorization

Chapter 14

Matrix Decompositions

Many complex matrix operations cannot be solved efficiently or with stability using the limited precision of computers. Matrix decompositions are methods that reduce a matrix into constituent parts that make it easier to calculate more complex matrix operations. Matrix decomposition methods, also called matrix factorization methods, are a foundation of linear algebra in computers, even for basic operations such as solving systems of linear equations, calculating the inverse, and calculating the determinant of a matrix. In this tutorial, you will discover matrix decompositions and how to calculate them in Python. After completing this tutorial, you will know:

- What a matrix decomposition is and why these types of operations are important.
- How to calculate an LU and QR matrix decompositions in Python.
- How to calculate a Cholesky matrix decomposition in Python.

Let's get started.

14.1 Tutorial Overview

This tutorial is divided into 3 parts; they are:

1. What is a Matrix Decomposition
2. LU Decomposition
3. QR Decomposition
4. Cholesky Decomposition

14.2 What is a Matrix Decomposition

A matrix decomposition is a way of reducing a matrix into its constituent parts. It is an approach that can simplify more complex matrix operations that can be performed on the decomposed matrix rather than on the original matrix itself. A common analogy for matrix decomposition is the factoring of numbers, such as the factoring of 10 into 2×5. For this reason, matrix decomposition is also called matrix factorization. Like factoring real values, there are

many ways to decompose a matrix, hence there are a range of different matrix decomposition techniques. Two simple and widely used matrix decomposition methods are the LU matrix decomposition and the QR matrix decomposition. Next, we will take a closer look at each of these methods.

14.3 LU Decomposition

The LU decomposition is for square matrices and decomposes a matrix into L and U components.

$$A = L \cdot U \tag{14.1}$$

Or, without the dot notation.

$$A = LU \tag{14.2}$$

Where A is the square matrix that we wish to decompose, L is the lower triangle matrix and U is the upper triangle matrix.

> The factors L and U are triangular matrices. The factorization that comes from elimination is $A = LU$.
>
> — Page 97, *Introduction to Linear Algebra*, Fifth Edition, 2016.

The LU decomposition is found using an iterative numerical process and can fail for those matrices that cannot be decomposed or decomposed easily. A variation of this decomposition that is numerically more stable to solve in practice is called the LUP decomposition, or the LU decomposition with partial pivoting.

$$A = L \cdot U \cdot P \tag{14.3}$$

The rows of the parent matrix are re-ordered to simplify the decomposition process and the additional P matrix specifies a way to permute the result or return the result to the original order. There are also other variations of the LU. The LU decomposition is often used to simplify the solving of systems of linear equations, such as finding the coefficients in a linear regression, as well as in calculating the determinant and inverse of a matrix.

The LU decomposition can be implemented in Python with the `lu()` function. More specifically, this function calculates an LPU decomposition. The example below first defines a 3×3 square matrix. The LU decomposition is calculated, then the original matrix is reconstructed from the components.

```
# LU decomposition
from numpy import array
from scipy.linalg import lu
# define a square matrix
A = array([
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]])
print(A)
# factorize
```

```
P, L, U = lu(A)
print(P)
print(L)
print(U)
# reconstruct
B = P.dot(L).dot(U)
print(B)
```

Listing 14.1: Example of calculating an LU decomposition.

Running the example first prints the defined 3×3 matrix, then the P, L, and U components of the decomposition, then finally the original matrix is reconstructed.

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

[[ 0.  1.  0.]
 [ 0.  0.  1.]
 [ 1.  0.  0.]]

[[ 1.         0.         0.        ]
 [ 0.14285714 1.         0.        ]
 [ 0.57142857 0.5        1.        ]]

[[ 7.00000000e+00  8.00000000e+00  9.00000000e+00]
 [  0.00000000e+00  8.57142857e-01  1.71428571e+00]
 [  0.00000000e+00  0.00000000e+00 -1.58603289e-16]]

[[ 1.  2.  3.]
 [ 4.  5.  6.]
 [ 7.  8.  9.]]
```

Listing 14.2: Sample output from calculating an LU decomposition.

14.4 QR Decomposition

The QR decomposition is for $n \times m$ matrices (not limited to square matrices) and decomposes a matrix into Q and R components.

$$A = Q \cdot R \tag{14.4}$$

Or, without the dot notation.

$$A = QR \tag{14.5}$$

Where A is the matrix that we wish to decompose, Q a matrix with the size $m \times m$, and R is an upper triangle matrix with the size $m \times n$. The QR decomposition is found using an iterative numerical method that can fail for those matrices that cannot be decomposed, or decomposed easily. Like the LU decomposition, the QR decomposition is often used to solve systems of linear equations, although is not limited to square matrices.

The QR decomposition can be implemented in NumPy using the `qr()` function. By default, the function returns the Q and R matrices with smaller or *reduced* dimensions that is more

economical. We can change this to return the expected sizes of $m \times m$ for Q and $m \times n$ for R by specifying the mode argument as `'complete'`, although this is not required for most applications. The example below defines a 3×2 matrix, calculates the QR decomposition, then reconstructs the original matrix from the decomposed elements.

```
# QR decomposition
from numpy import array
from numpy.linalg import qr
# define rectangular matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# factorize
Q, R = qr(A, 'complete')
print(Q)
print(R)
# reconstruct
B = Q.dot(R)
print(B)
```

Listing 14.3: Example of calculating an QR decomposition.

Running the example first prints the defined 3×2 matrix, then the Q and R elements, then finally the reconstructed matrix that matches what we started with.

```
[[1 2]
 [3 4]
 [5 6]]

[[-0.16903085 0.89708523 0.40824829]
 [-0.50709255 0.27602622 -0.81649658]
 [-0.84515425 -0.34503278 0.40824829]]

[[-5.91607978 -7.43735744]
 [ 0.         0.82807867]
 [ 0.         0.        ]]

[[ 1. 2.]
 [ 3. 4.]
 [ 5. 6.]]
```

Listing 14.4: Sample output from calculating an QR decomposition.

14.5 Cholesky Decomposition

The Cholesky decomposition is for square symmetric matrices where all values are greater than zero, so-called positive definite matrices. For our interests in machine learning, we will focus on the Cholesky decomposition for real-valued matrices and ignore the cases when working with complex numbers. The decomposition is defined as follows:

$$A = L \cdot L^T \tag{14.6}$$

Or without the dot notation:

$$A = LL^T \tag{14.7}$$

Where A is the matrix being decomposed, L is the lower triangular matrix and L^T is the transpose of L. The decompose can also be written as the product of the upper triangular matrix, for example:

$$A = U^T \cdot U \tag{14.8}$$

Where U is the upper triangular matrix. The Cholesky decomposition is used for solving linear least squares for linear regression, as well as simulation and optimization methods. When decomposing symmetric matrices, the Cholesky decomposition is nearly twice as efficient as the LU decomposition and should be preferred in these cases.

> While symmetric, positive definite matrices are rather special, they occur quite frequently in some applications, so their special factorization, called Cholesky decomposition, is good to know about. When you can use it, Cholesky decomposition is about a factor of two faster than alternative methods for solving linear equations.
>
> — Page 100, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.

The Cholesky decomposition can be implemented in NumPy by calling the `cholesky()` function. The function only returns L as we can easily access the L transpose as needed. The example below defines a 3×3 symmetric and positive definite matrix and calculates the Cholesky decomposition, then the original matrix is reconstructed.

```
# Cholesky decomposition
from numpy import array
from numpy.linalg import cholesky
# define symmetrical matrix
A = array([
  [2, 1, 1],
  [1, 2, 1],
  [1, 1, 2]])
print(A)
# factorize
L = cholesky(A)
print(L)
# reconstruct
B = L.dot(L.T)
print(B)
```

Listing 14.5: Example of calculating an Cholesky decomposition.

Running the example first prints the symmetric matrix, then the lower triangular matrix from the decomposition followed by the reconstructed matrix.

```
[[2 1 1]
 [1 2 1]
 [1 1 2]]

[[ 1.41421356 0.         0.        ]
 [ 0.70710678 1.22474487 0.        ]
```

```
 [ 0.70710678 0.40824829 1.15470054]]

[[ 2.  1.  1.]
 [ 1.  2.  1.]
 [ 1.  1.  2.]]
```

Listing 14.6: Sample output from calculating an Cholesky decomposition.

14.6 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Write a summary of matrix decomposition to explain the principle to other students.
- Create one example using each operation with your own small array data.
- Search machine learning papers and find 1 example of each operation being used.

If you explore any of these extensions, I'd love to know.

14.7 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

14.7.1 Books

- Section 6.6 Matrix decompositions. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Lecture 7 QR Factorization, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2BI9kRH
- Section 2.3 LU Decomposition and Its Applications, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.
  http://amzn.to/2BezVEE
- Section 2.10 QR Decomposition, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.
  http://amzn.to/2BezVEE
- Section 2.9 Cholesky Decomposition, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.
  http://amzn.to/2BezVEE
- Lecture 23, Cholesky Decomposition, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2BI9kRH

14.7.2 API

- `scipy.linalg.lu()` API.
  https://docs.scipy.org/doc/scipy-1.0.0/reference/generated/scipy.linalg.lu.html
- `numpy.linalg.qr()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.qr.html
- `numpy.linalg.cholesky()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.cholesky.html

14.7.3 Articles

- Matrix decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/Matrix_decomposition
- LU decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/LU_decomposition
- QR Decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/QR_decomposition
- Cholesky decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/Cholesky_decomposition

14.8 Summary

In this tutorial, you discovered matrix decompositions and how to calculate them in Python. Specifically, you learned:

- What a matrix decomposition is and why these types of operations are important.
- How to calculate an LU and QR matrix decompositions in Python.
- How to calculate a Cholesky matrix decomposition in Python.

14.8.1 Next

In the next chapter you will discover the eigendecomposition, eigenvalues, and eigenvectors.

Chapter 15

Eigendecomposition

Matrix decompositions are a useful tool for reducing a matrix to their constituent parts in order to simplify a range of more complex operations. Perhaps the most used type of matrix decomposition is the eigendecomposition that decomposes a matrix into eigenvectors and eigenvalues. This decomposition also plays a role in methods used in machine learning, such as in the Principal Component Analysis method or PCA. In this tutorial, you will discover the eigendecomposition, eigenvectors, and eigenvalues in linear algebra. After completing this tutorial, you will know:

- What an eigendecomposition is and the role of eigenvectors and eigenvalues.
- How to calculate an eigendecomposition in Python with NumPy.
- How to confirm a vector is an eigenvector and how to reconstruct a matrix from eigenvectors and eigenvalues.

Let's get started.

15.1 Tutorial Overview

This tutorial is divided into 5 parts; they are:

1. Eigendecomposition of a Matrix
2. Eigenvectors and Eigenvalues
3. Calculation of Eigendecomposition
4. Confirm an Eigenvector and Eigenvalue
5. Reconstruct Matrix

15.2 Eigendecomposition of a Matrix

Eigendecomposition of a matrix is a type of decomposition that involves decomposing a square matrix into a set of eigenvectors and eigenvalues.

> One of the most widely used kinds of matrix decomposition is called eigendecomposition, in which we decompose a matrix into a set of eigenvectors and eigenvalues.
>
> — Page 42, *Deep Learning*, 2016.

A vector is an eigenvector of a matrix if it satisfies the following equation.

$$A \cdot v = \lambda \cdot v \tag{15.1}$$

This is called the eigenvalue equation, where A is the parent square matrix that we are decomposing, v is the eigenvector of the matrix, and λ is the lowercase Greek letter lambda and represents the eigenvalue scalar. Or without the dot notation.

$$Av = \lambda v \tag{15.2}$$

A matrix could have one eigenvector and eigenvalue for each dimension of the parent matrix. Not all square matrices can be decomposed into eigenvectors and eigenvalues, and some can only be decomposed in a way that requires complex numbers. The parent matrix can be shown to be a product of the eigenvectors and eigenvalues.

$$A = Q \cdot \Lambda \cdot Q^T \tag{15.3}$$

Or, without the dot notation.

$$A = Q\Lambda Q^T \tag{15.4}$$

Where Q is a matrix comprised of the eigenvectors, Λ is the uppercase Greek letter lambda and is the diagonal matrix comprised of the eigenvalues, and Q^T is the transpose of the matrix comprised of the eigenvectors.

> However, we often want to decompose matrices into their eigenvalues and eigenvectors. Doing so can help us to analyze certain properties of the matrix, much as decomposing an integer into its prime factors can help us understand the behavior of that integer.
>
> — Page 43, *Deep Learning*, 2016.

Eigen is not a name, e.g. the method is not named after "Eigen"; eigen (pronounced eye-gan) is a German word that means *own* or *innate*, as in belonging to the parent matrix. A decomposition operation does not result in a compression of the matrix; instead, it breaks it down into constituent parts to make certain operations on the matrix easier to perform. Like other matrix decomposition methods, Eigendecomposition is used as an element to simplify the calculation of other more complex matrix operations.

> Almost all vectors change direction, when they are multiplied by A. Certain exceptional vectors x are in the same direction as Ax. Those are the "eigenvectors". Multiply an eigenvector by A, and the vector Ax is the number λ times the original x. [...] The eigenvalue λ tells whether the special vector x is stretched or shrunk or reversed or left unchanged — when it is multiplied by A.
>
> — Page 289, *Introduction to Linear Algebra*, Fifth Edition, 2016.

Eigendecomposition can also be used to calculate the principal components of a matrix in the Principal Component Analysis method or PCA that can be used to reduce the dimensionality of data in machine learning.

15.3 Eigenvectors and Eigenvalues

Eigenvectors are unit vectors, which means that their length or magnitude is equal to 1.0. They are often referred as right vectors, which simply means a column vector (as opposed to a row vector or a left vector). A right-vector is a vector as we understand them. Eigenvalues are coefficients applied to eigenvectors that give the vectors their length or magnitude. For example, a negative eigenvalue may reverse the direction of the eigenvector as part of scaling it. A matrix that has only positive eigenvalues is referred to as a positive definite matrix, whereas if the eigenvalues are all negative, it is referred to as a negative definite matrix.

> Decomposing a matrix in terms of its eigenvalues and its eigenvectors gives valuable insights into the properties of the matrix. Certain matrix calculations, like computing the power of the matrix, become much easier when we use the eigendecomposition of the matrix.
>
> — Page 262, *No Bullshit Guide To Linear Algebra*, 2017.

15.4 Calculation of Eigendecomposition

An eigendecomposition is calculated on a square matrix using an efficient iterative algorithm, of which we will not go into the details. Often an eigenvalue is found first, then an eigenvector is found to solve the equation as a set of coefficients. The eigendecomposition can be calculated in NumPy using the `eig()` function. The example below first defines a 3×3 square matrix. The eigendecomposition is calculated on the matrix returning the eigenvalues and eigenvectors.

```
# eigendecomposition
from numpy import array
from numpy.linalg import eig
# define matrix
A = array([
  [1, 2, 3],
  [4, 5, 6],
   [7, 8, 9]])
print(A)
# factorize
values, vectors = eig(A)
```

```
print(values)
print(vectors)
```

Listing 15.1: Example of calculating an eigendecomposition.

Running the example first prints the defined matrix, followed by the eigenvalues and the eigenvectors. More specifically, the eigenvectors are the right-hand side eigenvectors and are normalized to unit length.

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

[  1.61168440e+01 -1.11684397e+00 -9.75918483e-16]

[[-0.23197069 -0.78583024 0.40824829]
 [-0.52532209 -0.08675134 -0.81649658]
 [-0.8186735  0.61232756 0.40824829]]
```

Listing 15.2: Sample output from calculating an eigendecomposition.

15.5 Confirm an Eigenvector and Eigenvalue

We can confirm that a vector is indeed an eigenvector of a matrix. We do this by multiplying the candidate eigenvector by the value vector and comparing the result with the eigenvalue. First, we will define a matrix, then calculate the eigenvalues and eigenvectors. We will then test whether the first vector and value are in fact an eigenvalue and eigenvector for the matrix. We know they are, but it is a good exercise.

The eigenvectors are returned as a matrix with the same dimensions as the parent matrix, where each column is an eigenvector, e.g. the first eigenvector is `vectors[:, 0]`. Eigenvalues are returned as a list, where value indices in the returned array are paired with eigenvectors by column index, e.g. the first eigenvalue at `values[0]` is paired with the first eigenvector at `vectors[:, 0]`.

```
# confirm eigenvector
from numpy import array
from numpy.linalg import eig
# define matrix
A = array([
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]])
# factorize
values, vectors = eig(A)
# confirm first eigenvector
B = A.dot(vectors[:, 0])
print(B)
C = vectors[:, 0] * values[0]
print(C)
```

Listing 15.3: Example of calculating a confirmation of an eigendecomposition.

The example multiplies the original matrix with the first eigenvector and compares it to the first eigenvector multiplied by the first eigenvalue. Running the example prints the results of these two multiplications that show the same resulting vector, as we would expect.

```
[ -3.73863537 -8.46653421 -13.19443305]

[ -3.73863537 -8.46653421 -13.19443305]
```

Listing 15.4: Sample output from calculating a confirmation of an eigendecomposition.

15.6 Reconstruct Matrix

We can reverse the process and reconstruct the original matrix given only the eigenvectors and eigenvalues. First, the list of eigenvectors must be taken together as a matrix, where each vector becomes a row. The eigenvalues need to be arranged into a diagonal matrix. The NumPy `diag()` function can be used for this. Next, we need to calculate the inverse of the eigenvector matrix, which we can achieve with the `inv()` NumPy function. Finally, these elements need to be multiplied together with the `dot()` function.

```
# reconstruct matrix
from numpy import diag
from numpy.linalg import inv
from numpy import array
from numpy.linalg import eig
# define matrix
A = array([
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]])
print(A)
# factorize
values, vectors = eig(A)
# create matrix from eigenvectors
Q = vectors
# create inverse of eigenvectors matrix
R = inv(Q)
# create diagonal matrix from eigenvalues
L = diag(values)
# reconstruct the original matrix
B = Q.dot(L).dot(R)
print(B)
```

Listing 15.5: Example of reconstructing a matrix from an eigendecomposition.

The example calculates the eigenvalues and eigenvectors again and uses them to reconstruct the original matrix. Running the example first prints the original matrix, then the matrix reconstructed from eigenvalues and eigenvectors matching the original matrix.

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

[[ 1.  2.  3.]
 [ 4.  5.  6.]
```

```
[ 7.  8.  9.]]
```

Listing 15.6: Sample output from reconstructing a matrix from an eigendecomposition.

15.7 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Develop an eigendecomposition and reconstruction of your own small contrived array data.
- List ten high-level operations that make use of the eigendecomposition.
- Implement the eigendecomposition operation from scratch for matrices defined as lists of lists.

If you explore any of these extensions, I'd love to know.

15.8 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

15.8.1 Books

- Section 6.1 Eigenvalues and eigenvectors. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Chapter 6 Eigenvalues and Eigenvectors, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Section 2.7 Eigendecomposition, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU
- Chapter 5 Eigenvalues, Eigenvectors, and Invariant Subspaces, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI
- Lecture 24, Eigenvalue Problems, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2BI9kRH

15.8.2 API

- `numpy.linalg.eig()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.eig.html
- `numpy.diag()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.diag.html

- `numpy.dot()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.dot.html
- `numpy.linalg.inv()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.inv.html

15.8.3 Articles

- eigen on Wiktionary.
  https://en.wiktionary.org/wiki/eigen
- Eigenvalues and eigenvectors.
  https://en.wikipedia.org/wiki/Eigenvalues_and_eigenvectors
- Eigendecomposition of a matrix.
  https://en.wikipedia.org/wiki/Eigendecomposition_of_a_matrix
- Eigenvalue algorithm.
  https://en.wikipedia.org/wiki/Eigenvalue_algorithm
- Matrix decomposition.
  https://en.wikipedia.org/wiki/Matrix_decomposition

15.9 Summary

In this tutorial, you discovered the eigendecomposition, eigenvectors, and eigenvalues in linear algebra. Specifically, you learned:

- What an eigendecomposition is and the role of eigenvectors and eigenvalues.
- How to calculate an eigendecomposition in Python with NumPy.
- How to confirm a vector is an eigenvector and how to reconstruct a matrix from eigenvectors and eigenvalues.

15.9.1 Next

In the next chapter you will discover the singular-value decomposition method.

Chapter 16

Singular Value Decomposition

Matrix decomposition, also known as matrix factorization, involves describing a given matrix using its constituent elements. Perhaps the most known and widely used matrix decomposition method is the Singular-Value Decomposition, or SVD. All matrices have an SVD, which makes it more stable than other methods, such as the eigendecomposition. As such, it is often used in a wide array of applications including compressing, denoising, and data reduction. In this tutorial, you will discover the Singular-Value Decomposition method for decomposing a matrix into its constituent elements. After completing this tutorial, you will know:

- What Singular-value decomposition is and what is involved.
- How to calculate an SVD and reconstruct a rectangular and square matrix from SVD elements.
- How to calculate the pseudoinverse and perform dimensionality reduction using the SVD.

Let's get started.

16.1 Tutorial Overview

This tutorial is divided into 5 parts; they are:

1. What is the Singular-Value Decomposition
2. Calculate Singular-Value Decomposition
3. Reconstruct Matrix
4. Pseudoinverse
5. Dimensionality Reduction

16.2 What is the Singular-Value Decomposition

The Singular-Value Decomposition, or SVD for short, is a matrix decomposition method for reducing a matrix to its constituent parts in order to make certain subsequent matrix calculations simpler. For the case of simplicity we will focus on the SVD for real-valued matrices and ignore the case for complex numbers.

$$A = U \cdot \Sigma \cdot V^T \tag{16.1}$$

Where A is the real $n \times m$ matrix that we wish to decompose, U is an $m \times m$ matrix, Σ represented by the uppercase Greek letter sigma) is an $m \times n$ diagonal matrix, and V^T is the V transpose of an $n \times n$ matrix where T is a superscript.

> The Singular Value Decomposition is a highlight of linear algebra.
>
> — Page 371, *Introduction to Linear Algebra*, 2016.

The diagonal values in the Σ matrix are known as the singular values of the original matrix A. The columns of the U matrix are called the left-singular vectors of A, and the columns of V are called the right-singular vectors of A. The SVD is calculated via iterative numerical methods. We will not go into the details of these methods. Every rectangular matrix has a singular value decomposition, although the resulting matrices may contain complex numbers and the limitations of floating point arithmetic may cause some matrices to fail to decompose neatly.

> The singular value decomposition (SVD) provides another way to factorize a matrix, into singular vectors and singular values. The SVD allows us to discover some of the same kind of information as the eigendecomposition. However, the SVD is more generally applicable.
>
> — Pages 44-45, *Deep Learning*, 2016.

The SVD is used widely both in the calculation of other matrix operations, such as matrix inverse, but also as a data reduction method in machine learning. SVD can also be used in least squares linear regression, image compression, and denoising data.

> The singular value decomposition (SVD) has numerous applications in statistics, machine learning, and computer science. Applying the SVD to a matrix is like looking inside it with X-ray vision...
>
> — Page 297, *No Bullshit Guide To Linear Algebra*, 2017.

16.3 Calculate Singular-Value Decomposition

The SVD can be calculated by calling the `svd()` function. The function takes a matrix and returns the U, Σ and V^T elements. The Σ diagonal matrix is returned as a vector of singular values. The V matrix is returned in a transposed form, e.g. V^T. The example below defines a 3×2 matrix and calculates the singular-value decomposition.

```
# singular-value decomposition
from numpy import array
from scipy.linalg import svd
# define a matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# factorize
U, s, V = svd(A)
print(U)
print(s)
print(V)
```

Listing 16.1: Example of calculating a singular-value decomposition.

Running the example first prints the defined 3×2 matrix, then the 3×3 U matrix, 2 element Σ vector, and 2×2 V^T matrix elements calculated from the decomposition.

```
[[1 2]
 [3 4]
 [5 6]]

[[-0.2298477  0.88346102 0.40824829]
 [-0.52474482 0.24078249 -0.81649658]
 [-0.81964194 -0.40189603 0.40824829]]

[ 9.52551809 0.51430058]

[[-0.61962948 -0.78489445]
 [-0.78489445 0.61962948]]
```

Listing 16.2: Sample output from calculating a singular-value decomposition.

16.4 Reconstruct Matrix

The original matrix can be reconstructed from the U, Σ, and V^T elements. The U, s, and V elements returned from the `svd()` cannot be multiplied directly. The s vector must be converted into a diagonal matrix using the `diag()` function. By default, this function will create a square matrix that is $m \times m$, relative to our original matrix. This causes a problem as the size of the matrices do not fit the rules of matrix multiplication, where the number of columns in a matrix must match the number of rows in the subsequent matrix. After creating the square Σ diagonal matrix, the sizes of the matrices are relative to the original $n \times m$ matrix that we are decomposing, as follows:

$$U(m \times m) \cdot \Sigma(m \times m) \cdot V^T(n \times n) \tag{16.2}$$

Where, in fact, we require:

$$U(m \times m) \cdot \Sigma(m \times n) \cdot V^T(n \times n) \tag{16.3}$$

We can achieve this by creating a new Σ matrix of all zero values that is $m \times n$ (e.g. more rows) and populate the first $n \times n$ part of the matrix with the square diagonal matrix calculated via `diag()`.

```
# reconstruct rectangular matrix from svd
from numpy import array
from numpy import diag
from numpy import zeros
from scipy.linalg import svd
# define matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# factorize
U, s, V = svd(A)
# create m x n Sigma matrix
Sigma = zeros((A.shape[0], A.shape[1]))
# populate Sigma with n x n diagonal matrix
Sigma[:A.shape[1], :A.shape[1]] = diag(s)
# reconstruct matrix
B = U.dot(Sigma.dot(V))
print(B)
```

Listing 16.3: Example of reconstructing a rectangular matrix from a SVD.

Running the example first prints the original matrix, then the matrix reconstructed from the SVD elements.

```
[[1 2]
 [3 4]
 [5 6]]

[[ 1.  2.]
 [ 3.  4.]
 [ 5.  6.]]
```

Listing 16.4: Sample output from reconstructing a rectangular matrix from a SVD.

The above complication with the Σ diagonal only exists with the case where m and n are not equal. The diagonal matrix can be used directly when reconstructing a square matrix, as follows.

```
# reconstruct square matrix from svd
from numpy import array
from numpy import diag
from scipy.linalg import svd
# define matrix
A = array([
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]])
print(A)
# factorize
U, s, V = svd(A)
# create n x n Sigma matrix
```

```
Sigma = diag(s)
# reconstruct matrix
B = U.dot(Sigma.dot(V))
print(B)
```

Listing 16.5: Example of reconstructing a square matrix from a SVD.

Running the example prints the original 3×3 matrix and the version reconstructed directly from the SVD elements.

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

[[ 1.  2.  3.]
 [ 4.  5.  6.]
 [ 7.  8.  9.]]
```

Listing 16.6: Sample output from reconstructing a square matrix from a SVD.

16.5 Pseudoinverse

The pseudoinverse is the generalization of the matrix inverse for square matrices to rectangular matrices where the number of rows and columns are not equal. It is also called the Moore-Penrose Inverse after two independent discoverers of the method or the Generalized Inverse.

> Matrix inversion is not defined for matrices that are not square. [...] When A has more columns than rows, then solving a linear equation using the pseudoinverse provides one of the many possible solutions.
>
> — Page 46, *Deep Learning*, 2016.

The pseudoinverse is denoted as A^+, where A is the matrix that is being inverted and $+$ is a superscript. The pseudoinverse is calculated using the singular value decomposition of A:

$$A^+ = V \cdot D^+ \cdot U^T \tag{16.4}$$

Or, without the dot notation:

$$A^+ = V \cdot D^+ \cdot U^T \tag{16.5}$$

Where A^+ is the pseudoinverse, D^+ is the pseudoinverse of the diagonal matrix Σ and V^T is the transpose of V^T. We can get U and V from the SVD operation.

$$A = U \cdot \Sigma \cdot V^T \tag{16.6}$$

The D^+ can be calculated by creating a diagonal matrix from Σ, calculating the reciprocal of each non-zero element in Σ, and taking the transpose if the original matrix was rectangular.

$$\Sigma = \begin{pmatrix} s_{1,1} & 0 & 0 \\ 0 & s_{2,2} & 0 \\ 0 & 0 & s_{3,3} \end{pmatrix} \tag{16.7}$$

$$D^{+} = \begin{pmatrix} \frac{1}{s_{1,1}} & 0 & 0 \\ 0 & \frac{1}{s_{2,2}} & 0 \\ 0 & 0 & \frac{1}{s_{3,3}} \end{pmatrix} \tag{16.8}$$

The pseudoinverse provides one way of solving the linear regression equation, specifically when there are more rows than there are columns, which is often the case. NumPy provides the function `pinv()` for calculating the pseudoinverse of a rectangular matrix. The example below defines a 4×2 matrix and calculates the pseudoinverse.

```
# pseudoinverse
from numpy import array
from numpy.linalg import pinv
# define matrix
A = array([
  [0.1, 0.2],
  [0.3, 0.4],
  [0.5, 0.6],
  [0.7, 0.8]])
print(A)
# calculate pseudoinverse
B = pinv(A)
print(B)
```

Listing 16.7: Example of calculating the pseudoinverse.

Running the example first prints the defined matrix, and then the calculated pseudoinverse.

```
[[ 0.1  0.2]
 [ 0.3  0.4]
 [ 0.5  0.6]
 [ 0.7  0.8]]

[[ -1.00000000e+01 -5.00000000e+00  9.04289323e-15  5.00000000e+00]
 [  8.50000000e+00  4.50000000e+00  5.00000000e-01 -3.50000000e+00]]
```

Listing 16.8: Sample output from calculating the pseudoinverse.

We can calculate the pseudoinverse manually via the SVD and compare the results to the `pinv()` function. First we must calculate the SVD. Next we must calculate the reciprocal of each value in the s array. Then the s array can be transformed into a diagonal matrix with an added row of zeros to make it rectangular. Finally, we can calculate the pseudoinverse from the elements. The specific implementation is:

$$A^{+} = V^{T} \cdot D^{T} \cdot U^{T} \tag{16.9}$$

The full example is listed below.

```
# pseudoinverse via svd
from numpy import array
from numpy.linalg import svd
from numpy import zeros
from numpy import diag
# define matrix
A = array([
  [0.1, 0.2],
```

```
  [0.3, 0.4],
  [0.5, 0.6],
  [0.7, 0.8]])
print(A)
# factorize
U, s, V = svd(A)
# reciprocals of s
d = 1.0 / s
# create m x n D matrix
D = zeros(A.shape)
# populate D with n x n diagonal matrix
D[:A.shape[1], :A.shape[1]] = diag(d)
# calculate pseudoinverse
B = V.T.dot(D.T).dot(U.T)
print(B)
```

Listing 16.9: Example of calculating the pseudoinverse via the SVD.

Running the example first prints the defined rectangular matrix and the pseudoinverse that matches the above results from the `pinv()` function.

```
[[ 0.1  0.2]
 [ 0.3  0.4]
 [ 0.5  0.6]
 [ 0.7  0.8]]

[[ -1.00000000e+01 -5.00000000e+00  9.04831765e-15  5.00000000e+00]
 [  8.50000000e+00  4.50000000e+00  5.00000000e-01 -3.50000000e+00]]
```

Listing 16.10: Sample output from calculating the pseudoinverse via the SVD.

16.6 Dimensionality Reduction

A popular application of SVD is for dimensionality reduction. Data with a large number of features, such as more features (columns) than observations (rows) may be reduced to a smaller subset of features that are most relevant to the prediction problem. The result is a matrix with a lower rank that is said to approximate the original matrix. To do this we can perform an SVD operation on the original data and select the top k largest singular values in Σ. These columns can be selected from Σ and the rows selected from V^T. An approximate B of the original vector A can then be reconstructed.

$$B = U \cdot \Sigma_k \cdot V_k^T \tag{16.10}$$

In natural language processing, this approach can be used on matrices of word occurrences or word frequencies in documents and is called Latent Semantic Analysis or Latent Semantic Indexing. In practice, we can retain and work with a descriptive subset of the data called T. This is a dense summary of the matrix or a projection.

$$T = U \cdot \Sigma_k \tag{16.11}$$

Further, this transform can be calculated and applied to the original matrix A as well as other similar matrices.

$$T = A \cdot V_k^T \tag{16.12}$$

The example below demonstrates data reduction with the SVD. First a 3×10 matrix is defined, with more columns than rows. The SVD is calculated and only the first two features are selected. The elements are recombined to give an accurate reproduction of the original matrix. Finally the transform is calculated two different ways.

```
# data reduction with svd
from numpy import array
from numpy import diag
from numpy import zeros
from scipy.linalg import svd
# define matrix
A = array([
  [1,2,3,4,5,6,7,8,9,10],
  [11,12,13,14,15,16,17,18,19,20],
  [21,22,23,24,25,26,27,28,29,30]])
print(A)
# factorize
U, s, V = svd(A)
# create m x n Sigma matrix
Sigma = zeros((A.shape[0], A.shape[1]))
# populate Sigma with n x n diagonal matrix
Sigma[:A.shape[0], :A.shape[0]] = diag(s)
# select
n_elements = 2
Sigma = Sigma[:, :n_elements]
V = V[:n_elements, :]
# reconstruct
B = U.dot(Sigma.dot(V))
print(B)
# transform
T = U.dot(Sigma)
print(T)
T = A.dot(V.T)
print(T)
```

Listing 16.11: Example of calculating data reduction with the SVD.

Running the example first prints the defined matrix then the reconstructed approximation, followed by two equivalent transforms of the original matrix.

```
[[ 1  2  3  4  5  6  7  8  9 10]
 [11 12 13 14 15 16 17 18 19 20]
 [21 22 23 24 25 26 27 28 29 30]]

[[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
 [ 11. 12. 13. 14. 15. 16. 17. 18. 19. 20.]
 [ 21. 22. 23. 24. 25. 26. 27. 28. 29. 30.]]

[[-18.52157747  6.47697214]
 [-49.81310011  1.91182038]
 [-81.10462276 -2.65333138]]
```

```
[[-18.52157747  6.47697214]
 [-49.81310011  1.91182038]
 [-81.10462276 -2.65333138]]
```

Listing 16.12: Sample output from calculating data reduction with the SVD.

The scikit-learn provides a `TruncatedSVD` class that implements this capability directly. The `TruncatedSVD` class can be created in which you must specify the number of desirable features or components to select, e.g. 2. Once created, you can fit the transform (e.g. calculate V_k^T) by calling the `fit()` function, then apply it to the original matrix by calling the `transform()` function. The result is the transform of A called T above. The example below demonstrates the `TruncatedSVD` class.

```
# svd data reduction in scikit-learn
from numpy import array
from sklearn.decomposition import TruncatedSVD
# define matrix
A = array([
  [1,2,3,4,5,6,7,8,9,10],
  [11,12,13,14,15,16,17,18,19,20],
  [21,22,23,24,25,26,27,28,29,30]])
print(A)
# create transform
svd = TruncatedSVD(n_components=2)
# fit transform
svd.fit(A)
# apply transform
result = svd.transform(A)
print(result)
```

Listing 16.13: Example of calculating data reduction with the SVD in scikit-learn.

Running the example first prints the defined matrix, followed by the transformed version of the matrix. We can see that the values match those calculated manually above, except for the sign on some values. We can expect there to be some instability when it comes to the sign given the nature of the calculations involved and the differences in the underlying libraries and methods used. This instability of sign should not be a problem in practice as long as the transform is trained for reuse.

```
[[ 1  2  3  4  5  6  7  8  9 10]
 [11 12 13 14 15 16 17 18 19 20]
 [21 22 23 24 25 26 27 28 29 30]]

[[ 18.52157747  6.47697214]
 [ 49.81310011  1.91182038]
 [ 81.10462276 -2.65333138]]
```

Listing 16.14: Sample output from calculating data reduction with the SVD in scikit-learn.

16.7 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Experiment with the SVD method on your own data.
- Research and list 10 applications of SVD in machine learning.
- Apply SVD as a data reduction technique on a real-world tabular dataset.

If you explore any of these extensions, I'd love to know.

16.8 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

16.8.1 Books

- Chapter 12, Singular-Value and Jordan Decompositions, *Linear Algebra and Matrix Analysis for Statistics*, 2014.
  http://amzn.to/2A9ceNv
- Chapter 4, The Singular Value Decomposition and Chapter 5, More on the SVD, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2kjEF4S
- Section 2.4 The Singular Value Decomposition, *Matrix Computations*, 2012.
  http://amzn.to/2B9xnLD
- Chapter 7 The Singular Value Decomposition (SVD), *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Section 2.8 Singular Value Decomposition, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU
- Section 7.D Polar Decomposition and Singular Value Decomposition, *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI
- Lecture 3 The Singular Value Decomposition, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2BI9kRH
- Section 2.6 Singular Value Decomposition, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.
  http://amzn.to/2BezVEE
- Section 2.9 The Moore-Penrose Pseudoinverse, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU

16.8.2 API

- `numpy.linalg.svd()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.svd.html
- `numpy.matrix.H` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.matrix.H.html
- `numpy.diag()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.diag.html
- `numpy.linalg.pinv()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.pinv.html
- `sklearn.decomposition.TruncatedSVD` API.
  http://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html

16.8.3 Articles

- Matrix decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/Matrix_decomposition
- Singular-value decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/Singular-value_decomposition
- Singular value on Wikipedia.
  https://en.wikipedia.org/wiki/Singular_value
- Moore-Penrose inverse on Wikipedia.
  https://en.wikipedia.org/wiki/Moore%E2%80%93Penrose_inverse
- Latent semantic analysis on Wikipedia.
  https://en.wikipedia.org/wiki/Latent_semantic_analysis

16.9 Summary

In this tutorial, you discovered the Singular-value decomposition method for decomposing a matrix into its constituent elements. Specifically, you learned:

- What Singular-value decomposition is and what is involved.
- How to calculate an SVD and reconstruct a rectangular and square matrix from SVD elements.
- How to calculate the pseudoinverse and perform dimensionality reduction using the SVD.

16.9.1 Next

This is the end of the part on matrix factorization. In the next part you will discover the intersection of statistics and linear algebra, starting with simple statistical calculations on vectors and matrices.

Part VI

Statistics

Chapter 17

Introduction to Multivariate Statistics

Fundamental statistics are useful tools in applied machine learning for a better understanding your data. They are also the tools that provide the foundation for more advanced linear algebra operations and machine learning methods, such as the covariance matrix and principal component analysis respectively. As such, it is important to have a strong grip on fundamental statistics in the context of linear algebra notation. In this tutorial, you will discover how fundamental statistical operations work and how to implement them using NumPy with notation and terminology from linear algebra.

After completing this tutorial, you will know:

- What the expected value, average, and mean are and how to calculate them.
- What the variance and standard deviation are and how to calculate them.
- What the covariance, correlation, and covariance matrix are and how to calculate them.

Let's get started.

17.1 Tutorial Overview

This tutorial is divided into 4 parts; they are:

1. Expected Value and Mean
2. Variance and Standard Deviation
3. Covariance and Correlation
4. Covariance Matrix

17.2 Expected Value and Mean

In probability, the average value of some random variable X is called the expected value or the expectation. The expected value uses the notation E with square brackets around the name of the variable; for example:

$$E[X] \tag{17.1}$$

It is calculated as the probability weighted sum of values that can be drawn.

$$E[X] = \sum x_1 \times p_1, x_2 \times p_2, x_3 \times p_3, \cdots, x_n \times p_n \tag{17.2}$$

In simple cases, such as the flipping of a coin or rolling a dice, the probability of each event is just as likely. Therefore, the expected value can be calculated as the sum of all values multiplied by the reciprocal of the number of values.

$$E[X] = \frac{1}{n} \times \sum x_1, x_2, x_3, \cdots, x_n \tag{17.3}$$

In statistics, the mean, or more technically the arithmetic mean or sample mean, can be estimated from a sample of examples drawn from the domain. It is confusing because mean, average, and expected value are used interchangeably. In the abstract, the mean is denoted by the lower case Greek letter mu μ and is calculated from the sample of observations, rather than all possible values.

$$\mu = \frac{1}{n} \times \sum x_1, x_2, x_3, \cdots, x_n \tag{17.4}$$

Or, written more compactly:

$$\mu = P(x) \times \sum x \tag{17.5}$$

Where x is the vector of observations and $P(x)$ is the calculated probability for each value. When calculated for a specific variable, such as x, the mean is denoted as a lower case variable name with a line above, called x-bar e.g. $\bar{x}$.

$$\bar{x} = \frac{1}{n} \times \sum_{i=1}^{n} x_i \tag{17.6}$$

The arithmetic mean can be calculated for a vector or matrix in NumPy by using the `mean()` function. The example below defines a 6-element vector and calculates the mean.

```
# vector mean
from numpy import array
from numpy import mean
# define vector
v = array([1,2,3,4,5,6])
print(v)
# calculate mean
result = mean(v)
print(result)
```

Listing 17.1: Example of calculating a vector mean.

Running the example first prints the defined vector and the mean of the values in the vector.

```
[1 2 3 4 5 6]

3.5
```

Listing 17.2: Sample output from calculating a vector mean.

The mean function can calculate the row or column means of a matrix by specifying the axis argument and the value 0 or 1 respectively. The example below defines a 2×6 matrix and calculates both column and row means.

```
# matrix means
from numpy import array
from numpy import mean
# define matrix
M = array([
  [1,2,3,4,5,6],
  [1,2,3,4,5,6]])
print(M)
# column means
col_mean = mean(M, axis=0)
print(col_mean)
# row means
row_mean = mean(M, axis=1)
print(row_mean)
```

Listing 17.3: Example of calculating matrix means.

Running the example first prints the defined matrix, then the calculated column and row mean values.

```
[[1 2 3 4 5 6]
 [1 2 3 4 5 6]]

[ 1.  2.  3.  4.  5.  6.]

[ 3.5 3.5]
```

Listing 17.4: Sample output from calculating matrix means.

17.3 Variance and Standard Deviation

In probability, the variance of some random variable X is a measure of how much values in the distribution vary on average with respect to the mean. The variance is denoted as the function $Var()$ on the variable.

$$Var[X] \tag{17.7}$$

Variance is calculated as the average squared difference of each value in the distribution from the expected value. Or the expected squared difference from the expected value.

$$Var[X] = E[(X - E[X])^2] \tag{17.8}$$

Assuming the expected value of the variable has been calculated ($E[X]$), the variance of the random variable can be calculated as the sum of the squared difference of each example from the expected value multiplied by the probability of that value.

$$Var[X] = \sum p(x_1) \times (x_1 - E[X])^2, p(x_2) \times (x_2 - E[X])^2, \cdots, p(x_n) \times (x_n - E[X])^2 \tag{17.9}$$

If the probability of each example in the distribution is equal, variance calculation can drop the individual probabilities and multiply the sum of squared differences by the reciprocal of the number of examples in the distribution.

$$Var[X] = \frac{1}{n} \times \sum (x_1 - E[X])^2, (x_2 - E[X])^2, \cdots, (x_n - E[X])^2 \tag{17.10}$$

In statistics, the variance can be estimated from a sample of examples drawn from the domain. In the abstract, the sample variance is denoted by the lower case sigma with a 2 superscript indicating the units are squared (e.g. σ^2), not that you must square the final value. The sum of the squared differences is multiplied by the reciprocal of the number of examples minus 1 to correct for a bias (bias is related to a deeper discussion on degrees of freedom and I refer you to references at the end of the lesson).

$$\sigma^2 = \frac{1}{n-1} \times \sum_{i=1}^{n} (x_i - \mu)^2 \tag{17.11}$$

In NumPy, the variance can be calculated for a vector or a matrix using the `var()` function. By default, the `var()` function calculates the population variance. To calculate the sample variance, you must set the `ddof` argument to the value 1. The example below defines a 6-element vector and calculates the sample variance.

```
# vector variance
from numpy import array
from numpy import var
# define vector
v = array([1,2,3,4,5,6])
print(v)
# calculate variance
result = var(v, ddof=1)
print(result)
```

Listing 17.5: Example of calculating a vector variance.

Running the example first prints the defined vector and then the calculated sample variance of the values in the vector.

```
[1 2 3 4 5 6]

3.5
```

Listing 17.6: Example of calculating a vector variance.

The var function can calculate the row or column variances of a matrix by specifying the axis argument and the value 0 or 1 respectively, the same as the mean function above. The example below defines a 2×6 matrix and calculates both column and row sample variances.

```
# matrix variances
from numpy import array
from numpy import var
# define matrix
M = array([
  [1,2,3,4,5,6],
  [1,2,3,4,5,6]])
print(M)
```

```
# column variances
col_var = var(M, ddof=1, axis=0)
print(col_var)
# row variances
row_var = var(M, ddof=1, axis=1)
print(row_var)
```

Listing 17.7: Example of calculating matrix variances.

Running the example first prints the defined matrix and then the column and row sample variance values.

```
[[1 2 3 4 5 6]
 [1 2 3 4 5 6]]

[ 0.  0.  0.  0.  0.  0.]

[ 3.5  3.5]
```

Listing 17.8: Sample output from calculating matrix variances.

The standard deviation is calculated as the square root of the variance and is denoted as lowercase s.

$$s = \sqrt{\sigma^2} \tag{17.12}$$

To keep with this notation, sometimes the variance is indicated as s^2, with 2 as a superscript, again showing that the units are squared. NumPy also provides a function for calculating the standard deviation directly via the `std()` function. As with the `var()` function, the `ddof` argument must be set to 1 to calculate the unbiased sample standard deviation and column and row standard deviations can be calculated by setting the axis argument to 0 and 1 respectively. The example below demonstrates how to calculate the sample standard deviation for the rows and columns of a matrix.

```
# matrix standard deviation
from numpy import array
from numpy import std
# define matrix
M = array([
  [1,2,3,4,5,6],
  [1,2,3,4,5,6]])
print(M)
# column standard deviations
col_std = std(M, ddof=1, axis=0)
print(col_std)
# row standard deviations
row_std = std(M, ddof=1, axis=1)
print(row_std)
```

Listing 17.9: Example of calculating matrix standard deviations.

Running the example first prints the defined matrix and then the column and row sample standard deviation values.

```
[[1 2 3 4 5 6]
 [1 2 3 4 5 6]]
```

```
[ 0.  0.  0.  0.  0.  0.]

[ 1.87082869 1.87082869]
```

Listing 17.10: Sample output from calculating matrix standard deviations.

17.4 Covariance and Correlation

In probability, covariance is the measure of the joint probability for two random variables. It describes how the two variables change together. It is denoted as the function $cov(X, Y)$, where X and Y are the two random variables being considered.

$$cov(X, Y) \tag{17.13}$$

Covariance is calculated as expected value or average of the product of the differences of each random variable from their expected values, where $E[X]$ is the expected value for X and $E[Y]$ is the expected value of y.

$$cov(X, Y) = E[(X - E[X] \times (Y - E[Y])] \tag{17.14}$$

Assuming the expected values for X and Y have been calculated, the covariance can be calculated as the sum of the difference of x values from their expected value multiplied by the difference of the y values from their expected values multiplied by the reciprocal of the number of examples in the population.

$$cov(X, Y) = \frac{1}{n} \times \sum(x - E[X]) \times (y - E[Y]) \tag{17.15}$$

In statistics, the sample covariance can be calculated in the same way, although with a bias correction, the same as with the variance.

$$cov(X, Y) = \frac{1}{n - 1} \times \sum(x - E[X]) \times (y - E[Y]) \tag{17.16}$$

The sign of the covariance can be interpreted as whether the two variables increase together (positive) or decrease together (negative). The magnitude of the covariance is not easily interpreted. A covariance value of zero indicates that both variables are completely independent. NumPy does not have a function to calculate the covariance between two variables directly. Instead, it has a function for calculating a covariance matrix called `cov()` that we can use to retrieve the covariance. By default, the `cov()`function will calculate the unbiased or sample covariance between the provided random variables.

The example below defines two vectors of equal length with one increasing and one decreasing. We would expect the covariance between these variables to be negative. We access just the covariance for the two variables as the $[0, 1]$ element of the square covariance matrix returned.

```
# vector covariance
from numpy import array
from numpy import cov
# define first vector
x = array([1,2,3,4,5,6,7,8,9])
```

```
print(x)
# define second covariance
y = array([9,8,7,6,5,4,3,2,1])
print(y)
# calculate covariance
Sigma = cov(x,y)[0,1]
print(Sigma)
```

Listing 17.11: Example of calculating a vector covariance.

Running the example first prints the two vectors followed by the covariance for the values in the two vectors. The value is negative, as we expected.

```
[1 2 3 4 5 6 7 8 9]
[9 8 7 6 5 4 3 2 1]

-7.5
```

Listing 17.12: Sample output from calculating vector covariance.

The covariance can be normalized to a score between -1 and 1 to make the magnitude interpretable by dividing it by the standard deviation of X and Y. The result is called the correlation of the variables, also called the Pearson correlation coefficient, named for the developer of the method.

$$r = \frac{cov(X,Y)}{s_X \times s_Y} \tag{17.17}$$

Where r is the correlation coefficient of X and Y, $cov(X,Y)$ is the sample covariance of X and Y and s_X and s_Y are the standard deviations of X and Y respectively. NumPy provides the `corrcoef()` function for calculating the correlation between two variables directly. Like `cov()`, it returns a matrix, in this case a correlation matrix. As with the results from `cov()` we can access just the correlation of interest from the [0,1] value from the returned squared matrix.

```
# vector correlation
from numpy import array
from numpy import corrcoef
# define first vector
x = array([1,2,3,4,5,6,7,8,9])
print(x)
# define second vector
y = array([9,8,7,6,5,4,3,2,1])
print(y)
# calculate correlation
corr = corrcoef(x,y)[0,1]
print(corr)
```

Listing 17.13: Example of calculating a vector correlation.

Running the example first prints the two defined vectors followed by the correlation coefficient. We can see that the vectors are maximally negatively correlated as we designed.

```
[1 2 3 4 5 6 7 8 9]
[9 8 7 6 5 4 3 2 1]

-1.0
```

Listing 17.14: Sample output from calculating vector correlation.

17.5 Covariance Matrix

The covariance matrix is a square and symmetric matrix that describes the covariance between two or more random variables. The diagonal of the covariance matrix are the variances of each of the random variables, as such it is often called the variance-covariance matrix. A covariance matrix is a generalization of the covariance of two variables and captures the way in which all variables in the dataset may change together. The covariance matrix is denoted as the uppercase Greek letter Sigma, e.g. Σ. The covariance for each pair of random variables is calculated as above.

$$\Sigma = E[(X - E[X] \times (Y - E[Y])] \tag{17.18}$$

Where:

$$\Sigma_{i,j} = cov(X_i, X_j) \tag{17.19}$$

And X is a matrix where each column represents a random variable. The covariance matrix provides a useful tool for separating the structured relationships in a matrix of random variables. This can be used to decorrelate variables or applied as a transform to other variables. It is a key element used in the Principal Component Analysis data reduction method, or PCA for short.

The covariance matrix can be calculated in NumPy using the `cov()` function. By default, this function will calculate the sample covariance matrix. The `cov()` function can be called with a single 2D array where each sub-array contains a feature (e.g. column). If this function is called with your data defined in a normal matrix format (rows then columns), then a transpose of the matrix will need to be provided to the function in order to correctly calculate the covariance of the columns. Below is an example that defines a dataset with 5 observations across 3 features and calculates the covariance matrix.

```
# covariance matrix
from numpy import array
from numpy import cov
# define matrix of observations
X = array([
  [1, 5, 8],
  [3, 5, 11],
  [2, 4, 9],
  [3, 6, 10],
  [1, 5, 10]])
print(X)
# calculate covariance matrix
Sigma = cov(X.T)
print(Sigma)
```

Listing 17.15: Example of calculating a covariance matrix.

Running the example first prints the defined dataset and then the calculated covariance matrix.

```
[[ 1  5  8]
 [ 3  5 11]
 [ 2  4  9]
 [ 3  6 10]
 [ 1  5 10]]

[[ 1.   0.25 0.75]
 [ 0.25 0.5  0.25]
 [ 0.75 0.25 1.3 ]]
```

Listing 17.16: Sample output from calculating a covariance matrix.

The covariance matrix is used widely in linear algebra and the intersection of linear algebra and statistics called multivariate analysis. We have only had a small taste in this chapter.

17.6 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Explore each example using your own small contrived array data.
- Load data from a CSV file and apply each operation to the data columns.
- Write your own functions to implement each statistical operation.

If you explore any of these extensions, I'd love to know.

17.7 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

17.7.1 Books

- *Applied Multivariate Statistical Analysis*, 2012.
  http://amzn.to/2AUcEc5
- *Applied Multivariate Statistical Analysis*, 2015.
  http://amzn.to/2AWIViz
- Chapter 12 Linear Algebra in Probability & Statistics, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Chapter 3, Probability and Information Theory, *Deep Learning*, 2016.
  http://amzn.to/2j4oKuP

17.7.2 API

- NumPy Statistics Functions.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/routines.statistics.html
- `numpy.mean()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.mean.html
- `numpy.var()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.var.html
- `numpy.std()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.std.html
- `numpy.cov()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.cov.html
- `numpy.corrcoef()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.corrcoef.html

17.7.3 Articles

- Expected value on Wikipedia.
  https://en.wikipedia.org/wiki/Expected_value
- Mean on Wikipedia.
  https://en.wikipedia.org/wiki/Mean
- Variance on Wikipedia.
  https://en.wikipedia.org/wiki/Variance
- Standard deviation on Wikipedia.
  https://en.wikipedia.org/wiki/Standard_deviation
- Covariance on Wikipedia.
  https://en.wikipedia.org/wiki/Covariance
- Sample mean and covariance.
  https://en.wikipedia.org/wiki/Sample_mean_and_covariance
- Pearson correlation coefficient.
  https://en.wikipedia.org/wiki/Pearson_correlation_coefficient
- Covariance matrix on Wikipedia.
  https://en.wikipedia.org/wiki/Covariance_matrix
- Estimation of covariance matrices on Wikipedia.
  https://en.wikipedia.org/wiki/Estimation_of_covariance_matrices

17.8 Summary

In this tutorial, you discovered how fundamental statistical operations work and how to implement them using NumPy with notation and terminology from linear algebra. Specifically, you learned:

- What the expected value, average, and mean are and how to calculate them in NumPy.
- What the variance and standard deviation are and how to calculate them in NumPy.
- What the covariance, correlation, and covariance matrix are and how to calculate them in NumPy.

17.8.1 Next

In the next chapter you will discover the principal component analysis method that makes use of the covariance matrix.

Chapter 18

Principal Component Analysis

An important machine learning method for dimensionality reduction is called Principal Component Analysis. It is a method that uses simple matrix operations from linear algebra and statistics to calculate a projection of the original data into the same number or fewer dimensions. In this tutorial, you will discover the Principal Component Analysis machine learning method for dimensionality reduction and how to implement it from scratch in Python. After completing this tutorial, you will know:

- The procedure for calculating the Principal Component Analysis and how to choose principal components.
- How to calculate the Principal Component Analysis from scratch in NumPy.
- How to calculate the Principal Component Analysis for reuse on more data in scikit-learn.

Let's get started.

18.1 Tutorial Overview

This tutorial is divided into 3 parts; they are:

1. What is Principal Component Analysis
2. Calculate Principal Component Analysis
3. Principal Component Analysis in scikit-learn

18.2 What is Principal Component Analysis

Principal Component Analysis, or PCA for short, is a method for reducing the dimensionality of data. It can be thought of as a projection method where data with m-columns (features) is projected into a subspace with m or fewer columns, whilst retaining the essence of the original data. The PCA method can be described and implemented using the tools of linear algebra.

PCA is an operation applied to a dataset, represented by an $n \times m$ matrix A that results in a projection of A which we will call B. Let's walk through the steps of this operation.

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{pmatrix} \tag{18.1}$$

$$B = PCA(A) \tag{18.2}$$

The first step is to calculate the mean values of each column.

$$M = mean(A) \tag{18.3}$$

Next, we need to center the values in each column by subtracting the mean column value.

$$C = A - M \tag{18.4}$$

The next step is to calculate the covariance matrix of the centered matrix C. Correlation is a normalized measure of the amount and direction (positive or negative) that two columns change together. Covariance is a generalized and unnormalized version of correlation across multiple columns. A covariance matrix is a calculation of covariance of a given matrix with covariance scores for every column with every other column, including itself.

$$V = cov(C) \tag{18.5}$$

Finally, we calculate the eigendecomposition of the covariance matrix V. This results in a list of eigenvalues and a list of eigenvectors.

$$values, vectors = eig(V) \tag{18.6}$$

The eigenvectors represent the directions or components for the reduced subspace of B, whereas the eigenvalues represent the magnitudes for the directions. The eigenvectors can be sorted by the eigenvalues in descending order to provide a ranking of the components or axes of the new subspace for A. If all eigenvalues have a similar value, then we know that the existing representation may already be reasonably compressed or dense and that the projection may offer little. If there are eigenvalues close to zero, they represent components or axes of B that may be discarded. A total of m or less components must be selected to comprise the chosen subspace. Ideally, we would select k eigenvectors, called principal components, that have the k largest eigenvalues.

$$B = select(values, vectors) \tag{18.7}$$

Other matrix decomposition methods can be used such as Singular-Value Decomposition, or SVD. As such, generally the values are referred to as singular values and the vectors of the subspace are referred to as principal components. Once chosen, data can be projected into the subspace via matrix multiplication.

$$P = B^T \cdot A \tag{18.8}$$

Where A is the original data that we wish to project, B^T is the transpose of the chosen principal components and P is the projection of A. This is called the covariance method for calculating the PCA, although there are alternative ways to calculate it.

18.3 Calculate Principal Component Analysis

There is no `pca()` function in NumPy, but we can easily calculate the Principal Component Analysis step-by-step using NumPy functions. The example below defines a small 3×2 matrix, centers the data in the matrix, calculates the covariance matrix of the centered data, and then the eigendecomposition of the covariance matrix. The eigenvectors and eigenvalues are taken as the principal components and singular values and used to project the original data.

```
# principal component analysis
from numpy import array
from numpy import mean
from numpy import cov
from numpy.linalg import eig
# define matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# column means
M = mean(A.T, axis=1)
# center columns by subtracting column means
C = A - M
# calculate covariance matrix of centered matrix
V = cov(C.T)
# factorize covariance matrix
values, vectors = eig(V)
print(vectors)
print(values)
# project data
P = vectors.T.dot(C.T)
print(P.T)
```

Listing 18.1: Example of calculating a PCA manually.

Running the example first prints the original matrix, then the eigenvectors and eigenvalues of the centered covariance matrix, followed finally by the projection of the original matrix. Interestingly, we can see that only the first eigenvector is required, suggesting that we could project our 3×2 matrix onto a 3×1 matrix with little loss.

```
[[1 2]
 [3 4]
 [5 6]]

[[ 0.70710678 -0.70710678]
 [ 0.70710678 0.70710678]]

[8. 0.]

[[-2.82842712 0.        ]
 [ 0.         0.        ]
 [ 2.82842712 0.        ]]
```

Listing 18.2: Sample output from calculating a PCA manually.

18.4 Principal Component Analysis in scikit-learn

We can calculate a Principal Component Analysis on a dataset using the `PCA()` class in the scikit-learn library. The benefit of this approach is that once the projection is calculated, it can be applied to new data again and again quite easily. When creating the class, the number of components can be specified as a parameter. The class is first fit on a dataset by calling the `fit()` function, and then the original dataset or other data can be projected into a subspace with the chosen number of dimensions by calling the `transform()` function. Once fit, the singular values and principal components can be accessed on the PCA class via the `explained_variance_` and `components_` attributes. The example below demonstrates using this class by first creating an instance, fitting it on a 3×2 matrix, accessing the values and vectors of the projection, and transforming the original data.

```
# principal component analysis with scikit-learn
from numpy import array
from sklearn.decomposition import PCA
# define matrix
A = array([
  [1, 2],
  [3, 4],
  [5, 6]])
print(A)
# create the transform
pca = PCA(2)
# fit transform
pca.fit(A)
# access values and vectors
print(pca.components_)
print(pca.explained_variance_)
# transform data
B = pca.transform(A)
print(B)
```

Listing 18.3: Example of calculating a PCA with scikit-learn.

Running the example first prints the 3×2 data matrix, then the principal components and values, followed by the projection of the original matrix. We can see, that with some very minor floating point rounding that we achieve the same principal components, singular values, and projection as in the previous example.

```
[[1 2]
 [3 4]
 [5 6]]

[[ 0.70710678 0.70710678]
 [ 0.70710678 -0.70710678]]

[  8.00000000e+00  2.25080839e-33]

[[ -2.82842712e+00 2.22044605e-16]
 [  0.00000000e+00  0.00000000e+00]
 [  2.82842712e+00 -2.22044605e-16]]
```

Listing 18.4: Sample output from calculating a PCA with scikit-learn.

18.5 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Re-run the examples with your own small contrived array data.
- Load a dataset and calculate the PCA on it and compare the results from the two methods.
- Search for and locate 10 examples where PCA has been used in machine learning papers.

If you explore any of these extensions, I'd love to know.

18.6 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

18.6.1 Books

- Section 7.3 Principal Component Analysis (PCA by the SVD), *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2CZgTTB
- Section 2.12 Example: Principal Components Analysis, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU

18.7 API

- `numpy.mean()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.mean.html
- `numpy.cov()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.cov.html
- `numpy.linalg.eig()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.eig.html
- `sklearn.decomposition.PCA` API.
  http://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

18.8 Articles

- Principal component analysis on Wikipedia.
  https://en.wikipedia.org/wiki/Principal_component_analysis
- Covariance matrix.
  https://en.wikipedia.org/wiki/Covariance_matrix

18.9 Summary

In this tutorial, you discovered the Principal Component Analysis machine learning method for dimensionality reduction. Specifically, you learned:

- The procedure for calculating the Principal Component Analysis and how to choose principal components.
- How to calculate the Principal Component Analysis from scratch in NumPy.
- How to calculate the Principal Component Analysis for reuse on more data in scikit-learn.

18.9.1 Next

In the next chapter you will discover the linear algebra reformulation of linear regression.

Chapter 19

Linear Regression

Linear regression is a method for modeling the relationship between one or more independent variables and a dependent variable. It is a staple of statistics and is often considered a good introductory machine learning method. It is also a method that can be reformulated using matrix notation and solved using matrix operations. In this tutorial, you will discover the matrix formulation of linear regression and how to solve it using direct and matrix factorization methods. After completing this tutorial, you will know:

- Linear regression and the matrix reformulation with the normal equations.
- How to solve linear regression using a QR matrix decomposition.
- How to solve linear regression using SVD and the pseudoinverse.

Let's get started.

19.1 Tutorial Overview

This tutorial is divided into 7 parts; they are:

1. What is Linear Regression
2. Matrix Formulation of Linear Regression
3. Linear Regression Dataset
4. Solve via Inverse
5. Solve via QR Decomposition
6. Solve via SVD and Pseudoinverse
7. Solve via Convenience Function

19.2 What is Linear Regression

Linear regression is a method for modeling the relationship between two scalar values: the input variable x and the output variable y. The model assumes that y is a linear function or a weighted sum of the input variable.

$$y = f(x) \tag{19.1}$$

Or, stated with the coefficients.

$$y = b_0 + b_1 \times x_1 \tag{19.2}$$

The model can also be used to model an output variable given multiple input variables called multivariate linear regression (below, brackets were added for readability).

$$y = b_0 + (b_1 \times x_1) + (b_2 \times x_2) + \cdots \tag{19.3}$$

The objective of creating a linear regression model is to find the values for the coefficient values (b) that minimize the error in the prediction of the output variable y.

19.3 Matrix Formulation of Linear Regression

Linear regression can be stated using Matrix notation; for example:

$$y = X \cdot b \tag{19.4}$$

Or, without the dot notation.

$$y = Xb \tag{19.5}$$

Where X is the input data and each column is a data feature, b is a vector of coefficients and y is a vector of output variables for each row in X.

$$X = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \\ x_{4,1} & x_{4,2} & x_{4,3} \end{pmatrix} \tag{19.6}$$

$$b = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} \tag{19.7}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} \tag{19.8}$$

Reformulated, the problem becomes a system of linear equations where the b vector values are unknown. This type of system is referred to as overdetermined because there are more equations than there are unknowns, i.e. each coefficient is used on each row of data. It is a

challenging problem to solve analytically because there are multiple inconsistent solutions, e.g. multiple possible values for the coefficients. Further, all solutions will have some error because there is no line that will pass nearly through all points, therefore the approach to solving the equations must be able to handle that. The way this is typically achieved is by finding a solution where the values for b in the model minimize the squared error. This is called linear least squares.

$$||X \cdot b - y||^2 = \sum_{i=1}^{m} \sum_{j=1}^{n} X_{i,j} \cdot (b_j - y_i)^2 \tag{19.9}$$

This formulation has a unique solution as long as the input columns are independent (e.g. uncorrelated).

> We cannot always get the error $e = b - Ax$ down to zero. When e is zero, x is an exact solution to $Ax = b$. When the length of e is as small as possible, $\hat{x}$ is a least squares solution.
>
> — Page 219, *Introduction to Linear Algebra*, Fifth Edition, 2016.

In matrix notation, this problem is formulated using the so-named normal equation:

$$X^T \cdot X \cdot b = X^T \cdot y \tag{19.10}$$

This can be re-arranged in order to specify the solution for b as:

$$b = (X^T \cdot X)^{-1} \cdot X^T \cdot y \tag{19.11}$$

This can be solved directly, although given the presence of the matrix inverse can be numerically challenging or unstable.

19.4 Linear Regression Dataset

In order to explore the matrix formulation of linear regression, let's first define a dataset as a context. We will use a simple 2D dataset where the data is easy to visualize as a scatter plot and models are easy to visualize as a line that attempts to fit the data points. The example below defines a 5×2 matrix dataset, splits it into X and y components, and plots the dataset as a scatter plot.

```
# linear regression dataset
from numpy import array
from matplotlib import pyplot
# define dataset
data = array([
  [0.05, 0.12],
  [0.18, 0.22],
  [0.31, 0.35],
  [0.42, 0.38],
  [0.5, 0.49]])
print(data)
# split into inputs and outputs
```

```
X, y = data[:,0], data[:,1]
X = X.reshape((len(X), 1))
# scatter plot
pyplot.scatter(X, y)
pyplot.show()
```

Listing 19.1: Example of example linear regression dataset.

Running the example first prints the defined dataset.

```
[[ 0.05 0.12]
 [ 0.18 0.22]
 [ 0.31 0.35]
 [ 0.42 0.38]
 [ 0.5  0.49]]
```

Listing 19.2: Sample output from example linear regression dataset.

A scatter plot of the dataset is then created showing that a straight line cannot fit this data exactly.

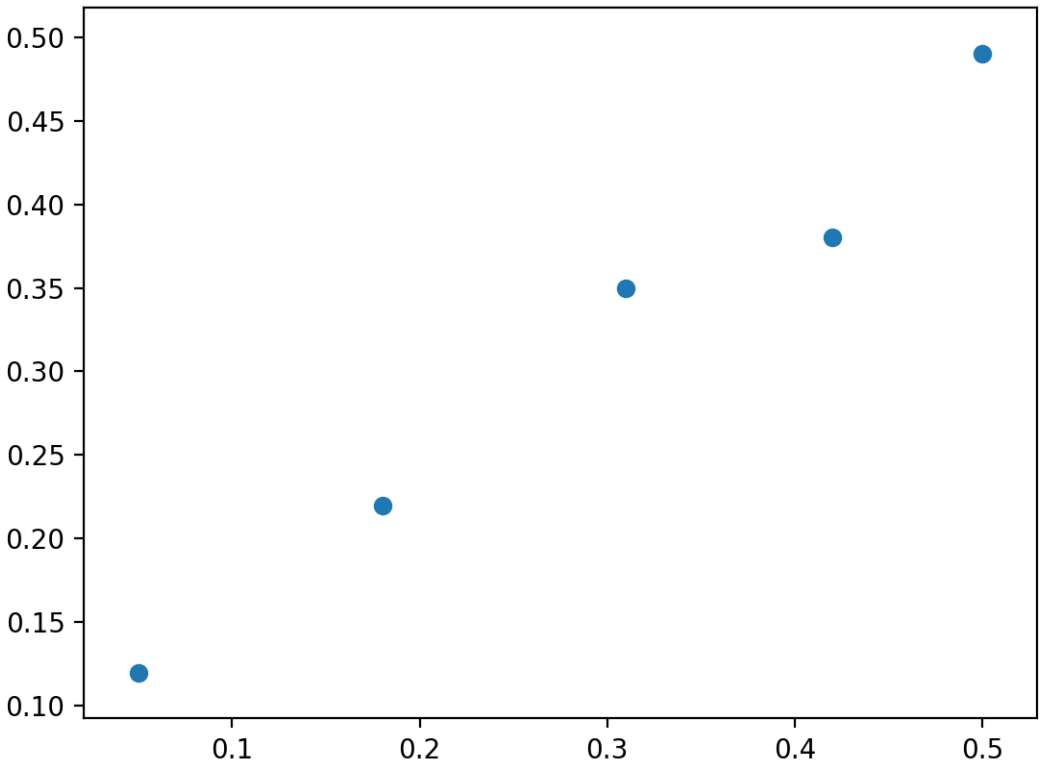


Figure 19.1: Scatter Plot of Linear Regression Dataset.

19.5 Solve via Inverse

The first approach is to attempt to solve the regression problem directly using the matrix inverse. That is, given X, what are the set of coefficients b that when multiplied by X will give y. As we saw in a previous section, the normal equations define how to calculate b directly.

$$b = (X^T \cdot X)^{-1} \cdot X^T \cdot y \tag{19.12}$$

This can be calculated directly in NumPy using the `inv()` function for calculating the matrix inverse.

```
b = inv(X.T.dot(X)).dot(X.T).dot(y)
```

Listing 19.3: Example code for solving linear least squares directly.

Once the coefficients are calculated, we can use them to predict outcomes given X.

```
yhat = X.dot(b)
```

Listing 19.4: Example code for using the coefficients to make a prediction.

Putting this together with the dataset defined in the previous section, the complete example is listed below.

```
# direct solution to linear least squares
from numpy import array
from numpy.linalg import inv
from matplotlib import pyplot
# define dataset
data = array([
  [0.05, 0.12],
  [0.18, 0.22],
  [0.31, 0.35],
  [0.42, 0.38],
  [0.5, 0.49]])
# split into inputs and outputs
X, y = data[:,0], data[:,1]
X = X.reshape((len(X), 1))
# linear least squares
b = inv(X.T.dot(X)).dot(X.T).dot(y)
print(b)
# predict using coefficients
yhat = X.dot(b)
# plot data and predictions
pyplot.scatter(X, y)
pyplot.plot(X, yhat, color='red')
pyplot.show()
```

Listing 19.5: Example of calculating a linear regression solution directly.

Running the example performs the calculation and prints the coefficient vector b.

```
[ 1.00233226]
```

Listing 19.6: Sample output from calculating a linear regression solution directly.

A scatter plot of the dataset is then created with a line plot for the model, showing a reasonable fit to the data.

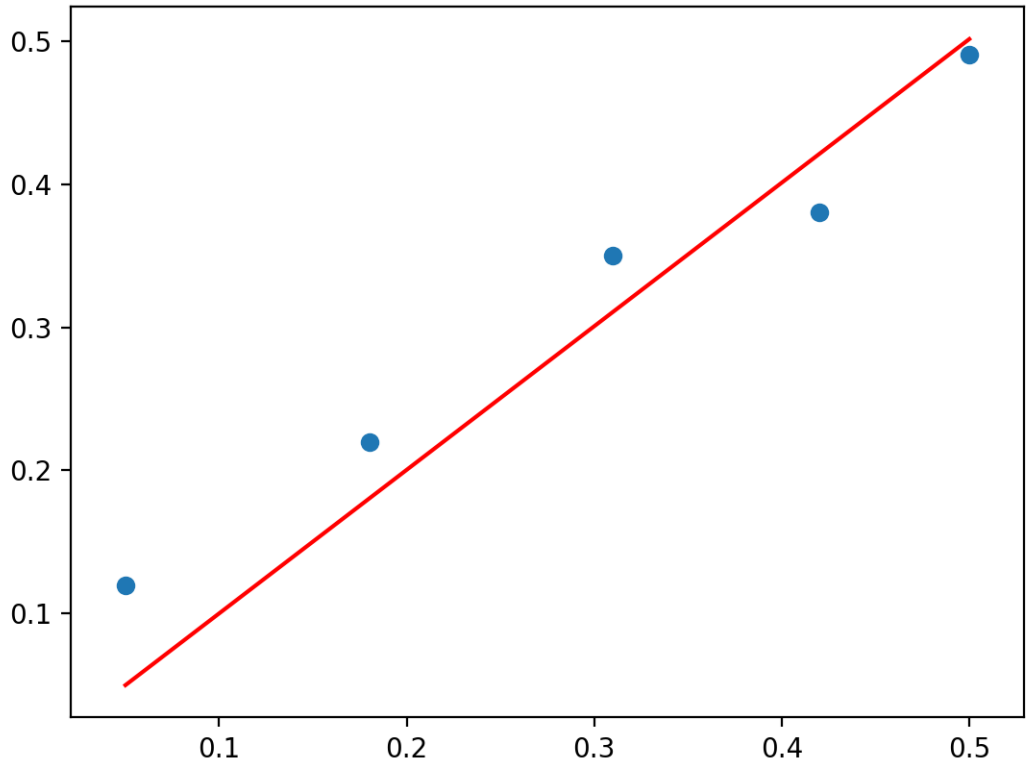


Figure 19.2: Scatter Plot of Direct Solution to the Linear Regression Problem.

A problem with this approach is the matrix inverse that is both computationally expensive and numerically unstable. An alternative approach is to use a matrix decomposition to avoid this operation. We will look at two examples in the following sections.

19.6 Solve via QR Decomposition

The QR decomposition is an approach of breaking a matrix down into its constituent elements.

$$A = Q \cdot R \tag{19.13}$$

Where A is the matrix that we wish to decompose, Q a matrix with the size $m \times m$, and R is an upper triangle matrix with the size $m \times n$. The QR decomposition is a popular approach for solving the linear least squares equation. Stepping over all of the derivation, the coefficients can be found using the Q and R elements as follows:

$$b = R^{-1} \cdot Q^T \cdot y \tag{19.14}$$

The approach still involves a matrix inversion, but in this case only on the simpler R matrix. The QR decomposition can be found using the `qr()` function in NumPy. The calculation of the coefficients in NumPy looks as follows:

```
# QR decomposition
Q, R = qr(X)
b = inv(R).dot(Q.T).dot(y)
```

Listing 19.7: Example of calculating a QR decomposition.

Tying this together with the dataset, the complete example is listed below.

```
# QR decomposition solution to linear least squares
from numpy import array
from numpy.linalg import inv
from numpy.linalg import qr
from matplotlib import pyplot
# define dataset
data = array([
  [0.05, 0.12],
  [0.18, 0.22],
  [0.31, 0.35],
  [0.42, 0.38],
  [0.5, 0.49]])
# split into inputs and outputs
X, y = data[:,0], data[:,1]
X = X.reshape((len(X), 1))
# factorize
Q, R = qr(X)
b = inv(R).dot(Q.T).dot(y)
print(b)
# predict using coefficients
yhat = X.dot(b)
# plot data and predictions
pyplot.scatter(X, y)
pyplot.plot(X, yhat, color='red')
pyplot.show()
```

Listing 19.8: Example of calculating a linear regression solution using a QR decomposition.

Running the example first prints the coefficient solution and plots the data with the model.

```
[ 1.00233226]
```

Listing 19.9: Sample output from calculating a linear regression using a QR decomposition.

The QR decomposition approach is more computationally efficient and more numerically stable than calculating the normal equation directly, but does not work for all data matrices.

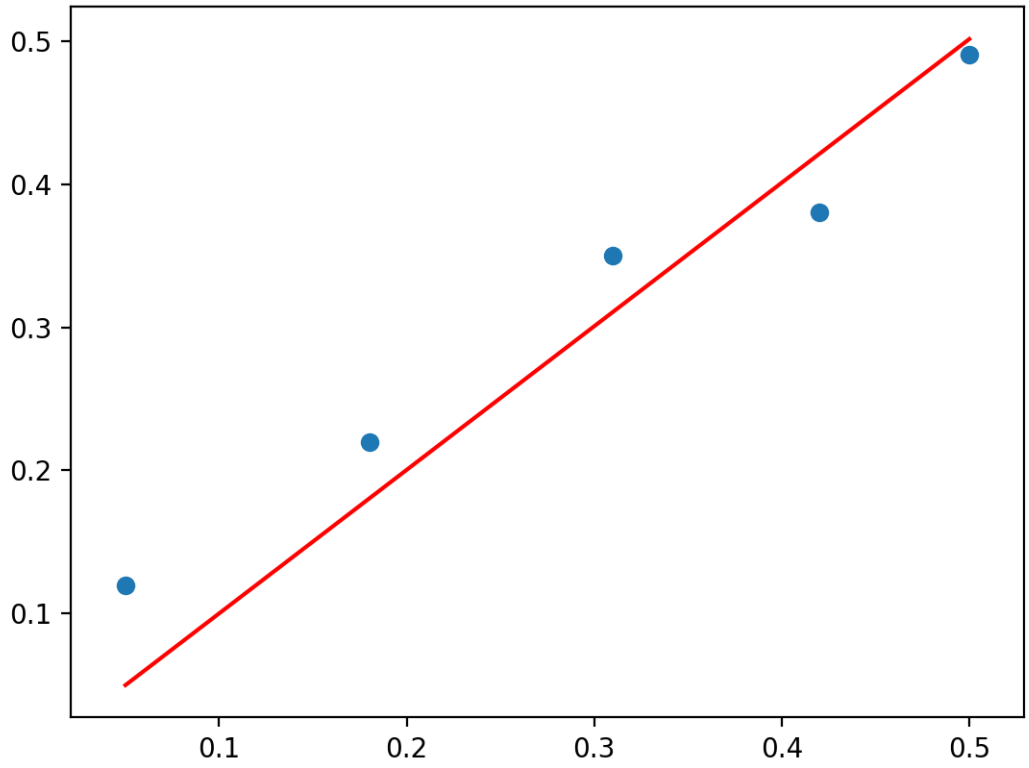


Figure 19.3: Scatter Plot of QR Decomposition Solution to the Linear Regression Problem.

19.7 Solve via SVD and Pseudoinverse

The Singular-Value Decomposition, or SVD for short, is a matrix decomposition method like the QR decomposition.

$$X = U \cdot \Sigma \cdot V^T \tag{19.15}$$

Where A is the real $n \times m$ matrix that we wish to decompose, U is a $m \times m$ matrix, Σ (often represented by the uppercase Greek letter Sigma) is an $m \times n$ diagonal matrix, and V^T is the transpose of an $n \times n$ matrix. Unlike the QR decomposition, all matrices have a singular-value decomposition. As a basis for solving the system of linear equations for linear regression, SVD is more stable and the preferred approach. Once decomposed, the coefficients can be found by calculating the pseudoinverse of the input matrix X and multiplying that by the output vector y.

$$b = X^+ \cdot y \tag{19.16}$$

Where the pseudoinverse X^+ is calculated as following:

$$X^+ = U \cdot D^+ \cdot V^T \tag{19.17}$$

Where X^+ is the pseudoinverse of X and the + is a superscript, D^+ is the pseudoinverse of the diagonal matrix Σ and V^T is the transpose of V. NumPy provides the function `pinv()` to calculate the pseudoinverse directly. The complete example is listed below.

```
# SVD solution via pseudoinverse to linear least squares
from numpy import array
from numpy.linalg import pinv
from matplotlib import pyplot
# define dataset
data = array([
  [0.05, 0.12],
  [0.18, 0.22],
  [0.31, 0.35],
  [0.42, 0.38],
  [0.5, 0.49]])
# split into inputs and outputs
X, y = data[:,0], data[:,1]
X = X.reshape((len(X), 1))
# calculate coefficients
b = pinv(X).dot(y)
print(b)
# predict using coefficients
yhat = X.dot(b)
# plot data and predictions
pyplot.scatter(X, y)
pyplot.plot(X, yhat, color='red')
pyplot.show()
```

Listing 19.10: Example of calculating a linear regression solution using an SVD.

Running the example prints the coefficient and plots the data with a red line showing the predictions from the model.

```
[ 1.00233226]
```

Listing 19.11: Sample output from calculating a linear regression using an SVD.

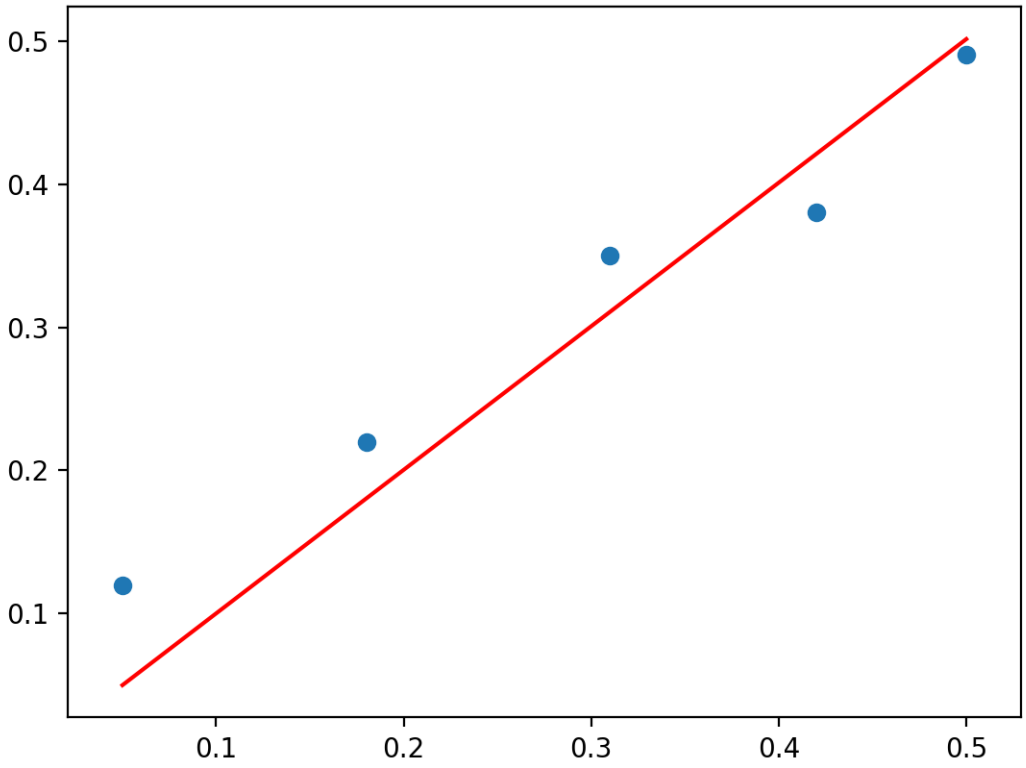


Figure 19.4: Scatter Plot of SVD Solution to the Linear Regression Problem.

19.8 Solve via Convenience Function

The pseudoinverse via SVD approach to solving linear least squares is the de facto standard. This is because it is stable and works with most datasets. NumPy provides a convenience function named `lstsq()` that solves the linear least squares function using the SVD approach. The function takes as input the X matrix and y vector and returns the b coefficients as well as residual errors, the rank of the provided X matrix and the singular values. The example below demonstrate the `lstsq()` function on the test dataset.

```
# least squares via convenience function
from numpy import array
from numpy.linalg import lstsq
from matplotlib import pyplot
# define dataset
data = array([
  [0.05, 0.12],
  [0.18, 0.22],
  [0.31, 0.35],
  [0.42, 0.38],
  [0.5, 0.49]])
# split into inputs and outputs
X, y = data[:,0], data[:,1]
```

```
X = X.reshape((len(X), 1))
# calculate coefficients
b, residuals, rank, s = lstsq(X, y)
print(b)
# predict using coefficients
yhat = X.dot(b)
# plot data and predictions
pyplot.scatter(X, y)
pyplot.plot(X, yhat, color='red')
pyplot.show()
```

Listing 19.12: Example of calculating a linear regression solution using `lstsq()`.

Running the example prints the coefficient and plots the data with a red line showing the predictions from the model.

```
[ 1.00233226]
```

Listing 19.13: Sample output from calculating a linear regression solution using `lstsq()`.

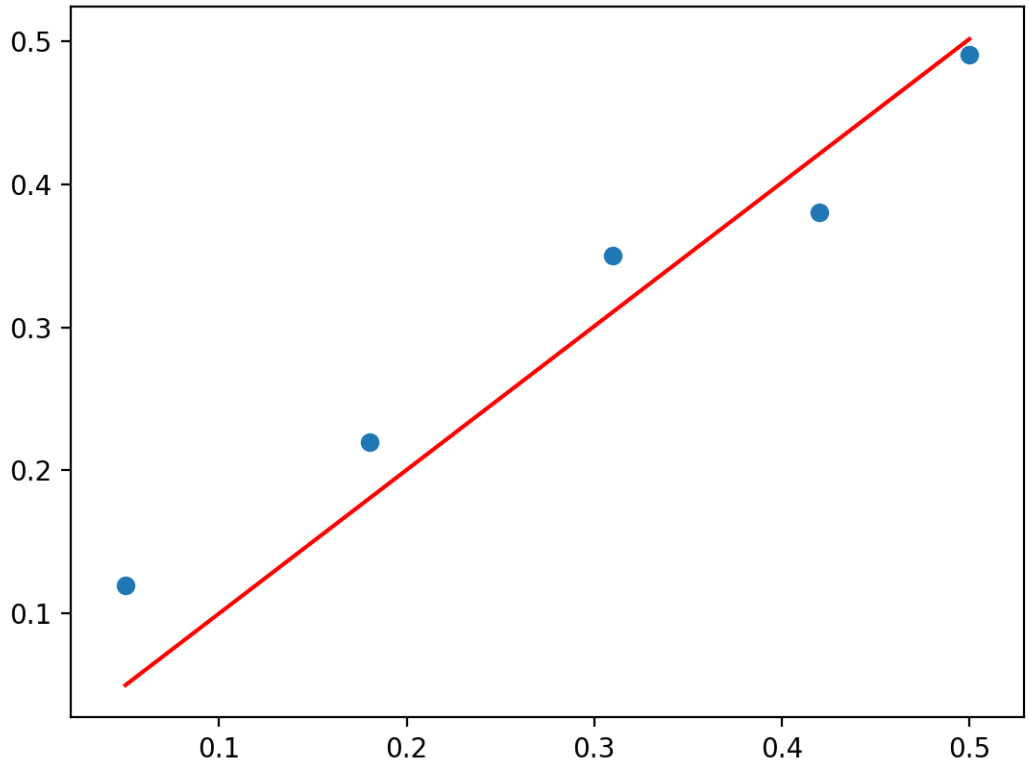


Figure 19.5: Scatter Plot of the `lstsq()` Solution to the Linear Regression Problem.

19.9 Extensions

This section lists some ideas for extending the tutorial that you may wish to explore.

- Test each linear regression on your own small contrived dataset.
- Load a tabular dataset and test each linear regression method and compare the results.
- Research and implement alternate ways of solving linear least squares using linear algebra.

If you explore any of these extensions, I'd love to know.

19.10 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

19.10.1 Books

- Section 7.7 Least squares approximate solutions. *No Bullshit Guide To Linear Algebra*, 2017.
  http://amzn.to/2k76D4
- Section 4.3 Least Squares Approximations, *Introduction to Linear Algebra*, Fifth Edition, 2016.
  http://amzn.to/2AZ7R8j
- Lecture 11, Least Squares Problems, *Numerical Linear Algebra*, 1997.
  http://amzn.to/2kjEF4S
- Chapter 5, Orthogonalization and Least Squares, *Matrix Computations*, 2012.
  http://amzn.to/2B9xnLD
- Chapter 12, Singular-Value and Jordan Decompositions, *Linear Algebra and Matrix Analysis for Statistics*, 2014.
  http://amzn.to/2A9ceNv
- Section 2.9 The Moore-Penrose Pseudoinverse, *Deep Learning*, 2016.
  http://amzn.to/2B3MsuU
- Section 15.4 General Linear Least Squares, *Numerical Recipes: The Art of Scientific Computing*, Third Edition, 2007.
  http://amzn.to/2BezVEE

19.10.2 API

- `numpy.linalg.inv()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.inv.html
- `numpy.linalg.qr()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.qr.html

- `numpy.linalg.svd()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.svd.html
- `numpy.diag()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.diag.html
- `numpy.linalg.pinv()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.pinv.html
- `numpy.linalg.lstsq()` API.
  https://docs.scipy.org/doc/numpy-1.13.0/reference/generated/numpy.linalg.lstsq.html

19.10.3 Articles

- Linear regression on Wikipedia.
  https://en.wikipedia.org/wiki/Linear_regression
- Least squares on Wikipedia.
  https://en.wikipedia.org/wiki/Least_squares
- Linear least squares (mathematics) on Wikipedia.
  https://en.wikipedia.org/wiki/Linear_least_squares_(mathematics)
- Overdetermined system on Wikipedia.
  https://en.wikipedia.org/wiki/Overdetermined_system
- QR decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/QR_decomposition
- Singular-value decomposition on Wikipedia.
  https://en.wikipedia.org/wiki/Singular-value_decomposition
- Moore-Penrose inverse.
  https://en.wikipedia.org/wiki/Moore%E2%80%93Penrose_inverse

19.11 Summary

In this tutorial, you discovered the matrix formulation of linear regression and how to solve it using direct and matrix factorization methods. Specifically, you learned:

- Linear regression and the matrix reformulation with the normal equations.
- How to solve linear regression using a QR matrix decomposition.
- How to solve linear regression using SVD and the pseudoinverse.

19.11.1 Next

This was the end of the part on statistics and the final chapter. Next you can get additional help in the appendix.

Part VII

Appendix

Appendix A

Getting Help

This is just the beginning of your journey with linear algebra. As you start to work on projects and expand your existing knowledge of the techniques, you may need help. This appendix points out some of the best sources of help.

A.1 Linear Algebra on Wikipedia

Wikipedia is a great place to start. All of the important topics are covered, the descriptions are concise, and the equations are consistent and readable. What is missing is the more human level descriptions such as analogies and intuitions. Nevertheless, when you have questions about linear algebra, I recommend stopping by Wikipedia first. Some good high-level pages to start on include:

- Linear Algebra.
  https://en.wikipedia.org/wiki/Linear_algebra
- Matrix (mathematics).
  https://en.wikipedia.org/wiki/Matrix_(mathematics)
- Matrix decomposition.
  https://en.wikipedia.org/wiki/Matrix_decomposition
- List of linear algebra topics.
  https://en.wikipedia.org/wiki/List_of_linear_algebra_topics

A.2 Linear Algebra Textbooks

I strongly recommend getting a good textbook on the topic of linear algebra and using it as a reference. The benefit of a good textbook is that the explanations of the various operations you require will be consistent (or should be). The downside of textbooks is that they can be very expensive. A good textbook is often easy to spot because it will be the basis for a range of undergraduate or postgraduate courses at top universities. Some introductory textbooks on linear algebra I recommend include:

- *Introduction to Linear Algebra*, Fifth Edition, Gilbert Strang, 2016.
  http://amzn.to/2j2J0g4
- *Linear Algebra Done Right*, Third Edition, 2015.
  http://amzn.to/2BGuEqI
- *No Bullshit Guide To Linear Algebra*, Ivan Savov, 2017.
  http://amzn.to/2k76D4C

Some more advanced textbooks I recommend include:

- *Matrix Computations*, Gene Golub and Charles Van Loan, 2012.
  http://amzn.to/2B9xnLD
- *Numerical Linear Algebra*, Lloyd Trefethen and David Bau 1997.
  http://amzn.to/2kjEF4S

I'd also recommend a good textbook on multivariate statistics, which is the intersection of linear algebra, and numerical statistical methods. Some good introductory textbooks include:

- *Applied Multivariate Statistical Analysis*, Richard Johnson and Dean Wichern, 2012.
  http://amzn.to/2AUcEc5
- *Applied Multivariate Statistical Analysis*, Wolfgang Karl Hardle and Leopold Simar, 2015.
  http://amzn.to/2AWIViz

There are also many good free online books written by academics. See the end of the *Linear Algebra* page on Wikipedia for an extensive (and impressive) reading list.

A.3 Linear Algebra University Courses

University courses on linear algebra are useful in that they layout the topics that an undergraduate student is expected to know. As a machine learning practitioner, it is more than you need, but does provide context for the elements that you do need to know. Many university courses now provide PDF versions of lecture slides, notes, and readings. Some even provide pre-recorded video lectures, which can be invaluable. I would encourage you to use university course material surgically by dipping into courses to get deeper knowledge on specific topics. I think working through a given course end-to-end is too time consuming and covers too much for the average machine learning practitioner. Some recommended courses from top US schools include:

- Linear Algebra at MIT by Gilbert Strang.
  https://ocw.mit.edu/courses/mathematics/18-06-linear-algebra-spring-2010/index.htm
- The Matrix in Computer Science at Brown by Philip Klein.
  http://cs.brown.edu/courses/cs053/current/index.htm
- Computational Linear Algebra for Coders at University of San Francisco by Rachel Thomas.
  https://github.com/fastai/numerical-linear-algebra/

A.4 Linear Algebra Online Courses

Online courses are different from university courses. They are designed for distance education and often are less complete or less rigorous than a full undergraduate course. This is a good feature for machine learning practitioners looking to get up to speed fast on the topic. If the course is short, it may be worth taking it through end-to-end. Generally, and like university courses, I would recommend being surgical with the topics and dip in as needed. Some online courses I recommend include:

- Linear Algebra on Khan Academy.
  https://www.khanacademy.org/math/linear-algebra
- Linear Algebra: Foundations to Frontiers on edX.
  https://www.edx.org/course/laff-linear-algebra-foundations-to-frontiers

A.5 NumPy Resources

You may need help with NumPy when implementing your linear algebra in Python. The NumPy API documentation is excellent, below are a few resources that you can use to learn more about how NumPy works or how to use specific NumPy functions.

- NumPy Reference.
  https://docs.scipy.org/doc/numpy/reference/
- NumPy Array Creation Routines.
  https://docs.scipy.org/doc/numpy/reference/routines.array-creation.html
- NumPy Array Manipulation Routines.
  https://docs.scipy.org/doc/numpy/reference/routines.array-manipulation.html
- NumPy Linear Algebra.
  https://docs.scipy.org/doc/numpy/reference/routines.linalg.html
- SciPy Linear Algebra.
  https://docs.scipy.org/doc/scipy/reference/linalg.html

If you are looking for a broader understanding on NumPy and SciPy usage, the below books provide a good starting reference:

- *Python for Data Analysis*, 2017.
  http://amzn.to/2B1sfXi
- *Elegant SciPy*, 2017.
  http://amzn.to/2yujXnT
- *Guide to NumPy*, 2015.
  http://amzn.to/2j3kEzd

A.6 Ask Questions About Linear Algebra

There are a lot of places that you can ask questions about linear algebra online given the current abundance of question-and-answer platforms. Below is a list of the top places I recommend posting a question. Remember to search for your question before posting in case it has been asked and answered before.

- Linear Algebra tag on the Mathematics Stack Exchange.
  https://math.stackexchange.com/?tags=linear-algebra
- Linear Algebra tag on Cross Validated.
  https://stats.stackexchange.com/questions/tagged/linear-algebra
- Linear Algebra tag on Stack Overflow.
  https://stackoverflow.com/questions/tagged/linear-algebra
- Linear Algebra on Quora.
  https://www.quora.com/topic/Linear-Algebra
- Math Subreddit.
  https://www.reddit.com/r/math/

A.7 How to Ask Questions

Knowing where to get help is the first step, but you need to know how to get the most out of these resources. Below are some tips that you can use:

- Boil your question down to the simplest form. E.g. not something broad like *my model does not work* or *how does x work*.
- Search for answers before asking questions.
- Provide complete code and error messages.
- Boil your code down to the smallest possible working example that demonstrates the issue.

A.8 Contact the Author

You are not alone. If you ever have any questions about deep learning, natural language processing, or this book, please contact me directly. I will do my best to help.

Jason Brownlee
Jason@MachineLearningMastery.com

Appendix B

How to Setup a Workstation for Python

It can be difficult to install a Python machine learning environment on some platforms. Python itself must be installed first and then there are many packages to install, and it can be confusing for beginners. In this tutorial, you will discover how to setup a Python machine learning development environment using Anaconda.

After completing this tutorial, you will have a working Python environment to begin learning, practicing, and developing machine learning software. These instructions are suitable for Windows, Mac OS X, and Linux platforms. I will demonstrate them on OS X, so you may see some mac dialogs and file extensions.

B.1 Overview

In this tutorial, we will cover the following steps:

1. Download Anaconda
2. Install Anaconda
3. Start and Update Anaconda

Note: The specific versions may differ as the software and libraries are updated frequently.

B.2 Download Anaconda

In this step, we will download the Anaconda Python package for your platform. Anaconda is a free and easy-to-use environment for scientific Python.

- 1. Visit the Anaconda homepage.
  https://www.continuum.io/
- 2. Click `Anaconda` from the menu and click `Download` to go to the download page.
  https://www.continuum.io/downloads

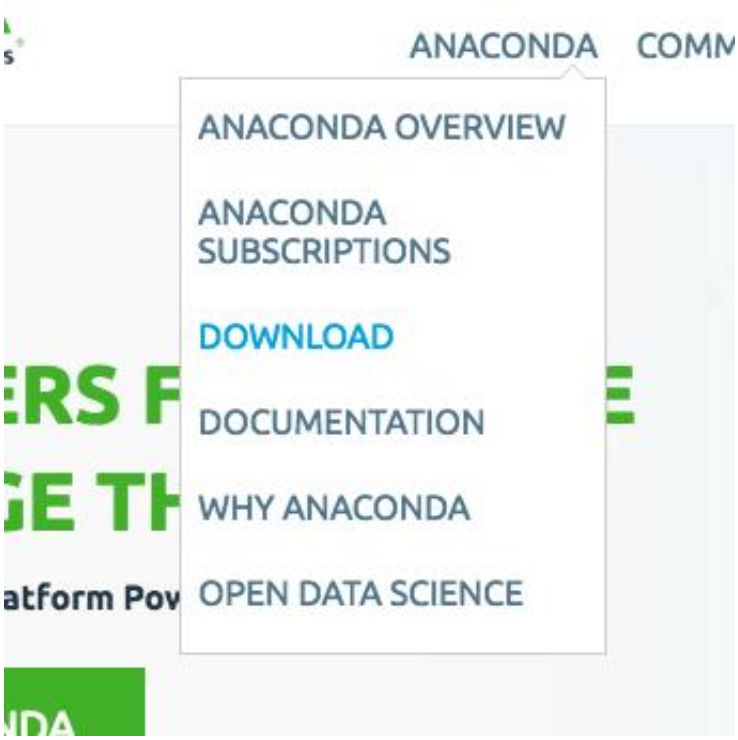


Figure B.1: Click Anaconda and Download.

- 3. Choose the download suitable for your platform (Windows, OSX, or Linux):
  - Choose Python 3.6
  - Choose the Graphical Installer

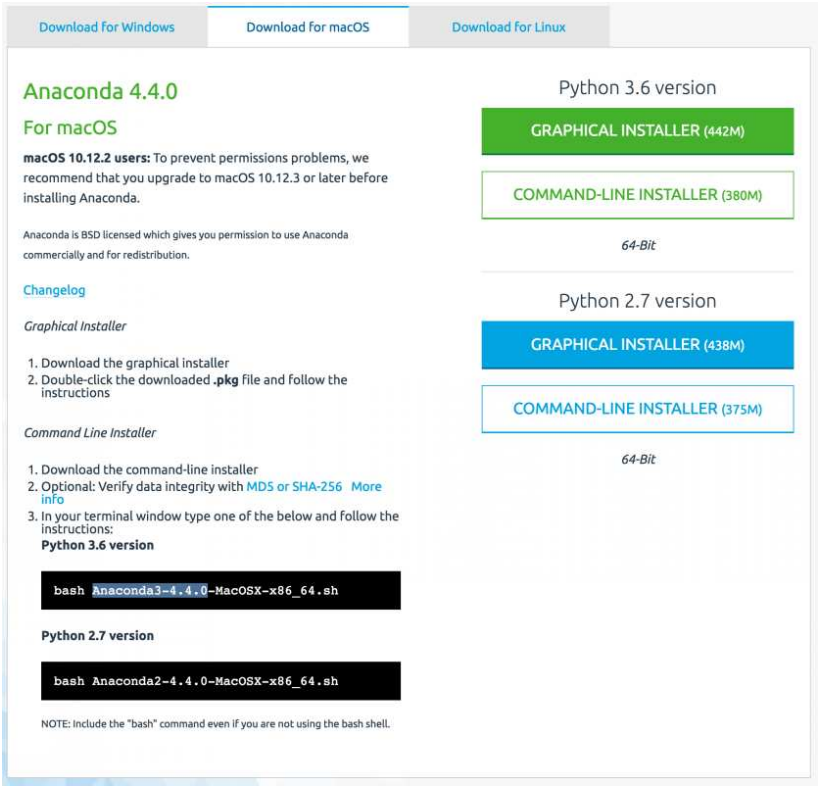


Figure B.2: Choose Anaconda Download for Your Platform.

This will download the Anaconda Python package to your workstation. I'm on OS X, so I chose the OS X version. The file is about 426 MB. You should have a file with a name like:

```
Anaconda3-4.4.0-MacOSX-x86_64.pkg
```

Listing B.1: Example filename on Mac OS X.

B.3 Install Anaconda

In this step, we will install the Anaconda Python software on your system. This step assumes you have sufficient administrative privileges to install software on your system.

- 1. Double click the downloaded file.
- 2. Follow the installation wizard.

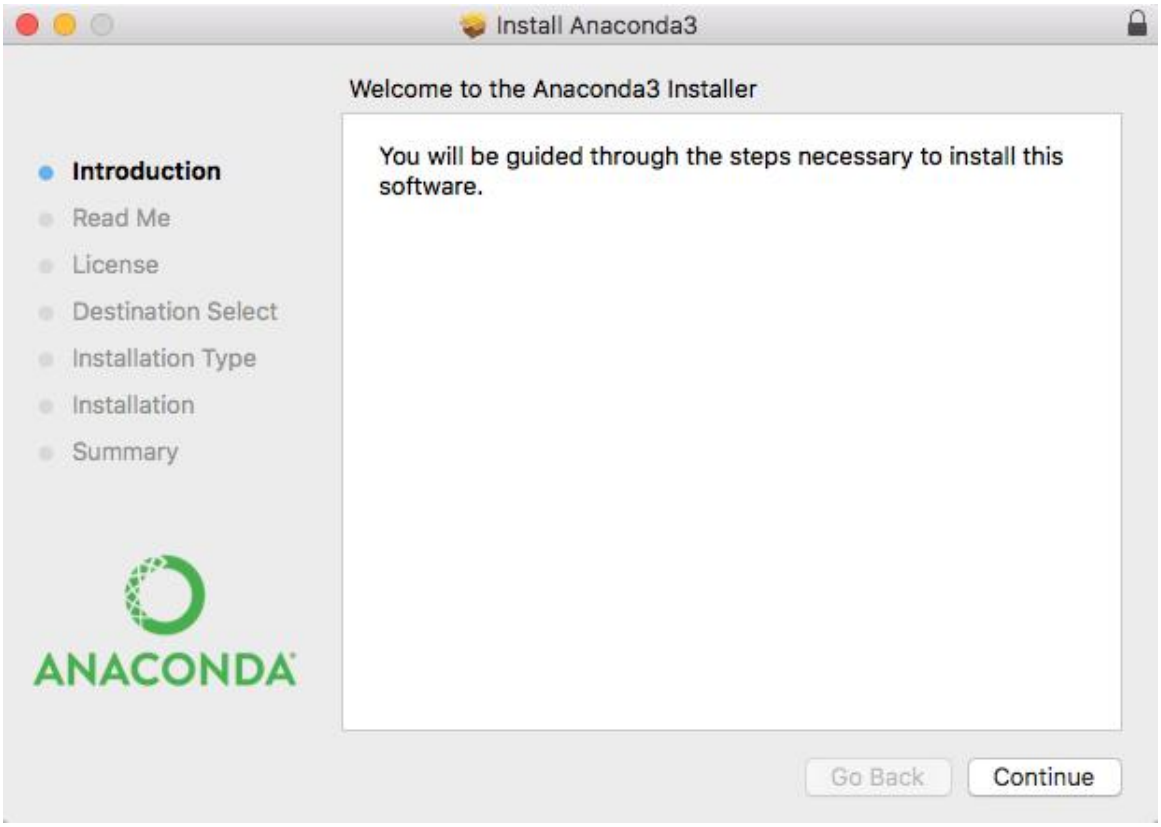


Figure B.3: Anaconda Python Installation Wizard.

Installation is quick and painless. There should be no tricky questions or sticking points.

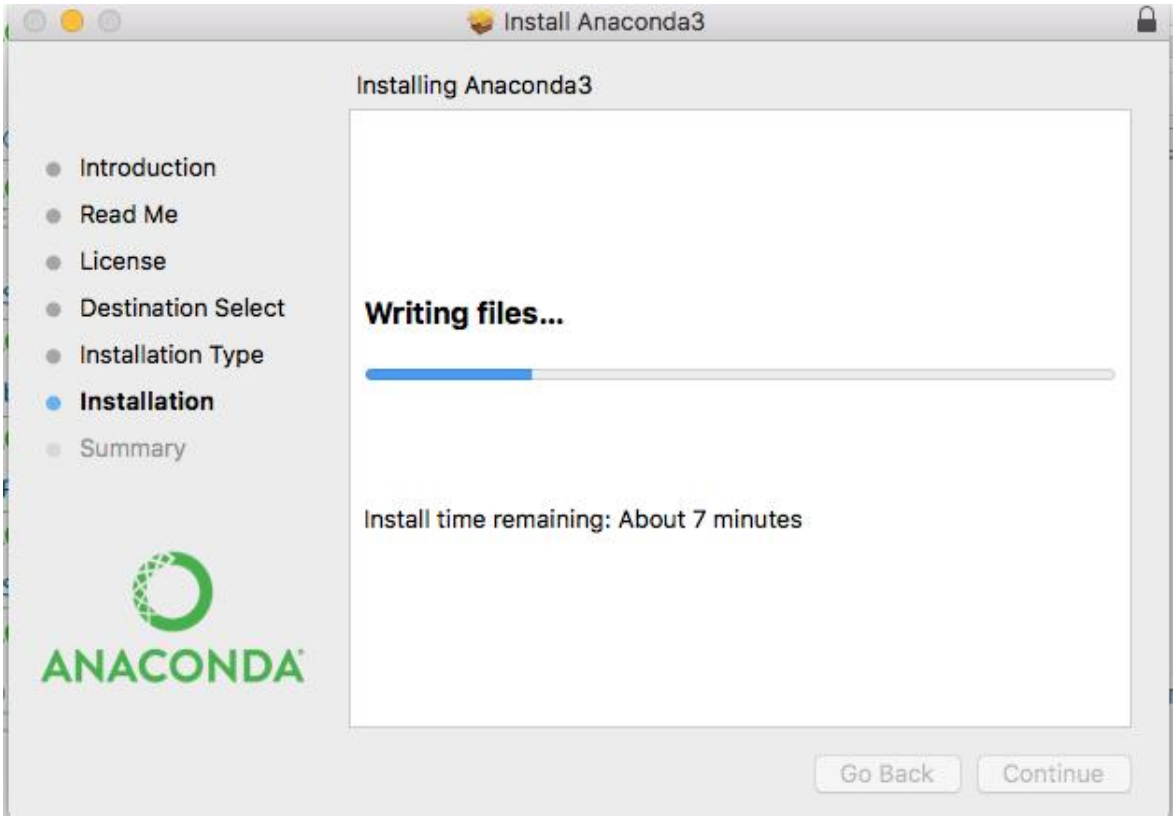


Figure B.4: Anaconda Python Installation Wizard Writing Files.

The installation should take less than 10 minutes and take up a little more than 1 GB of space on your hard drive.

B.4 Start and Update Anaconda

In this step, we will confirm that your Anaconda Python environment is up to date. Anaconda comes with a suite of graphical tools called Anaconda Navigator. You can start Anaconda Navigator by opening it from your application launcher.

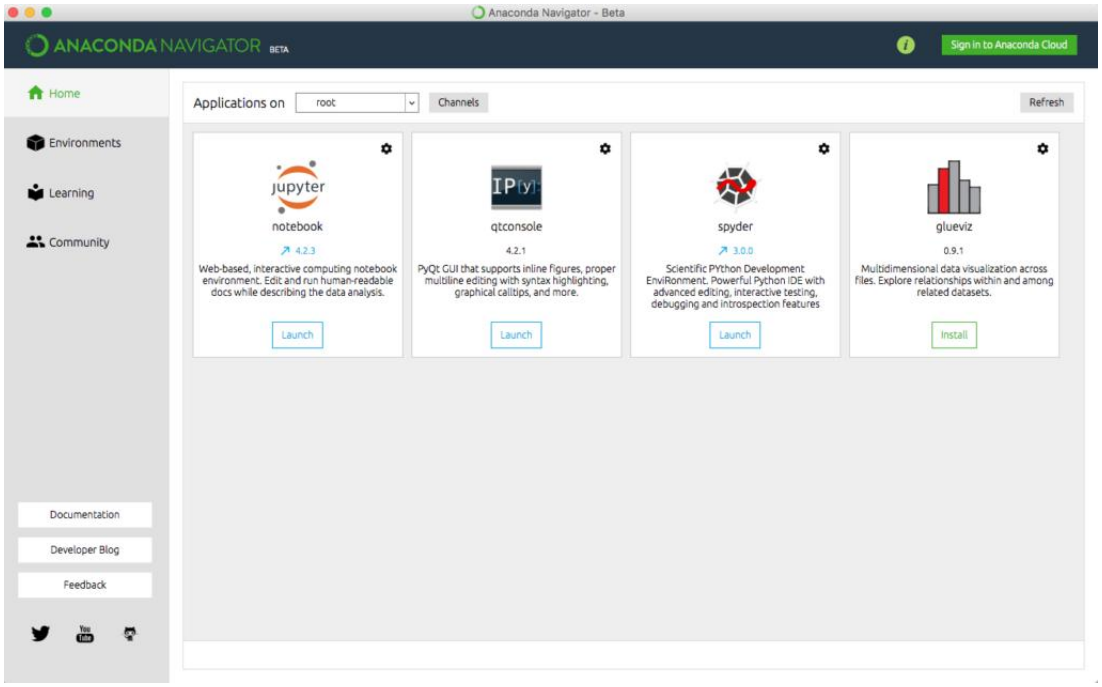


Figure B.5: Anaconda Navigator GUI.

You can use the Anaconda Navigator and graphical development environments later; for now, I recommend starting with the Anaconda command line environment called conda. Conda is fast, simple, it's hard for error messages to hide, and you can quickly confirm your environment is installed and working correctly.

- 1. Open a terminal (command line window).
- 2. Confirm conda is installed correctly, by typing:

```
conda -V
```

Listing B.2: Check the conda version.

You should see the following (or something similar):

```
conda 4.3.21
```

Listing B.3: Example conda version.

- 3. Confirm Python is installed correctly by typing:

```
python -V
```

Listing B.4: Check the Python version.

You should see the following (or something similar):

```
Python 3.6.1 :: Anaconda 4.4.0 (x86_64)
```

Listing B.5: Example Python version.

If the commands do not work or have an error, please check the documentation for help for your platform. See some of the resources in the *Further Reading* section.

- 4. Confirm your conda environment is up-to-date, type:

```
conda update conda
conda update anaconda
```

Listing B.6: Update conda and anaconda.

You may need to install some packages and confirm the updates.

- 5. Confirm your SciPy environment.

The script below will print the version number of the key SciPy libraries you require for machine learning development, specifically: SciPy, NumPy, Matplotlib, Pandas, Statsmodels, and Scikit-learn. You can type `python` and type the commands in directly. Alternatively, I recommend opening a text editor and copy-pasting the script into your editor.

```
# scipy
import scipy
print('scipy: %s' % scipy.__version__)
# numpy
import numpy
print('numpy: %s' % numpy.__version__)
# matplotlib
import matplotlib
print('matplotlib: %s' % matplotlib.__version__)
# pandas
import pandas
print('pandas: %s' % pandas.__version__)
# statsmodels
import statsmodels
print('statsmodels: %s' % statsmodels.__version__)
# scikit-learn
import sklearn
print('sklearn: %s' % sklearn.__version__)
```

Listing B.7: Code to check that key Python libraries are installed.

Save the script as a file with the name: `versions.py`. On the command line, change your directory to where you saved the script and type:

```
python versions.py
```

Listing B.8: Run the script from the command line.

You should see output like the following:

```
scipy: 1.0.0
numpy: 1.14.0
matplotlib: 2.1.1
pandas: 0.22.0
statsmodels: 0.8.0
sklearn: 0.19.1
```

Listing B.9: Sample output of versions script.

B.5 Further Reading

This section provides resources if you want to know more about Anaconda.

- Anaconda homepage.
  https://www.continuum.io/
- Anaconda Navigator.
  https://docs.continuum.io/anaconda/navigator.html
- The conda command line tool.
  http://conda.pydata.org/docs/index.html

B.6 Summary

Congratulations, you now have a working Python development environment for machine learning. You can now learn and practice machine learning on your workstation.

Appendix C

Linear Algebra Cheat Sheet

This appendix provides a quick-reference for NumPy examples for common linear algebra operations.

C.1 Array Creation

There are many ways to create NumPy arrays.

C.1.1 Array

```
from numpy import array
A = array([[1,2,3],[1,2,3],[1,2,3]])
```

C.1.2 Empty

```
from numpy import empty
A = empty([3,3])
```

C.1.3 Zeros

```
from numpy import zeros
A = zeros([3,5])
```

C.1.4 Ones

```
from numpy import ones
A = ones([5, 5])
```

C.2 Vectors

A vector is a list or column of scalars.

C.2.1 Vector Addition

```
c = a + b
```

C.2.2 Vector Subtraction

```
c = a - b
```

C.2.3 Vector Multiplication

```
c = a * b
```

C.2.4 Vector Division

```
c = a / b
```

C.2.5 Vector Dot Product

```
c = a.dot(b)
```

C.2.6 Vector-Scalar Multiplication

```
c = a * 2.2
```

C.2.7 Vector Norm

```
from numpy.linalg import norm
l2 = norm(v)
```

C.3 Matrices

A matrix is a two-dimensional array of scalars.

C.3.1 Matrix Addition

```
C = A + B
```

C.3.2 Matrix Subtraction

```
C = A - B
```

C.3.3 Matrix Multiplication (Hadamard Product)

```
C = A * B
```

C.3.4 Matrix Division

```
C = A / B
```

C.3.5 Matrix-Matrix Multiplication (Dot Product)

```
C = A.dot(B)
```

C.3.6 Matrix-Vector Multiplication (Dot Product)

```
C = A.dot(b)
```

C.3.7 Matrix-Scalar Multiplication

```
C = A.dot(2.2)
```

C.4 Types of Matrices

Different types of matrices are often used as elements in broader calculations.

C.4.1 Triangle Matrix

```
# lower
from numpy import tril
lower = tril(M)
# upper
from numpy import triu
upper = triu(M)
```

C.4.2 Diagonal Matrix

```
from numpy import diag
d = diag(M)
```

C.4.3 Identity Matrix

```
from numpy import identity
I = identity(3)
```

C.5 Matrix Operations

Matrix operations are often used as elements in broader calculations.

C.5.1 Matrix Transpose

```
B = A.T
```

C.5.2 Matrix Inversion

```
from numpy.linalg import inv
B = inv(A)
```

C.5.3 Matrix Trace

```
from numpy import trace
B = trace(A)
```

C.5.4 Matrix Determinant

```
from numpy.linalg import det
B = det(A)
```

C.5.5 Matrix Rank

```
from numpy.linalg import matrix_rank
r = matrix_rank(A)
```

C.6 Factorization

Matrix factorization, or matrix decomposition, breaks a matrix down into its constituent parts to make other operations simpler and more numerically stable.

C.6.1 LU Decomposition

```
from scipy.linalg import lu
P, L, U = lu(A)
```

C.6.2 QR Decomposition

```
from numpy.linalg import qr
Q, R = qr(A, 'complete')
```

C.6.3 Cholesky Decomposition

```
from numpy.linalg import cholesky
L = cholesky(A)
```

C.6.4 Eigendecomposition

```
from numpy.linalg import eig
values, vectors = eig(A)
```

C.6.5 Singular-Value Decomposition

```
from scipy.linalg import svd
U, s, V = svd(A)
```

C.7 Statistics

Statistics summarize the contents of vectors or matrices and are often used as components in broader operations.

C.7.1 Mean

```
from numpy import mean
result = mean(v)
```

C.7.2 Variance

```
from numpy import var
result = var(v, ddof=1)
```

C.7.3 Standard Deviation

```
from numpy import std
result = std(v, ddof=1)
```

C.7.4 Covariance Matrix

```
from numpy import cov
sigma = cov(A)
```

C.7.5 Linear Least Squares

```
from numpy.linalg import lstsq
b = lstsq(X, y)
```

Appendix D

Basic Math Notation

You cannot avoid mathematical notation when reading the descriptions of machine learning methods. Often, all it takes is one term or one fragment of notation in an equation to completely derail your understanding of the entire procedure. This can be extremely frustrating, especially for machine learning beginners coming from the world of development. You can make great progress if you know a few basic areas of mathematical notation and some tricks for working through the description of machine learning methods in papers and books. In this tutorial, you will discover the basics of mathematical notation that you may come across when reading descriptions of techniques in machine learning. After completing this tutorial, you will know:

- Notation for arithmetic including variations of multiplication, exponents, roots and logarithms.
- Notation for sequences and sets including indexing, summation and set membership.
- 5 Techniques you can use to get help if you are struggling with mathematical notation.

Let's get started.

D.1 Tutorial Overview

This tutorial is divided into 7 parts; they are:

1. The Frustration with Math Notation
2. Arithmetic Notation
3. Greek Alphabet
4. Sequence Notation
5. Set Notation
6. Other Notation
7. Tips for Getting More Help

D.2 The Frustration with Math Notation

You will encounter mathematical notation when reading about machine learning algorithms. For example, notation may be used to:

- Describe an algorithm.
- Describe data preparation.
- Describe results.
- Describe a test harness.
- Describe implications.

These descriptions may be in research papers, textbooks, blog posts and elsewhere. Often the terms are well defined, but there are also mathematical notation norms that you may not be familiar with. All it takes is one term or one equation that you do not understand and your understanding of the entire method will be lost. I've suffered this problem myself many times and it is incredibly frustrating! In this tutorial we will review some basic mathematical notation that will help you when reading descriptions of machine learning methods.

D.3 Arithmetic Notation

In this section we will go over some less obvious notations for basic arithmetic as well as a few concepts you may have forgotten since school.

D.3.1 Simple Arithmetic

The notation for basic arithmetic is as you would write it. For example:

- Addition: $1 + 1 = 2$
- Subtraction: $2 - 1 = 1$
- Multiplication: $2 \times 2 = 4$
- Division: $\frac{2}{2} = 1$

Most mathematical operations have an sister operation that performs the inverse operation, for example subtraction is the inverse of addition and division is the inverse of multiplication.

D.3.2 Algebra

We often want to describe operations abstractly to separate them from specific data or specific implementations. For this reason we see heavy use of algebra, that is uppercase and/or lowercase letters or words to represents terms or concepts in mathematical notation. It is also common to use letters from the Greek alphabet. Each sub-field of math may have reserved letters, that is terms or letters that always mean the same thing. Nevertheless, algebraic terms should be defined as part of the description and if they are not, it may just be a poor description, not your fault.

D.3.3 Multiplication Notation

Multiplication is a common notation and has a few short hands. Often a little "x" ($\times$) or an asterisk "*" is used to represent multiplication:

$$c = a \times b \tag{D.1}$$

Or

$$c = a * b \tag{D.2}$$

You may see a dot notation used, for example:

$$c = a \cdot b \tag{D.3}$$

Alternately, you may see no operation and no white space separation between previously defined terms, for example:

$$c = ab \tag{D.4}$$

Which again is the same thing.

D.3.4 Exponents and Square Roots

An exponent is a number raised to a power. The notation is written as the original number or the base with a second number or the exponent shown as a superscript, for example:

$$2^3 \tag{D.5}$$

Which would be calculated as 2 multiplied by itself 3 times or cubing:

$$2 \times 2 \times 2 = 8 \tag{D.6}$$

A number raised to the power to is said to be it's square

$$2^2 = 2 \times 2 = 4 \tag{D.7}$$

The square of a number can be inverted by calculating the square root. This is shown using the notation of a number and with a tick above $\sqrt{x}$.

$$\sqrt{4} = 2 \tag{D.8}$$

Here, we know the result and the exponent and we wish to find the base. In fact, the root operation can be used to inverse any exponent, it just so happens that the default square root assumes an exponent of 2, represented by a subscript 2 in front of the square root tick. For example, we can invert the cubing of a number by taking the cube root:

$$2^3 = 8 \tag{D.9}$$

$$\sqrt[3]{6} = 2 \tag{D.10}$$

D.3.5 Logarithms and e

When we raise 10 to an integer exponent we often call this an order of magnitude.

$$10^2 = 10 \times 10 \tag{D.11}$$

Another way to reverse this operation is by calculating the logarithm of the result 100 assuming a base of 10, in notation this is written as `log10()`.

$$log10(100) = 2 \tag{D.12}$$

Here, we know the result and the base and wish to find the exponent. This allows us to move up and down orders of magnitude very easily. Taking the logarithm assuming the base of 2 is also commonly used, given the use of binary arithmetic used in computers. For example:

$$2^6 = 64 \tag{D.13}$$

$$log2(64) = 6 \tag{D.14}$$

Another popular logarithm is to assume the natural base called e. The e is reserved and is a special number or a constant called Euler's number (pronounced *oy-ler*) that refers to a value with practically infinite precision.

$$e = 2.71828\cdots \tag{D.15}$$

Raising e to a power is called a natural exponential function:

$$e^2 = 7.38905\cdots \tag{D.16}$$

It can be inverted using the natural logarithm which is denoted as `ln()`:

$$ln(7.38905\cdots) = 2 \tag{D.17}$$

Without going into detail, the natural exponent and natural logarithm prove useful throughout mathematics to abstractly describe the continuous growth of some systems, e.g. systems that grow exponentially such as compound interest.

D.4 Greek Alphabet

Greek letters are used throughout mathematical notation for variables, constants, functions and more. For example in statistics we talk about the mean using the lowercase Greek letter mu (μ), and the standard deviation as the lowercase Greek letter sigma (σ). In linear regression we talk about the coefficients as the lowercase letter beta (β). And so on. It is useful to know all of the uppercase and lowercase Greek letters and how to pronounce them. When I was a grad student, I printed the Greek alphabet and stuck it on my computer monitor so that I could memorize it. A useful trick! Below is the full Greek alphabet.

Greek letters															
Name	TeX	HTML	Name	TeX	HTML	Name	TeX	HTML	Name	TeX	HTML	Name	TeX	HTML
Alpha	$\mathrm{A}\,\alpha$	Α α	Digamma	$\digamma\ \mathrm{F}$	Ϝ ϝ	Kappa	$\mathrm{K}\,\kappa\ \varkappa$	Κ κ ϰ	Omicron	$\mathrm{O}\,o$	Ο ο	Upsilon	$\Upsilon\,\upsilon$	Υ υ
Beta	$\mathrm{B}\,\beta$	Β β	Zeta	$\mathrm{Z}\,\zeta$	Ζ ζ	Lambda	$\Lambda\,\lambda$	Λ λ	Pi	$\Pi\,\pi\,\varpi$	Π π ϖ	Phi	$\Phi\,\phi\,\varphi$	Φ ϕ φ
Gamma	$\Gamma\,\gamma$	Γ γ	Eta	$\mathrm{H}\,\eta$	Η η	Mu	$\mathrm{M}\,\mu$	Μ μ	Rho	$\mathrm{P}\,\rho\,\varrho$	Ρ ρ ϱ	Chi	$\mathrm{X}\,\chi$	Χ χ
Delta	$\Delta\,\delta$	Δ δ	Theta	$\Theta\,\theta\,\vartheta$	Θ θ ϑ	Nu	$\mathrm{N}\,\nu$	Ν ν	Sigma	$\Sigma\,\sigma\,\varsigma$	Σ σ ς	Psi	$\Psi\,\psi$	Ψ ψ
Epsilon	$\mathrm{E}\,\epsilon\,\varepsilon$	Ε ε ε	Iota	$\mathrm{I}\,\iota$	Ι ι	Xi	$\Xi\,\xi$	Ξ ξ	Tau	$\mathrm{T}\,\tau$	Τ τ	Omega	$\Omega\,\omega$	Ω ω

Figure D.1: Greek Alphabet, Taken from Wikipedia.

The Wikipedia page titled *Greek letters used in mathematics, science, and engineering*[1] is also a useful guide as it lists common uses for each Greek letter in different sub-fields of math and science.

D.5 Sequence Notation

Machine learning notation often describes an operation on a sequence. A sequence may be an array of data or a list of terms.

D.5.1 Indexing

A key to reading notation for sequences is the notation of indexing elements in the sequence. Often the notation will specify the beginning and end of the sequence, such as 1 to n, where n will be the extent or length of the sequence. Items in the sequence are index by a variable such as i, j, k as a subscript. This is just like array notation. For example a_i is the ith element of the sequence a. If the sequence is two dimensional, two indices may be used, for example: $b_{i,j}$ is the $(i,j)^{th}$ element of the sequence b.

D.5.2 Sequence Operations

Mathematical operations can be performed over a sequence. Two operations are performed on sequences so often that they have their own shorthand, the sum and the multiplication.

Sequence Summation

The sum over a sequence is denoted as the uppercase Greek letter sigma (Σ). It is specified with the variable and start of the sequence summation below the sigma (e.g. $i = 1$) and the index of the end of the summation above the sigma (e.g. n).

$$\sum_{i=1}^{n} a_i \tag{D.18}$$

This is the sum of the sequence a starting at element 1 to element n.

[1] https://en.wikipedia.org/wiki/Greek_letters_used_in_mathematics,_science,_and_engineering

Sequence Multiplication

The multiplication over a sequence is denoted as the uppercase Greek letter pi (Π). It is specified in the same way as the sequence summation with the beginning and end of the operation below and above the letter respectively.

$$\prod_{i=1}^{n} a_i \tag{D.19}$$

This is the product of the sequence a starting at element 1 to element n.

D.6 Set Notation

A set is a group of unique items. We may see set notation used when defining terms in machine learning.

D.6.1 Set of Numbers

A common set you may see is a set of numbers, such as a term defined as being within the set of integers or the set of real numbers. Some common sets of numbers you may see include:

- Set of all natural numbers: $\mathbb{N}$
- Set of all integers: $\mathbb{Z}$
- Set of all real numbers: $\mathbb{R}$

There are other sets, see *Special sets on Wikipedia*[2]. We often talk about real-values or real numbers when defining terms rather than floating point values, which are really discrete creations for operations in computers.

D.6.2 Set Membership

It is common to see set membership in definitions of terms. Set membership is denoted as a symbol that looks like an uppercase "E" ($\in$).

$$a \in \mathbb{R} \tag{D.20}$$

Which means a is defined as being a member of the set $\mathbb{R}$ or the set of real numbers. There is also a host of set operations, two common set operations include:

- Union, or aggregation: $A \cup B$
- Intersection, or overlap: $A \cap B$

Learn more about sets on Wikipedia[3].

[2] https://en.wikipedia.org/wiki/Set_(mathematics)#Special_sets
[3] https://en.wikipedia.org/wiki/Set_(mathematics)

D.7 Other Notation

There is other notation that you may come across. I try to lay some of it out in this section. It is common to define a method in the abstract and then define it again as a specific implementation with separate notation. For example, if we are estimating a variable x we may represent it using a notation that modifies the x, for example:

- x-bar ($\bar{x}$)
- x-prime ($\grave{x}$)
- x-hat ($\hat{x}$)
- x-tilde ($\tilde{x}$)

The same notation may have different meaning in a different context, such as use on different objects or sub-fields of mathematics. For example, a common point of confusion is $|x|$, which, depending on context can mean:

- $|x|$: The absolute or positive value of x.
- $|x|$: The length of the vector x.
- $|x|$: The cardinality of the set x.

This tutorial only covered the basics of mathematical notation. There are some subfields of mathematics that are more relevant to machine learning and should be reviewed in more detail. They are:

- Linear Algebra.
- Statistics.
- Probability.
- Calculus.

And perhaps a little bit of multivariate analysis and information theory.

D.8 Tips for Getting More Help

This section lists some tips that you can use when you are struggling with mathematical notation in machine learning.

D.8.1 Think About the Author

People wrote the paper or book you are reading. People that can make mistakes, make omissions, and even make things confusing because they don't fully understand what they are writing. Relax the constraints of the notation you are reading slightly and think about the intent of the author. What are they trying to get across? Perhaps you can even contact the author via email, Twitter, Facebook, Linked-in, etc, and seek clarification. Remember that academics want other people to understand and use their work (mostly).

D.8.2 Check Wikipedia

Wikipedia has lists of notation which can help narrow down on the meaning or intent of the notation you are reading. Two places I recommend you start are:

- List of mathematical symbols on Wikipedia.
  https://en.wikipedia.org/wiki/List_of_mathematical_symbols
- Greek letters used in mathematics, science, and engineering on Wikipedia.
  https://en.wikipedia.org/wiki/Greek_letters_used_in_mathematics,_science,_and_engineering

D.8.3 Sketch in Code

Mathematical operations are just functions on data. Map everything you're reading to pseudocode with variables, for-loops and more. You might want to use a scripting language as you go along with small arrays of contrived data or even an Excel spreadsheet. As your reading and understanding of the technique improves, your code-sketch of the technique will make more sense and at the end you will have a mini prototype to play with.

I never used to take much stock in this approach until I saw an academic sketch out a very complex paper in a few lines of Matlab with some contrived data. It knocked my socks off because I believed the system had to be coded completely and run with a *real* dataset and that the only option was to get the original code and data. I was very wrong. Also, looking back, the guy was gifted. I now use this method all the time and sketch techniques in Python.

D.8.4 Seek Alternatives

There is a trick I use when I'm trying to understand a new technique. I find and read all the papers that reference the paper I'm reading with the new technique. Reading other academics interpretation and re-explanation of the technique can often clarify my misunderstandings in the original description. Not always though. Sometimes it can muddy the waters and introduce misleading explanations or new notation. But more often than not, it helps. After circling back to the original paper and re-reading it, I can often find cases where subsequent papers have actually made errors and misinterpretations of the original method.

D.8.5 Post a Question

There are places online where people love to explain math to others. Seriously! Consider taking a screen shot of the notation you are struggling with, write out the full reference or link to it and put it and your area of misunderstanding to a question and answer site. Two great places to start are:

- Mathematics Stack Exchange.
  https://math.stackexchange.com/
- Cross Validated.
  https://stats.stackexchange.com/

D.9 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

- Section 0.1. Reading Mathematics, *Vector Calculus, Linear Algebra, and Differential Forms*, 2009.
  http://amzn.to/2qarp8L
- The Language and Grammar of Mathematics, Timothy Gowers.
  http://assets.press.princeton.edu/chapters/gowers/gowers_I_2.pdf
- Understanding Mathematics, a guide, Peter Alfeld.
  http://www.math.utah.edu/~pa/math.html

D.10 Summary

In this tutorial, you discovered the basics of mathematical notation that you may come across when reading descriptions of techniques in machine learning. Specifically, you learned:

- Notation for arithmetic including variations of multiplication, exponents, roots and logarithms.
- Notation for sequences and sets including indexing, summation and set membership.
- 5 Techniques you can use to get help if you are struggling with mathematical notation.

Part VIII

Conclusions

How Far You Have Come

You made it. Well done. Take a moment and look back at how far you have come. You now know:

- What linear algebra is and why it is relevant and important to machine learning.
- How to create, index, and generally manipulate data in NumPy arrays.
- What a vector is and how to perform vector arithmetic and calculate vector norms.
- What a matrix is and how to perform matrix arithmetic, including matrix multiplication.
- A suite of types of matrices, their properties, and advanced operations involving matrices.
- What a tensor is and how to perform basic tensor arithmetic.
- Matrix factorization methods, including the eigendecomposition and singular-value decomposition.
- How to calculate and interpret basic statistics using the tools of linear algebra.
- How to implement methods using the tools of linear algebra, such as principal component analysis and linear least squares regression.

Don't make light of this. You have come a long way in a short amount of time. You have developed the important and valuable foundational skills in linear algebra. You can now confidently:

- Read the linear algebra mathematics in machine learning papers.
- Implement the linear algebra descriptions of machine learning algorithms.
- Describe your machine learning models using the notation and operations of linear algebra.

The sky's the limit.

Thank You!

I want to take a moment and sincerely thank you for letting me help you start your linear algebra journey. I hope you keep learning and have fun as you continue to master machine learning.

Jason Brownlee
2018